

INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO

ESCOM

Trabajo Terminal

"Navegación con asistencia auditiva en tiempo real para personas con visión reducida, mediante el uso de lentes adaptados e inteligencia artificial, para el manejo de siniestros"

2026-B045

Presentan

Thania Paola Esquivel García

José Carlos Espinosa Martínez

Carrera

Ingeniería en Inteligencia Artifical

Director

M. en C. Cristhian Alejandro Ávila Sánchez



Mayo, 2026

INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO
SUBDIRECCIÓN ACADÉMICA

No de TT: 2026-B045 Fecha de presentación de TT-I: Junio, 2026

Documento técnico

"Navegación con asistencia auditiva en tiempo real para personas con visión reducida, mediante el uso de lentes adaptados e inteligencia artificial, para el manejo de siniestros"

Presentan
Thania Paola Esquivel García[1]
José Carlos Espinosa Martínez [2]

Director
M. en C. Ávila Sánchez Cristhian Alejandro

RESUMEN

La vista es uno de los sentidos que nos ayudan a interpretar el entorno e interactuar de manera efectiva con él. Sin embargo, personas que carecen de manera total o parcial de este sentido se enfrentan a desafíos diarios que limitan su independencia y acceso a diversas áreas cotidianas. La navegación para personas cuya visión es reducida o presentan discapacidad visual es esencial para su autonomía y bienestar. Si bien existen métodos tradicionales de asistencia, estos suelen tener un enfoque que no se ajusta a las necesidades de viajar a diversos puntos. Este trabajo terminal tiene como objetivo utilizar la inteligencia artificial para proporcionar asistencia auditiva en tiempo real a personas con visión reducida, mediante el uso de lentes adaptados, así como un sistema de navegación. El sistema permite la identificación del entorno para guiar al usuario de manera adecuada, promoviendo una mayor independencia y seguridad para los usuarios.

Palabras clave: Discapacidad visual, Inteligencia artificial, Reconocimiento de voz, Visión artificial, Navegación asistida.

[1]tesquivelg2000@alumno.ipn.mx
[2]jespinosam1900@alumno.ipn.mx

Advertencia

“Este documento contiene información desarrollada por la Escuela Superior de Cómputo del Instituto Politécnico Nacional, a partir de datos y documentos con derecho de propiedad y por lo tanto, su uso quedará restringido a las aplicaciones que explícitamente se convengan.”

La aplicación no convenida exime a la escuela su responsabilidad técnica y da lugar a las consecuencias legales que para tal efecto se determinen.

Información adicional sobre este reporte técnico podrá obtenerse en:

La Subdirección Académica de la Escuela Superior de Cómputo del Instituto Politécnico Nacional, situada en Av. Juan de Dios Bátiz s/n Teléfono: 57296000, extensión 52000.

Agradecimientos

Resumen

La vista es uno de los sentidos que nos ayudan a interpretar el entorno y a interactuar de manera efectiva con él. Sin embargo, personas que carecen de manera total o parcial de este sentido se enfrentan a desafíos diarios que limitan su independencia y acceso a diversas áreas cotidianas. La navegación para personas cuya visión es reducida o presentan discapacidad visual es esencial para su autonomía y bienestar. Si bien existen métodos tradicionales de asistencia, estos suelen tener un enfoque que no se ajuste a las necesidades de viajar a diversos puntos. Este trabajo terminal tiene como objetivo utilizar la inteligencia artificial para proporcionar asistencia auditiva en tiempo real a personas con visión reducida, mediante el uso de lentes adaptados, así como un sistema de navegación. El sistema permite la identificación del entorno para guiar al usuario de manera adecuada, promoviendo una mayor independencia y seguridad para los usuarios.

Para lograrlo, se implementa un sistema inteligente capaz de percibir y analizar el entorno, identificando elementos relevantes como obstáculos, señales y personas. Esta información se integra con datos provenientes de distintos sensores para construir una representación dinámica del espacio circundante. A partir de ello, el sistema es capaz de estimar la ubicación del usuario y generar rutas de navegación que se ajustan continuamente ante cambios en el entorno. Asimismo, se incorpora un módulo de análisis contextual que permite emitir alertas oportunas, junto con mecanismos de interacción natural que facilitan la comunicación con el usuario y hacen al sistema fácil de comprender. Finalmente, la información procesada se transforma en indicaciones claras y accesibles, conformando una solución integral de asistencia a la navegación.

Abstract

Sight is one of the senses that helps us interpret our surroundings and interact effectively with them. However, people who are totally or partially blind face daily challenges that limit their independence and access to various areas of everyday life, such as mobility. Navigation for people with low vision or visual impairments is essential for their autonomy and well-being. While traditional methods of assistance exist, they lack an accessible and efficient approach that meets the needs of traveling to various locations. This final project aims to use artificial intelligence to provide real-time auditory assistance to people with low vision, through the use of adapted lenses, an optimized navigation system, and location assistance in emergency situations. The system allows for the identification of the environment to guide the user appropriately, promoting greater independence and safety for users.

To achieve this, an object detection model based on convolutional neural networks is implemented to identify obstacles, traffic signs, and people in the user's surroundings in real time. This information is combined with depth sensor data through sensor fusion, generating a dynamic map of the environment using simultaneous localization and mapping techniques. Route planning is performed using the A* algorithm, supported by digital mapping, allowing for the recalculation of paths in the event of unforeseen obstacles. A probabilistic inference module evaluates the context to issue relevant alerts, while a voice recognition system enables natural user interaction with the device. All processed information is translated into clear and timely audio instructions, forming a comprehensive assisted navigation system.

Índice

1. Introducción **12**

1.1. Planteamiento del problema 15

1.1.1. Preguntas de investigación 16

1.2. Objetivos 18

1.2.1. Objetivo General 18

1.2.2. Objetivos Específicos 18

1.3. Hipótesis 18

1.3.1. Hipótesis específicas 19

1.4. Justificación 20

1.5. Alcances y limitaciones 22

1.5.1. Alcances 22

1.5.2. Limitaciones 22

2. Estado del arte: Trabajos previos **24**

2.1. Trabajos academicos 24

2.2. Análisis crítico y limitaciones de los trabajos existentes 25

2.3. Tendencias actuales en tecnologías de asistencia visual 26

3. Marco teórico **28**

3.1. Inteligencia Artificial (IA) y rutas 28
3.1.1. Definición y fundamentos de la inteligencia artificial 28
3.1.2. IA aplicada a sistemas de navegación 29
3.1.3. Representación del entorno para planificación de rutas 30
3.1.4. Algoritmos de búsqueda de caminos 32
3.1.5. Planificación de rutas seguras mediante inteligencia artificial 33
3.1.6. Replanificación dinámica de rutas en tiempo real 33
3.1.7. Ray tracing aplicado a la navegación y detección de obstáculos 34
3.1.8. Sistemas de navegación inteligente para personas con discapacidad visual . 34
3.1.9. Modelos de entrenamiento de la IA 34
3.2. Visión por computadora 36
3.2.1. Fundamentos de la visión por computadora 36
3.2.2. Procesamiento de imágenes digitales 40
3.2.3. Detección y reconocimiento de objetos 42
3.2.4. Aplicaciones de la visión por computadora en asistencia a la movilidad . . 47
3.3. Sensores y cámaras para percepción del entorno 49
3.3.1. Sensores para detección de obstáculos 50
3.3.2. Cámaras para visión por computadora 54
3.3.3. Integración en gafas inteligentes 57
3.3.4. Análisis de viabilidad de plataformas de gafas inteligentes 58
3.4. Discapacidad visual 59
3.4.1. Concepto de discapacidad visual 59
3.4.2. Tipos de discapacidad visual 62
3.4.3. Impacto de la discapacidad visual en la movilidad 64

3.4.4. Tecnologías de asistencia para personas con discapacidad visual 66

3.5. Navegación digital en mapas 67

3.5.1. Sistemas de navegación digital 68

3.5.2. Sistemas de información geográfica (SIG) 70

3.5.3. Tecnologías utilizadas en navegación digital 71

3.5.4. Aplicaciones de la navegación digital en movilidad urbana 73

3.6. Ray casting 74

3.6.1. Fundamentos del ray casting 75

3.6.2. Diferencias entre ray casting y ray tracing 76

3.6.3. Aplicaciones del ray casting 77

3.6.4. Uso del ray casting en mapeo y navegación en tiempo real 79

4. Marco metodológico **81**

4.1. Enfoque metodológico 81

4.2. El sistema 82

4.3. Metodología de desarrollo 82

4.4. Descripción de módulos 83

4.4.1. Módulos de percepción 83

4.4.2. Módulos de representación 83

4.4.3. Módulo de navegación y rutas 84

4.4.4. Módulos de interacción 84

4.5. Integración del sistema 84

4.6. Pruebas y validación 84

5. Requerimientos del sistema **86**

5.1. Requerimientos Funcionales 86

5.2. Requerimientos No Funcionales 89

6. Análisis y diseño **93**

6.1. Casos de Uso 93

6.1.1. Diagrama de casos de uso 95

6.2. Diagramas de flujo 97

6.2.1. Flujo de percepción 100

6.2.2. Flujo de Navegación 102

6.2.3. Flujo de la Interacción 104

6.3. Arquitectura 106

6.3.1. Diagrama de arquitectura 107

6.4. Análisis de Requerimientos 108

6.4.1. Módulos de percepción 108

6.4.2. Módulos de representación 111

6.4.3. Módulo de navegación y rutas 114

6.4.4. Módulos de interacción 116

6.5. Selección y justificación de hardware principal 118

6.5.1. Comparación de sensores de distancia para dispositivos portables 118

6.5.2. Comparación de módulos de cámara para sistemas embebidos 119

6.5.3. Resumen de decisiones tecnológicas 120

7. Implementación de los módulos **122**

7.1. Módulos de percepción 122

7.1.1. Procesamiento de imágenes 122

7.1.2. Módulo de visión 123

7.1.3. Módulo sensorial 125

7.2. Módulos de representación 127
7.2.1. SLAM simplificado 127
7.2.2. Ray Casting 130
7.3. Módulo de navegación y rutas 131
7.3.1. Modelo de navegación y rutas 131
7.3.2. Submódulo de dirección 134
7.3.3. Submódulo de MAPS 136
7.3.4. Módulo de alerta de obstáculos 137
7.4. Módulos de interacción 139
7.4.1. Módulo de reconocimiento de voz 139
7.4.2. Módulo de PLN 141
7.4.3. Módulo de salida 143
7.5. Algoritmo Main: Integración del sistema 145
7.6. Electrónica 147
7.6.1. Costos y viabilidad 150

8. Trabajo a futuro y Conclusiones **153**

8.1. Conclusiones de Trabajo Terminal I 154

Referencias **155**

A. Pseudocódigos auxiliares **163**

A.1. Pseudocódigo del módulo de Ray Casting 163
A.2. Pseudocódigo integrado de los módulos de representación 164
A.3. Pseudocódigo integrado de los módulos de representación 164
A.4. Pseudocódigos Voz 166

Capítulo 1

Introducción

La visión desempeña un papel crucial en la vida diaria, al ser el principal medio a través del cual percibimos e interactuamos con nuestro entorno. La visión, el más dominante de nuestros sentidos, desempeña un papel fundamental en cada faceta y etapa de nuestras vidas [1]. Según la OMS, al menos 2.2 mil millones de personas en el mundo tienen alguna deficiencia visual, y de estas, al menos mil millones podrían haberse evitado o aún no se han tratado [1].

Las personas con discapacidad visual enfrentan desafíos diarios que limitan su independencia y acceso a diversas actividades cotidianas como la movilidad y la navegación, incluso ponen en riesgo su seguridad en entornos desconocidos. Esta discapacidad, puede variar en grado, desde dificultades para ver objetos cercanos, objetos lejanos, hasta la ceguera total. Las barreras físicas y sociales que encuentran a menudo les impiden desenvolverse con facilidad en entornos urbanos, lo que puede afectar su bienestar emocional y psicológico.

La creación de entornos accesibles es fundamental para mejorar la calidad de vida de las personas con discapacidad visual. Las calles y las aceras pueden ser difíciles de transitar sin la ayuda de un bastón o un perro guía. Además, las personas con discapacidad visual pueden tener dificultades para subir escaleras o usar medios de transporte público que no están adaptados para ellos [2].

Se ha señalado que la movilidad y la orientación son preocupaciones constantes para las personas ciegas [3]. Desplazarse por entornos desconocidos o altamente transitados, identificar obstáculos o acceder al transporte público representa un gran desafío. Las calles concurridas, los edificios sin señalización adecuada o las superficies irregulares suelen convertirse en barreras que limitan su independencia [3].

En los últimos años, se ha observado un interés creciente por el desarrollo de tecnologías inclusivas que favorezcan la autonomía de personas con discapacidad visual. Este proyecto surge a partir de esa necesidad, con el objetivo de explorar nuevas formas de asistencia en tiempo real que

permitan mejorar la orientación espacial y asegurar la seguridad de los usuarios en caso de estar expuestos a siniestros como lo son los incendios.

Se han propuesto diversas soluciones para asistir la movilidad de personas con discapacidad visual. A nivel nacional, el IPN reportó un prototipo portátil basado en sensores ultrasónicos que coloca dispositivos en el cuello, rodillas y abdomen para detectar obstáculos y avisar al usuario mediante vibraciones y alertas de voz [4]. Asimismo, estudiantes del IPN desarrollaron la "Primera Guía Robot" (un pequeño robot asistente) que integra cámaras 3D, algoritmos de IA y retroalimentación auditiva/vibratoria para guiar al usuario en tiempo real evitando obstáculos [5].

- Lentes de Meta Ray-Ban: Las gafas inteligentes Ray-Ban Meta. Aunque inicialmente no fueron diseñadas específicamente para personas con discapacidad visual, sus capacidades impulsadas por inteligencia artificial ofrecen funcionalidades de asistencia. La función de Meta AI con cámara permite que las gafas describan el entorno en tiempo real, identifiquen objetos, traduzcan texto y respondan a preguntas basadas en lo que ven. [6]

- Lentes inteligentes con IA: En Cinvestav Guadalajara crearon el prototipo SmartOjo (2019), unas gafas equipadas con doble cámara, parlantes y una microcomputadora que procesan el entorno en tiempo real. SmartOjo permite navegar, evadir obstáculos móviles, reconocer rostros, leer texto y billetes, entregando instrucciones auditivas al usuario [7]. Similarmente, la UPC coordinó el proyecto BCNMob (2024) usando las gafas Biel Glasses (tecnología 5G + realidad extendida): estas gafas 3D detectan y evitan obstáculos urbanos y ofrecen al usuario indicaciones visuales y sonoras personalizadas mediante IA y posicionamiento Galileo [8]. Recientemente, el reto MIT Solve 2023 presentó el proyecto Eye in AI, un par de gafas inteligentes que mediante redes neuronales describen el entorno en voz alta (reconocimiento de objetos, rostros, texto a voz) [9].

- Aplicaciones móviles: Existen apps de navegación accesible por voz. Como, Lazarillo App, en México, es un GPS accesible que da instrucciones de audio en tiempo real al desplazarse por la ciudad o en interiores, permitiendo explorar entornos de forma más segura [10]. También Google Lookout y Seeing AI de Microsoft ofrecen reconocimiento de objetos y lectura de texto mediante IA, aunque su enfoque principal es asistencia visual, no rutas específicas.

- Tecnologías emergentes: Se han investigado sistemas con realidad virtual/aumentada y feedback háptico. El estudio "Nature Machine Intelligence 2025" describió un sistema chino compuesto por gafas con cámara 3D, un 'A-skin' vibratorio en la muñeca y audio óseo. Este dispositivo emite señales hápticas direccionales y sonidos envolventes discretos para guiar al usuario. [11]

- Dispositivos comerciales: En el mercado existen ayudas consolidadas, aunque no todas enfocadas en navegación. Por ejemplo, OrCam MyEye y eSight son gafas inteligentes que

identifican texto y objetos para ciegos; el bastón “smart” WeWALK incorpora sensores ultrasónicos, GPS y conectividad móvil para dar alertas de voz. Sin embargo, muchos de estos productos no proveen navegación guiada activa.

La mayoría de los trabajos académicos se centran en prototipos experimentales con visión computacional y retroalimentación sonora para evitar obstáculos y mejorar la orientación. En este trabajo terminal se propone integrar lentes adaptado con IA y un sistema de rutas en tiempo real.

Para desarrollar el proyecto, resulta esencial examinar la problemática que da origen a esta investigación. Comprender sus causas y consecuencias permite justificar la pertinencia del proyecto y definir el enfoque adoptado. En la siguiente sección se aborda el planteamiento del problema.

1.1. Planteamiento del problema

La movilidad independiente es un elemento fundamental para la autonomía y calidad de vida de las personas. Sin embargo, para las personas con discapacidad visual o visión reducida, desplazarse de manera segura en entornos urbanos representa un desafío constante debido a la dificultad para percibir obstáculos, identificar rutas adecuadas y orientarse en espacios desconocidos. De acuerdo con la Organización Mundial de la Salud, más de 2,200 millones de personas en el mundo presentan algún grado de discapacidad visual, lo que evidencia la magnitud de esta problemática a nivel global (OMS, 2023). [1]

En el caso de las personas con visión reducida o ceguera, la movilidad cotidiana suele depender de herramientas tradicionales como el bastón blanco, el perro guía o la asistencia de otras personas. Aunque el bastón es una herramienta ampliamente utilizada, su alcance se limita a detectar obstáculos cercanos al usuario y no proporciona información anticipada sobre el entorno ni orientación en rutas específicas. Esto puede generar situaciones de riesgo al transitar por calles, cruzar avenidas o desplazarse en espacios con múltiples obstáculos.

En los últimos años se han desarrollado diversas tecnologías orientadas a mejorar la accesibilidad para personas con discapacidad visual, como aplicaciones móviles de navegación, sistemas de reconocimiento de voz y herramientas de lectura de texto mediante cámaras.

Paralelamente, el avance de la inteligencia artificial ha permitido el desarrollo de sistemas capaces de interpretar información visual mediante técnicas de visión por computadora y aprendizaje profundo. Modelos de detección de objetos basados en redes neuronales convolucionales, como YOLO (You Only Look Once), han demostrado ser capaces de identificar múltiples objetos dentro de una imagen en tiempo real con altos niveles de precisión. Estas tecnologías han sido aplicadas en áreas como la conducción autónoma, la robótica y la vigilancia inteligente, lo que demuestra su potencial para analizar entornos complejos.

Asimismo, el desarrollo de dispositivos de bajo costo y alto rendimiento, como la Raspberry Pi, ha facilitado la implementación de sistemas inteligentes portátiles capaces de procesar información en tiempo real mediante cámaras y sensores. La integración de estos dispositivos con tecnologías de reconocimiento de voz y navegación mediante GPS abre la posibilidad de desarrollar soluciones accesibles que permitan a los usuarios interactuar con el sistema mediante comandos de voz y recibir indicaciones auditivas sobre su entorno.

A pesar de estos avances tecnológicos, todavía existe una brecha significativa entre las capacidades actuales de la inteligencia artificial y su aplicación práctica en sistemas de asistencia para personas con discapacidad visual. Muchas de las soluciones existentes no integran de manera eficiente la detección de obstáculos, la navegación geográfica y la interacción auditiva dentro de un

mismo sistema portátil que pueda ser utilizado en la vida cotidiana.

En este contexto surge la necesidad de desarrollar un sistema que combine tecnologías de visión artificial, inteligencia artificial, sensores y navegación GPS para asistir a personas con visión reducida durante su desplazamiento. Un sistema de este tipo podría capturar imágenes del entorno mediante una cámara, analizar la información visual utilizando algoritmos de detección de objetos, identificar obstáculos relevantes y proporcionar indicaciones auditivas al usuario sobre la presencia de objetos o rutas a seguir.

Además, la incorporación de tecnologías de reconocimiento de voz permitiría que el usuario interactúe con el sistema de manera natural, solicitando indicaciones de navegación o información sobre su entorno sin necesidad de utilizar interfaces visuales. La integración de estos elementos en un prototipo basado en hardware portátil podría contribuir a mejorar la seguridad, autonomía y calidad de vida de las personas con discapacidad visual.

Por lo tanto, el problema central que aborda esta investigación se relaciona con la falta de sistemas accesibles e inteligentes que permitan a las personas con visión reducida desplazarse de manera más segura y autónoma mediante el uso de tecnologías de inteligencia artificial, visión por computadora y navegación auditiva en tiempo real.

Ante esta problemática, el presente trabajo propone el desarrollo de un sistema de navegación asistida basado en inteligencia artificial que permita detectar obstáculos mediante visión artificial, procesar la información en tiempo real y proporcionar indicaciones auditivas al usuario para facilitar su desplazamiento en diferentes entornos.

1.1.1. Preguntas de investigación

Con base en la problemática identificada sobre las dificultades de movilidad y orientación que enfrentan las personas con discapacidad visual, surgen diversas interrogantes que orientan el desarrollo de la presente investigación. A nivel mundial, millones de personas presentan algún grado de pérdida de visión, y en México se estima que alrededor de 11 millones de personas viven con ceguera o discapacidad visual, lo que refleja la relevancia social de desarrollar tecnologías de apoyo que faciliten su desplazamiento y autonomía.
Asimismo, en el país se han registrado aproximadamente 8.8 millones de personas con algún tipo de discapacidad, lo que representa cerca del $7{,}2\,\%$ de la población, evidenciando la necesidad de implementar soluciones tecnológicas inclusivas que contribuyan a mejorar la calidad de vida de este sector.

- ¿Qué es la inteligencia artificial?

- ¿Qué es la vista?
- ¿Qué es discapacidad visual?
- ¿Qué es un sistema de navegación?
- ¿Qué es persona con discapacidad visual?
- ¿Qué es asistencia?
- ¿Qué es GPS?
- ¿Qué es geolocalización?
- ¿Qué es ruta?
- ¿De qué manera puede desarrollarse un sistema de navegación asistida que utilice inteligencia artificial, visión por computadora y guía auditiva para mejorar la movilidad de personas con visión reducida en entornos urbanos?
- ¿Cómo puede utilizarse la visión por computadora para detectar obstáculos y objetos relevantes en el entorno de una persona con discapacidad visual?
- ¿Qué modelos de inteligencia artificial permiten realizar detección de objetos en tiempo real con suficiente precisión para aplicaciones de asistencia a la movilidad?
- ¿Cómo puede integrarse un sistema de navegación basado en GPS para proporcionar rutas e indicaciones auditivas al usuario?
- ¿De qué manera el reconocimiento de voz puede facilitar la interacción entre el usuario y el sistema sin depender de interfaces visuales?
- ¿Qué impacto tiene la implementación de un prototipo basado en inteligencia artificial en la seguridad y autonomía de las personas con visión reducida durante su desplazamiento?

1.2. Objetivos

1.2.1. Objetivo General

Desarrollar e implementar un sistema de navegación asistida en tiempo real para personas con discapacidad visual, basado en el uso de lentes adaptados con sensores de proximidad, visión artificial, trazado de rayos y tecnologías de inteligencia artificial, que permita identificar obstáculos, interpretar el entorno urbano y generar instrucciones auditivas contextuales.

1.2.2. Objetivos Específicos

1. Diseñar un modelo de visión por computadora que sea capaz de detectar y segmentar los objetos, identifique y delimite con precisión los objetos relevantes en el entorno de manera óptima y cuya precisión sea mayor al $80\,\%$.

2. Reconstruir el escenario utilizado tecnologías relacionadas al raytraicing para obtener una mayor fidelidad

3. Diseñar y desarrollar un modelo de navegación adecuado que permita al usuario desplazarse con autonomía y cuya medición del terreno sea en un $90\,\%$ segura, minimizando el riesgo de colisiones y caídas.

4. Modelar un sistema de interacción por voz que sea capaz de reconocer ciertas indicaciones del usuario y leer en voz alta las indicaciones del sistema.

5. Desarrollar un modulo de lenguaje natural que traduzca y permita el manejo adecuado de indicaciones de manera cotidiana y durante las situaciones de emergencia.

1.3. Hipótesis

Si se desarrolla un sistema de navegación asistida que integre visión por computadora, inteligencia artificial y guía auditiva en tiempo real, entonces será posible mejorar la seguridad y autonomía en el desplazamiento de personas con visión reducida al identificar obstáculos y proporcionar indicaciones de navegación durante su movilidad.

1.3.1. Hipótesis específicas

- La utilización de modelos de detección de objetos basados en aprendizaje profundo permitirá identificar obstáculos relevantes en el entorno con suficiente precisión para aplicaciones de asistencia a la movilidad.
- La integración de tecnologías de geolocalización y navegación permitirá generar rutas accesibles y proporcionar indicaciones auditivas al usuario durante su desplazamiento.
- La interacción mediante reconocimiento de voz facilitará el uso del sistema por parte de personas con discapacidad visual al eliminar la dependencia de interfaces visuales.

1.4. Justificación

Las personas con discapacidad visual pueden enfrentar problemas de salud mental relacionados con la adaptación, la ansiedad o la depresión debido a su condición [3].

Se ha observado que la capacidad de adaptación en la movilidad independiente de las personas con discapacidad visual varía significativamente según el momento en que se presenta esta condición.

Aquellos que nacen con discapacidad visual, ya sea total o parcial, suelen desarrollar desde temprana edad estrategias sensoriales y cognitivas que les permiten desenvolverse con mayor autonomía en su entorno. Aun así, enfrentan limitaciones en su desarrollo y orientación espacial; especialmente al interactuar con entornos nuevos o no accesibles, donde, al presenciar alguna situación de emergencia, como los incendios, se encuentran vulnerables debido al desconocimiento del entorno. Por ello, es fundamental contemplar estas situaciones.

Este contraste evidencia la necesidad de soluciones tecnológicas que no solo brinden asistencia en la navegación cotidiana, sino que también se adapten a los diferentes niveles de experiencia, aprendizaje y necesidades específicas de cada persona.

La propuesta de este trabajo terminal plantea una solución que integra dispositivos físicos (lentes inteligentes), visión por computadora, trazado de rayos, sensores de proximidad e inteligencia artificial en un sistema de navegación guiada por voz, especialmente pensado para la movilidad urbana de personas con discapacidad visual. A diferencia de soluciones existentes como aplicaciones móviles o bastones inteligentes, este proyecto propone una interacción manos libres, contextualizada y modular, adaptable a las necesidades individuales del usuario y con potencial de expansión hacia otros sectores vulnerables.

En cuanto a la vinculación con los usuarios potenciales, este proyecto está dirigido principalmente a personas con discapacidad visual, tanto aquellas con ceguera total como parcial, quienes constituyen un sector históricamente desatendido en términos de movilidad autónoma urbana y manejo de riesgos.

Adicionalmente, el sistema podría beneficiar a personas con dificultades temporales de movilidad.

La complejidad técnica del proyecto es significativa, ya que involucra el diseño y ensamblado de hardware portátil, el desarrollo de algoritmos de inteligencia artificial, la fusión de datos sensoriales, reconocimiento de objetos tanto estáticos como en movimiento, el trazado de rayos para la

reconstrucción de escenarios, la integración con APIs de navegación y la creación de una interfaz auditiva accesible.

Todo esto exige la aplicación conjunta de conocimientos adquiridos en asignaturas como visión artificial, aprendizaje automático, procesamiento de lenguaje natural, reconocimiento de voz, desarrollo de sistemas embebidos y experiencia de usuario.

Finalmente, el trabajo propuesto es viable dentro del periodo establecido para el desarrollo del TT, considerando que su implementación será gradual y modular siguiendo el método científico, permitiendo validar cada componente de forma independiente. Los recursos requeridos, como sensores de bajo costo, plataformas de desarrollo, software libre para entrenamiento de modelos y acceso a APIs gratuitas o educativas, hacen posible el cumplimiento del proyecto con un presupuesto limitado pero funcional.

1.5. Alcances y limitaciones

1.5.1. Alcances

El presente proyecto tiene como objetivo el desarrollo de un sistema de navegación asistida que permita apoyar la movilidad de personas con visión reducida mediante el uso de tecnologías de inteligencia artificial, visión por computadora y guía auditiva.

- Diseñar e implementar un sistema capaz de capturar información del entorno mediante una cámara integrada en un dispositivo portátil.
- Desarrollar un módulo de procesamiento basado en inteligencia artificial que permita detectar objetos y obstáculos relevantes utilizando técnicas de visión por computadora y aprendizaje profundo.
- Integrar un sistema de navegación basado en geolocalización que permita calcular rutas y proporcionar indicaciones al usuario mediante audio.
- Implementar un sistema de interacción mediante reconocimiento de voz que permita al usuario comunicarse con el sistema a través de comandos verbales.
- Construir un prototipo funcional que integre hardware y software para demostrar la viabilidad del sistema propuesto.
- Evaluar el desempeño del sistema mediante pruebas que permitan analizar su capacidad de detección de objetos, tiempo de respuesta y funcionamiento general.
- Establecer un protocolo de interacción accesible que minimice la curva de aprendizaje del sistema, considerando que los usuarios potenciales pueden no tener experiencia previa con tecnología de asistencia basada en inteligencia artificial.
- El sistema ha sido diseñado exclusivamente para la navegación peatonal, por lo que su uso en cualquier otro modo de desplazamiento queda fuera del alcance del presente proyecto.

1.5.2. Limitaciones

Es importante reconocer que, a pesar de los alcances planteados, el desarrollo del presente proyecto presenta algunas limitaciones que deben considerarse durante su implementación.

- El desempeño del sistema puede verse afectado por condiciones del entorno como la iluminación, la presencia de múltiples objetos o escenarios altamente dinámicos, lo cual puede influir en la precisión de los algoritmos de visión por computadora.
- La capacidad de procesamiento del hardware utilizado en el prototipo, ya que dispositivos portátiles como microcomputadoras o sistemas embebidos tienen recursos computacionales limitados en comparación con servidores de alto rendimiento.
- La precisión de los sistemas de navegación basados en GPS puede variar dependiendo de factores como la ubicación geográfica, la presencia de edificios altos o interferencias en la señal satelital.
- Es importante considerar que el sistema se desarrollará como un prototipo de investigación, por lo que su implementación y pruebas se realizarán en escenarios controlados y no necesariamente en todas las condiciones posibles de uso en la vida cotidiana.
- El proyecto se enfocará principalmente en el desarrollo e integración de las tecnologías necesarias para el funcionamiento del sistema, por lo que aspectos relacionados con la producción a gran escala, certificaciones médicas o implementación comercial quedan fuera del alcance de la presente investigación.
- El sistema constituye una herramienta de asistencia complementaria y no un sustituto de los apoyos tradicionales de movilidad. El uso del sistema no elimina la necesidad del bastón blanco ni del perro guía, los cuales siguen siendo elementos indispensables para la navegación segura e independiente. El sistema está diseñado para operar en conjunto con dichos apoyos, proporcionando una capa adicional de información auditiva sobre el entorno.
- El sistema no contempla la detección de obstáculos a nivel del suelo con la misma precisión que un bastón blanco (como desniveles, bordillos o superficies irregulares), por lo que su capacidad de alerta en esos escenarios es limitada y no debe considerarse suficiente para reemplazar dicha herramienta.
- La fiabilidad del reconocimiento de voz puede verse reducida en entornos con alto nivel de ruido ambiental, como zonas de tráfico intenso, espacios concurridos o áreas industriales, lo que podría limitar la capacidad del usuario para interactuar con el sistema mediante comandos verbales en dichos contextos.
- El sistema identificará y notificará al usuario cuando las condiciones del entorno no sean aptas para su funcionamiento, tales como iluminación insuficiente, pérdida de señal o contextos no contemplados en su diseño como el interior de vehículos en movimiento. En dichos casos, el sistema suspenderá la emisión de instrucciones priorizando la seguridad del usuario sobre la continuidad del servicio.

Capítulo 2

Estado del arte: Trabajos previos

2.1. Trabajos academicos

La mayoría de los trabajos académicos se centran en prototipos experimentales que emplean visión computacional y retroalimentación sonora con el objetivo de evitar obstáculos y mejorar la orientación de personas con discapacidad visual. No obstante, la propuesta desarrollada en este trabajo se diferencia al integrar un lente adaptado con inteligencia artificial junto con un sistema de rutas optimizado en tiempo real.

En la Tabla 2.1 se presenta una comparación general entre diversas soluciones existentes y la propuesta planteada.

Tabla 2.1: Comparativa de trabajos previos en asistencia para navegación de personas con discapacidad visual

Sistema / Proyecto	**Dispositivo**	**Función principal**	**Observaciones**
Cinvestav (2019) [7]	Lentes inteligentes con IA	Reconocimiento del entorno y asistencia visual	Desarrollo académico enfocado en guía para personas con discapacidad visual
IPN (2020) [4]	Arnés con sensores ultrasónicos	Alerta vibratoria/sonora de obstáculos	Prototipo nacional; no integrado en gafas ni navegación con rutas
IPN "Guía Robot" [5]	Robot móvil con cámaras e IA	Guía autónomo similar a perro guía	Prototipo avanzado desarrollado por estudiantes del IPN
BCNMob (UPC) [8]	Gafas Biel 3D (5G + XR)	Detección de escaleras, pasos y señales	Pilotaje en Barcelona; navegación precisa basada en IA
Lazarillo App [10]	Smartphone (GPS + audio)	Navegación GPS con guía por voz	Aplicación móvil basada en comunidad de mapas
Eye in AI (MIT) [9]	Gafas con IA	Descripción auditiva del entorno	Prototipo de bajo costo enfocado en accesibilidad
VR con gafas (China) [11]	Gafas 3D + sistema háptico	Guía mediante retroalimentación sensorial	Investigación basada en realidad virtual aplicada a navegación

2.2. Análisis crítico y limitaciones de los trabajos existentes

A partir de la revisión de los trabajos presentados, se observa que existen múltiples propuestas orientadas a la asistencia de personas con discapacidad visual.

En primer lugar, varios sistemas se enfocan únicamente en la detección de obstáculos mediante sensores o visión artificial, sin considerar la planificación de rutas completas o la integración con sistemas de navegación global. Esto limita su utilidad en entornos reales, donde no solo es necesario evitar obstáculos, sino también llegar a un destino específico de manera eficiente [4].

Por otro lado, las aplicaciones móviles como Lazarillo dependen fuertemente de la información de mapas digitales y del GPS, lo cual reduce su efectividad en entornos interiores o en zonas con baja precisión de posicionamiento [10]. Asimismo, estos sistemas no suelen incorporar mecanismos avanzados de percepción del entorno inmediato.

En el caso de dispositivos más avanzados, como gafas inteligentes o sistemas basados en realidad aumentada, se observa un alto nivel de complejidad tecnológica y, en muchos casos, costos elevados o dependencia de infraestructura adicional, lo que dificulta su adopción masiva [8].

Por otro lado, se observa que la interacción con el usuario es un área donde aún se puede profundizar, especialmente en la creación de instrucciones claras y accesibles para personas con discapacidad visual, un elemento clave para lograr una usabilidad real.

De igual forma, el campo presenta el reto de integrar la percepción, el mapeo y la navegación de manera más fluida. Actualmente, es común que estos componentes se desarrollen de forma independiente, por lo que existe una valiosa oportunidad de unificarlos para responder mejor a las exigencias de los entornos reales.

Estas limitaciones evidencian la necesidad de desarrollar un sistema integral que combine visión artificial, mapeo dinámico del entorno, planificación de rutas y comunicación efectiva con el usuario, como se propone en el presente trabajo.

2.3. Tendencias actuales en tecnologías de asistencia visual

En los últimos años, el desarrollo de tecnologías de asistencia para personas con discapacidad visual ha evolucionado significativamente gracias al avance en inteligencia artificial, visión por computadora y sistemas embebidos.

Una de las principales tendencias es el uso de modelos de aprendizaje profundo para la detección y reconocimiento de objetos en tiempo real. Modelos como YOLO han permitido implementar sistemas capaces de identificar elementos del entorno con alta precisión y baja latencia, facilitando su integración en dispositivos portátiles [12].

Asimismo, se ha incrementado el uso de dispositivos inteligentes como gafas con cámara integrada, los cuales permiten una interacción más natural y manos libres con el entorno. Estos dispositivos buscan reemplazar o complementar herramientas tradicionales como el bastón blanco [7].

Otra tendencia relevante es la incorporación de sistemas de navegación híbridos, que combinan información global (GPS y mapas digitales) con percepción local mediante sensores y visión

artificial, lo cual permite una navegación más robusta en entornos dinámicos y no estructurados.

Finalmente, se observa un creciente interés en la integración de interfaces de voz y procesamiento de lenguaje natural, con el objetivo de mejorar la interacción humano-máquina y proporcionar instrucciones más intuitivas y accesibles para los usuarios [9].

Estas tendencias reflejan un enfoque hacia sistemas más integrales, autónomos y centrados en el usuario, lo cual motiva el desarrollo de la propuesta presentada en este trabajo.

Capítulo 3

Marco teórico

3.1. Inteligencia Artificial (IA) y rutas

3.1.1. Definición y fundamentos de la inteligencia artificial

La inteligencia artificial es una rama de la informática que desarrolla sistemas capaces de realizar tareas que normalmente requieren inteligencia humana, como aprender, razonar, tomar decisiones y resolver problemas complejos. En el contexto de la robótica y los sistemas de navegación, la IA permite que una máquina analice su entorno, interprete información sensorial y determine acciones adecuadas para alcanzar un objetivo [13].

Concepto de IA

De acuerdo con IBM, la inteligencia artificial es una tecnología que permite a las computadoras y máquinas simular capacidades cognitivas humanas, tales como el aprendizaje, la comprensión, la resolución de problemas, la toma de decisiones y la creatividad. Estos sistemas pueden analizar datos, aprender de la experiencia y actuar de manera autónoma para resolver tareas complejas [14].

Google define la inteligencia artificial como el conjunto de técnicas que permiten a las máquinas imitar la inteligencia humana, especialmente en tareas como el reconocimiento de voz, la interpretación de imágenes, el procesamiento del lenguaje natural y la toma de decisiones automatizada. Estas tecnologías permiten que los sistemas aprendan a partir de datos y mejoren su desempeño con el tiempo [15].

La NASA define la inteligencia artificial como el campo de la informática orientado a crear

sistemas capaces de realizar tareas que, de ser ejecutadas por humanos, requerirían inteligencia [16].

Investigadores de Stanford University describen la inteligencia artificial como una disciplina de la informática que busca desarrollar sistemas capaces de percibir su entorno, razonar sobre la información y actuar de manera inteligente para alcanzar objetivos específicos. Este enfoque resalta el papel de la percepción, el aprendizaje y la toma de decisiones dentro de los sistemas inteligentes.[17]

El término inteligencia artificial fue acuñado en 1956 por el científico John McCarthy, quien la definió como la ciencia y la ingeniería de crear máquinas inteligentes, especialmente programas informáticos capaces de realizar tareas que requieren inteligencia humana.[18]

Mientras tanto, Amazon Web Services menciona que es una tecnología con capacidades de resolución de problemas similares a las de las personas [19].

La Association for the Advancement of Artificial Intelligence establece que la inteligencia artificial es el campo de estudio dedicado a desarrollar sistemas computacionales que pueden realizar tareas que normalmente requieren inteligencia humana, como el aprendizaje, la planificación, el razonamiento, la percepción y la comprensión del lenguaje.[20]

Por último, la European Commission define la inteligencia artificial como sistemas diseñados por humanos que, dado un objetivo complejo, perciben su entorno mediante la adquisición de datos, interpretan la información y toman decisiones o acciones que influyen en dicho entorno.[21]

3.1.2. IA aplicada a sistemas de navegación

La aplicación de inteligencia artificial en sistemas de navegación ha permitido el desarrollo de robots móviles, drones y vehículos autónomos capaces de desplazarse de manera independiente en entornos complejos. Estos sistemas utilizan algoritmos que analizan continuamente el entorno para determinar la mejor trayectoria hacia un destino.

En robótica móvil, la navegación autónoma consiste en determinar cómo desplazarse desde una posición inicial hasta un punto objetivo dentro de un entorno que puede contener obstáculos o cambios dinámicos. Este proceso incluye la localización del robot, la construcción de mapas del entorno y la planificación de trayectorias seguras [22].

Los sistemas de navegación pueden dividirse en dos tipos principales: navegación global y navegación local. La navegación global se basa en un conocimiento previo del entorno y utiliza algoritmos de planificación de rutas para determinar una trayectoria óptima. Por otro lado, la na-

vegación local se basa en sensores que permiten detectar obstáculos en tiempo real y ajustar la trayectoria durante el desplazamiento [23].

En aplicaciones de asistencia para personas con discapacidad visual, estos sistemas pueden combinar cámaras y algoritmos de visión por computadora para identificar obstáculos, calcular rutas seguras y proporcionar indicaciones auditivas o táctiles al usuario.

Componentes de un sistema inteligente de navegación

Un sistema de navegación inteligente está compuesto por varios módulos que trabajan conjuntamente para permitir el desplazamiento autónomo dentro de un entorno.

- La percepción consiste en obtener información del entorno mediante sensores. En muchos sistemas modernos se utilizan cámaras o sensores láser que permiten detectar obstáculos, identificar superficies transitables y analizar la geometría del entorno [22].
- Modelado del entorno: una vez obtenida la información sensorial, el sistema genera una representación del entorno que puede utilizarse para la planificación de rutas. Esta representación suele adoptar la forma de un mapa digital que contiene información sobre obstáculos y espacios transitables.

  El proceso mediante el cual un robot construye o utiliza mapas del entorno se conoce como mapeo robótico. Este proceso permite que un sistema autónomo pueda localizarse dentro del entorno y planificar trayectorias adecuadas [24].
- La planificación de rutas consiste en calcular la trayectoria que conecta la posición actual con el destino. Este proceso implica evaluar diferentes rutas posibles y seleccionar aquella que optimice ciertos criterios como distancia o tiempo [25].
- Toma de decisiones en tiempo real: el sistema debe ser capaz de reaccionar ante cambios en el entorno. Para ello, los sistemas de navegación autónoma utilizan sensores que detectan obstáculos y permiten recalcular la ruta durante el desplazamiento [23].

3.1.3. Representación del entorno para planificación de rutas

Antes de calcular una trayectoria, el sistema necesita contar con una representación computacional del entorno que permita analizar posibles rutas.

Modelado del entorno mediante grafos

Una de las representaciones más utilizadas en planificación de rutas es el modelo basado en grafos. En este modelo, el entorno se representa como un conjunto de nodos conectados mediante aristas. Cada nodo representa una posición dentro del espacio y cada arista representa un camino posible entre dos posiciones.

A cada arista se le asigna un costo que puede representar la distancia, el tiempo o el riesgo asociado con desplazarse entre dos nodos. Este enfoque permite aplicar algoritmos de búsqueda que determinan la ruta óptima entre un nodo inicial y un nodo destino [25].

Mapas de ocupación

Otra técnica común es el uso de mapas de ocupación. En este modelo, el espacio se divide en una cuadrícula de celdas donde cada celda puede clasificarse como libre o ocupada por un obstáculo [24].

Este tipo de representación permite que un robot identifique las zonas transitables dentro de un entorno y evite regiones donde existan obstáculos.

Los mapas de ocupación son ampliamente utilizados en robótica móvil debido a su simplicidad y eficiencia para representar entornos complejos [24].

Mapas dinámicos del entorno

En muchos entornos reales, la configuración del espacio puede cambiar constantemente. Por ejemplo, pueden aparecer personas, vehículos o nuevos obstáculos que modifiquen la ruta inicialmente calculada.

Por esta razón, los sistemas de navegación modernos utilizan mapas dinámicos que se actualizan continuamente mediante la información obtenida por sensores [23].

Estos mapas permiten que el sistema adapte su trayectoria cuando se detectan cambios en el entorno.

3.1.4. Algoritmos de búsqueda de caminos

Los algoritmos de búsqueda de caminos permiten determinar la ruta óptima entre dos puntos dentro de un grafo o mapa [25].

Problema del camino más corto

El problema del camino más corto consiste en encontrar una secuencia de nodos que conecte un punto inicial con un destino minimizando un costo determinado, como la distancia o el tiempo de desplazamiento.

Este problema es uno de los más estudiados dentro de la inteligencia artificial y la robótica, ya que constituye la base para la planificación de trayectorias en sistemas autónomos [25].

Algoritmo de Dijkstra

El algoritmo de Dijkstra es uno de los métodos clásicos utilizados para resolver el problema del camino más corto [26]. Este algoritmo explora el grafo desde el nodo inicial y calcula la distancia mínima hacia todos los demás nodos.

El algoritmo garantiza encontrar la ruta óptima siempre que los costos de las aristas sean positivos. Debido a esta característica, ha sido ampliamente utilizado en sistemas de navegación y aplicaciones como sistemas de mapas digitales [26].

Sin embargo, su principal limitación es que puede resultar computacionalmente costoso en entornos grandes, ya que evalúa múltiples rutas antes de encontrar la mejor solución [25].

Algoritmo A*

El algoritmo A* es una mejora del algoritmo de Dijkstra que utiliza una heurística para estimar la distancia restante hasta el destino [27].

La función de evaluación del algoritmo se expresa como:

$$f(n) = g(n) + h(n)$$

donde $g(n)$ representa el costo desde el nodo inicial hasta el nodo actual y $h(n)$ representa una

estimación del costo desde el nodo actual hasta el destino.

El uso de esta heurística permite reducir el número de nodos explorados y acelerar el proceso de búsqueda. Por esta razón, el algoritmo A* es ampliamente utilizado en robótica, videojuegos y sistemas de navegación autónoma [27, 25].

3.1.5. Planificación de rutas seguras mediante inteligencia artificial

Los algoritmos tradicionales de planificación de rutas se enfocan principalmente en optimizar la distancia o el tiempo de desplazamiento. Sin embargo, en aplicaciones de movilidad asistida es necesario considerar factores adicionales relacionados con la seguridad [22].

La inteligencia artificial permite integrar múltiples variables en el proceso de toma de decisiones, como la presencia de obstáculos, la accesibilidad del terreno o el nivel de riesgo asociado con una determinada ruta [13].

En sistemas avanzados de navegación, la IA puede analizar información proveniente de sensores para evaluar el entorno y determinar cuál de las rutas posibles ofrece las mejores condiciones de seguridad [23].

3.1.6. Replanificación dinámica de rutas en tiempo real

En entornos reales, los obstáculos pueden aparecer de forma inesperada. Por ejemplo, una persona puede cruzar el camino o un objeto puede bloquear la trayectoria inicialmente calculada.

Por esta razón, los sistemas de navegación deben ser capaces de recalcular la ruta durante el desplazamiento.

La replanificación dinámica consiste en actualizar la trayectoria cuando se detectan cambios en el entorno. Este proceso implica detectar el obstáculo, actualizar el mapa del entorno y calcular una nueva ruta alternativa hacia el destino.

Los sistemas modernos de navegación utilizan sensores y algoritmos de planificación que permiten realizar este proceso de forma continua mientras el sistema se encuentra en movimiento [23].

3.1.7. Ray tracing aplicado a la navegación y detección de obstáculos

El ray tracing es una técnica originalmente utilizada en gráficos por computadora para simular el comportamiento de la luz mediante el trazado de rayos en un entorno tridimensional [25].

En sistemas de navegación, este principio puede utilizarse para analizar posibles trayectorias y detectar intersecciones entre el camino proyectado y los obstáculos detectados por sensores.

Mediante el uso de rayos virtuales proyectados desde la posición del agente, el sistema puede evaluar diferentes direcciones y determinar cuáles de ellas están libres de obstáculos.

Este enfoque permite detectar colisiones potenciales y seleccionar trayectorias alternativas en tiempo real [23].

3.1.8. Sistemas de navegación inteligente para personas con discapacidad visual

Los sistemas de navegación asistida utilizan inteligencia artificial y sensores para proporcionar información sobre el entorno a personas con discapacidad visual.

Estos sistemas pueden detectar obstáculos, identificar rutas transitables y proporcionar instrucciones auditivas o hápticas para guiar al usuario hacia su destino [22].

La combinación de visión por computadora, algoritmos de planificación de rutas y sensores de proximidad permite desarrollar sistemas capaces de mejorar la movilidad y la seguridad durante el desplazamiento [23, 25].

3.1.9. Modelos de entrenamiento de la IA

Los modelos modernos de inteligencia artificial se basan principalmente en enfoques estadísticos y probabilísticos. En particular, los modelos de aprendizaje automático (machine learning) permiten que los sistemas informáticos aprendan patrones a partir de datos sin necesidad de ser programados explícitamente con reglas detalladas para cada tarea.

A diferencia de los sistemas tradicionales basados en reglas —propios de la inteligencia artificial simbólica—, los modelos de aprendizaje automático utilizan algoritmos matemáticos capaces de identificar patrones en los datos y generar predicciones o decisiones a partir de la experiencia obtenida durante el proceso de entrenamiento. Este enfoque permite que los sistemas mejoren su

desempeño conforme se exponen a mayores volúmenes de información y ejemplos de entrenamiento [28].

De acuerdo con IBM, el aprendizaje automático es un subconjunto de la inteligencia artificial que se centra en el desarrollo de algoritmos capaces de aprender patrones a partir de datos de entrenamiento y posteriormente realizar inferencias o predicciones sobre nuevos datos sin instrucciones explícitas programadas previamente [28].

Según IBM, las técnicas de aprendizaje automático pueden clasificarse en cinco categorías principales: aprendizaje supervisado, aprendizaje no supervisado, aprendizaje autosupervisado, aprendizaje por refuerzo y aprendizaje semisupervisado [29].

- **Aprendizaje supervisado**

  El aprendizaje supervisado consiste en entrenar un modelo utilizando un conjunto de datos previamente etiquetado, en el cual cada instancia de entrada está asociada con una salida conocida. Durante el entrenamiento, el algoritmo aprende la relación entre las variables de entrada y la salida esperada, con el objetivo de poder predecir correctamente nuevos casos.

  Este tipo de aprendizaje es ampliamente utilizado en problemas de clasificación y regresión, tales como reconocimiento de imágenes, detección de fraude o análisis predictivo. Algunos de los algoritmos más comunes incluyen regresión lineal, árboles de decisión, Naive Bayes, redes neuronales y bosques aleatorios [29].

- **Aprendizaje no supervisado**

  El aprendizaje no supervisado utiliza conjuntos de datos que no contienen etiquetas previamente definidas. En este enfoque, el algoritmo analiza los datos con el objetivo de identificar patrones, relaciones o estructuras ocultas dentro del conjunto de datos.

  Este tipo de aprendizaje es comúnmente utilizado para tareas de agrupamiento o clustering, reducción de dimensionalidad y análisis exploratorio de datos. Entre los algoritmos más utilizados se encuentran el algoritmo k-means, el clustering jerárquico y los modelos de mezcla gaussiana [29].

- **Aprendizaje autosupervisado**

  El aprendizaje autosupervisado es una técnica en la cual los modelos se entrenan utilizando datos sin etiquetar, pero generando automáticamente sus propias etiquetas a partir de los datos disponibles. En este enfoque, el modelo aprende a predecir una parte de los datos a partir de otra parte de los mismos.

  Este tipo de aprendizaje se ha vuelto especialmente relevante en áreas como la visión por computadora y el procesamiento del lenguaje natural, donde la generación manual de grandes conjuntos de datos etiquetados puede resultar costosa o inviable [29].

- **Aprendizaje por refuerzo**

  El aprendizaje por refuerzo es un enfoque en el cual un agente aprende a tomar decisiones mediante la interacción con su entorno. Durante el proceso de entrenamiento, el agente realiza acciones y recibe retroalimentación en forma de recompensas o penalizaciones.

  El objetivo del algoritmo es aprender una política de acción que maximice la recompensa acumulada a lo largo del tiempo. Este enfoque se utiliza frecuentemente en áreas como robótica, videojuegos y sistemas de navegación autónoma [29].

- **Aprendizaje semisupervisado**

  El aprendizaje semisupervisado combina características del aprendizaje supervisado y del aprendizaje no supervisado. En este enfoque, el modelo se entrena utilizando una pequeña cantidad de datos etiquetados junto con un conjunto más grande de datos sin etiquetar.

  Los datos etiquetados sirven como guía para que el algoritmo pueda identificar patrones en el conjunto completo de datos, permitiendo mejorar el desempeño del modelo incluso cuando la disponibilidad de datos etiquetados es limitada [29].

3.2. Visión por computadora

3.2.1. Fundamentos de la visión por computadora

La visión por computadora constituye una de las áreas más importantes dentro del campo de la inteligencia artificial, ya que permite a los sistemas informáticos interpretar información visual proveniente del entorno físico. A través de técnicas de procesamiento digital de imágenes, aprendizaje automático y reconocimiento de patrones, los sistemas computacionales pueden analizar imágenes o secuencias de video para identificar objetos, comprender escenas y tomar decisiones basadas en dicha información visual.

El desarrollo de la visión por computadora ha evolucionado significativamente durante las últimas décadas gracias al avance de la capacidad de procesamiento de los computadores, la disponibilidad de grandes conjuntos de datos visuales y el progreso en algoritmos de aprendizaje profundo. Actualmente, esta tecnología se encuentra presente en múltiples aplicaciones, tales como sistemas de conducción autónoma, robótica industrial, vigilancia inteligente, diagnóstico médico asistido por computadora, realidad aumentada y sistemas de asistencia para personas con discapacidad visual.

Dentro de este contexto, la visión por computadora representa un elemento fundamental para los sistemas autónomos que requieren interactuar con el entorno físico. Mediante la captura y

análisis de imágenes, estos sistemas pueden percibir el ambiente que los rodea, identificar obstáculos, reconocer objetos relevantes y generar respuestas automáticas ante diferentes situaciones. Por esta razón, comprender los fundamentos teóricos y técnicos de la visión por computadora resulta esencial para el desarrollo de aplicaciones tecnológicas orientadas a la movilidad asistida y la navegación inteligente.

Definición de visión por computadora

La visión por computadora es una disciplina perteneciente al campo de la inteligencia artificial que tiene como objetivo permitir que las máquinas interpreten información visual proveniente del mundo real. Esta área combina conocimientos de informática, procesamiento digital de imágenes, aprendizaje automático y reconocimiento de patrones para desarrollar sistemas capaces de analizar imágenes y videos de manera automatizada.

De acuerdo con IBM, la visión por computadora es un subcampo de la inteligencia artificial que dota a las máquinas de la capacidad de procesar, analizar e interpretar entradas visuales como imágenes y videos [30]. Esto implica que los sistemas computacionales pueden adquirir imágenes mediante sensores o cámaras, procesarlas mediante algoritmos especializados y extraer información significativa que permita comprender la escena representada.

En términos generales, la visión por computadora puede entenderse como el conjunto de métodos y técnicas que permiten a los sistemas automatizados captar imágenes del entorno, analizarlas y generar información útil a partir de ellas [31]. Dicho proceso implica transformar datos visuales en conocimiento interpretable para la toma de decisiones.

Entre las principales características de la visión por computadora se encuentran:

- Permite a los sistemas informáticos capturar y procesar datos visuales como píxeles, colores, texturas y patrones, los cuales son analizados mediante algoritmos especializados para identificar estructuras relevantes dentro de la imagen [30][31].
- A diferencia de los procesos de edición o manipulación de imágenes, la visión por computadora no busca alterar la imagen original, sino analizarla para identificar su contenido, como objetos, acciones o escenarios presentes en la escena visual [30][32].
- Sus aplicaciones abarcan tanto el análisis de imágenes estáticas, como fotografías o imágenes médicas, como el análisis en tiempo real de secuencias de video, lo cual resulta fundamental en sistemas de vigilancia, robótica y conducción autónoma.

Gracias a estas capacidades, la visión por computadora permite que los sistemas tecnológicos

puedan percibir el entorno de una manera similar a la percepción visual humana, aunque utilizando métodos matemáticos y algoritmos computacionales para procesar la información.

Relación entre visión por computadora e inteligencia artificial

La visión por computadora se encuentra estrechamente relacionada con la inteligencia artificial, ya que utiliza técnicas de aprendizaje automático y aprendizaje profundo para interpretar información visual. En esencia, la visión por computadora puede considerarse una subdisciplina de la inteligencia artificial especializada en el análisis de datos visuales.

Los sistemas de visión computarizada emplean algoritmos de inteligencia artificial para identificar patrones dentro de las imágenes y reconocer objetos o estructuras presentes en ellas. Según la literatura especializada, los sistemas de visión computarizada suelen apoyar su funcionamiento en algoritmos de inteligencia artificial que permiten decodificar imágenes y reconocer formas, figuras y patrones [31].

Uno de los enfoques más utilizados en este campo es el aprendizaje profundo, particularmente mediante redes neuronales convolucionales (CNN). Estas redes están diseñadas específicamente para procesar datos visuales, ya que son capaces de aprender automáticamente características relevantes de las imágenes a partir de grandes conjuntos de datos etiquetados.

Entre los aspectos más importantes de la relación entre la visión por computadora y la inteligencia artificial se encuentran:

- La utilización de modelos de aprendizaje automático, como redes neuronales y algoritmos de clasificación, que permiten analizar imágenes y reconocer patrones complejos dentro de ellas [30][31].
- La dependencia de los avances en aprendizaje profundo y arquitecturas de redes neuronales, como las redes convolucionales (CNN) y los modelos basados en transformers visuales, los cuales permiten extraer características visuales de alto nivel [32][33].
- La capacidad de entrenar modelos utilizando grandes bases de datos de imágenes etiquetadas, lo que permite que los sistemas aprendan a reconocer diferentes categorías de objetos o escenas.

La visión por computadora constituye una aplicación directa de los principios de la inteligencia artificial aplicada al procesamiento de información visual. Gracias a estos algoritmos, los sistemas computacionales pueden interpretar imágenes con niveles de precisión cada vez más cercanos a la percepción humana.

Importancia de la visión por computadora en sistemas autónomos

La visión por computadora desempeña un papel fundamental en el desarrollo de sistemas autónomos, ya que proporciona la capacidad de percibir y comprender el entorno en el que dichos sistemas operan. En este sentido, la visión artificial funciona como los "ojos" de un sistema inteligente, permitiéndole obtener información visual necesaria para la toma de decisiones.

Uno de los ejemplos más representativos de esta aplicación es el caso de los vehículos autónomos. En estos sistemas, múltiples cámaras instaladas en el vehículo capturan continuamente imágenes del entorno. Estas imágenes son analizadas mediante algoritmos de visión por computadora que permiten identificar elementos importantes de la vía, como carriles, señales de tránsito, peatones y otros vehículos.

De acuerdo con Amazon Web Services, la tecnología de vehículos autónomos utiliza la visión artificial para reconocer imágenes en tiempo real y crear mapas tridimensionales del entorno, lo cual permite identificar usuarios de la carretera, señales de tráfico, peatones u obstáculos presentes en la vía [34].

Otro ámbito importante es el de los robots móviles e industriales. En este caso, los robots utilizan sistemas de visión artificial para comprender el entorno que los rodea y realizar tareas de manera autónoma. Según Ultralytics, los sistemas de visión en robótica funcionan como los ojos de un robot, procesando imágenes mediante detección de objetos y análisis de profundidad, lo que permite evaluar el entorno y tomar decisiones en tiempo real [35].

Gracias a estas capacidades, los robots pueden:

- Identificar objetos dentro de su entorno.
- Localizar elementos específicos que deben manipular.
- Evaluar distancias y evitar colisiones con obstáculos.
- Tomar decisiones automáticas durante la ejecución de tareas.

En entornos dinámicos y no estructurados, como fábricas automatizadas, almacenes logísticos o espacios urbanos, la visión por computadora resulta indispensable para garantizar la operación segura de sistemas autónomos. Sin esta tecnología, los sistemas robóticos no podrían interactuar de manera efectiva con el entorno ni adaptarse a cambios inesperados.

Por esta razón, la integración de cámaras y algoritmos de visión computacional constituye uno de los elementos clave para el desarrollo de tecnologías autónomas capaces de operar en el mundo real [34][35][35].

3.2.2. Procesamiento de imágenes digitales

El procesamiento de imágenes digitales constituye una etapa fundamental dentro de los sistemas de visión por computadora. Este proceso implica la manipulación y análisis de imágenes digitales mediante algoritmos computacionales con el objetivo de mejorar su calidad, extraer información relevante o preparar los datos visuales para etapas posteriores de análisis.

El procesamiento de imágenes se divide generalmente en varias fases, entre las que destacan la captura de imágenes, el preprocesamiento y la extracción de características visuales. Cada una de estas etapas cumple una función específica dentro del flujo de trabajo de un sistema de visión computacional.

Captura y adquisición de imágenes

El primer paso en cualquier sistema de visión por computadora consiste en obtener imágenes digitales del entorno mediante dispositivos de captura. Este proceso se conoce como adquisición de imágenes y se realiza utilizando cámaras o sensores capaces de transformar la información visual del mundo real en datos digitales.

Las cámaras utilizadas en sistemas de visión artificial generalmente emplean sensores CCD o CMOS, los cuales convierten la luz que incide sobre ellos en señales eléctricas que posteriormente son transformadas en datos digitales. Estos datos representan la imagen mediante una matriz de píxeles, donde cada píxel contiene información sobre la intensidad de luz o color.

Según la literatura especializada, la adquisición de imágenes es el primer paso en un sistema de visión artificial y consiste en capturar datos visuales mediante cámaras y sensores, siendo la calidad de estos datos un factor determinante para el rendimiento del sistema [36].

Entre los principales elementos involucrados en la adquisición de imágenes se encuentran:

- Sensores de imagen, como cámaras RGB tradicionales o sensores avanzados como RGB-D y LiDAR, que proporcionan información adicional sobre la profundidad del entorno [37].
- Parámetros técnicos como la resolución de la imagen, la frecuencia de captura de fotogramas (FPS) y el rango dinámico, los cuales influyen directamente en la calidad de los datos visuales obtenidos [36][38].
- Procesos de calibración de cámaras, necesarios para corregir distorsiones ópticas y garantizar mediciones precisas, especialmente en sistemas que utilizan múltiples cámaras o visión estéreo.

Una adecuada adquisición de imágenes resulta fundamental, ya que una mala calidad de imagen puede afectar negativamente todas las etapas posteriores del procesamiento visual.

Preprocesamiento de imágenes

Una vez capturada la imagen, es común que esta requiera ser procesada antes de aplicar algoritmos de análisis o reconocimiento. Esta etapa se conoce como preprocesamiento de imágenes y tiene como objetivo mejorar la calidad de los datos visuales, eliminar ruido o resaltar características importantes.

El preprocesamiento permite preparar la imagen para etapas posteriores del sistema de visión por computadora. Según UnitX Labs, el preprocesamiento de imágenes transforma los datos sin procesar en un formato adecuado para el análisis posterior, eliminando distorsiones y mejorando las características visuales relevantes [38].

Entre las técnicas más comunes de preprocesamiento se encuentran:

- Conversión a escala de grises, la cual reduce la complejidad de la imagen al eliminar la información de color y conservar únicamente los niveles de intensidad luminosa.
- Filtrado y suavizado mediante filtros como el gaussiano o el filtro de mediana, utilizados para reducir el ruido presente en la imagen.
- Ecualización de histograma, técnica que mejora el contraste de la imagen y permite resaltar detalles que podrían pasar desapercibidos.
- Binarización o umbralización, proceso que convierte la imagen en una representación binaria para separar objetos del fondo.
- Normalización de la imagen, que incluye procesos como redimensionamiento o ajuste de valores de intensidad para estandarizar las condiciones de iluminación.

Estas técnicas contribuyen a mejorar la calidad de la imagen y facilitan la posterior extracción de características visuales.

Extracción de características visuales

Después del preprocesamiento, la siguiente etapa consiste en identificar características relevantes dentro de la imagen. Este proceso se conoce como extracción de características y tiene como

objetivo representar la información visual mediante descriptores numéricos que faciliten el análisis computacional.

De acuerdo con la literatura, la extracción de características identifica y aísla los elementos clave presentes en una imagen para representar su contenido de manera más eficiente [39].

Entre las técnicas más utilizadas se encuentran:

- Detección de bordes mediante algoritmos como Sobel o Canny, los cuales identifican cambios abruptos de intensidad en la imagen para resaltar contornos de objetos [39].
- Detección de esquinas y puntos clave mediante algoritmos como Harris o FAST.
- Descriptores locales como SIFT, SURF, ORB o HOG, que generan representaciones numéricas de patrones visuales locales dentro de la imagen [39].
- Técnicas de reducción de dimensionalidad como PCA o autoencoders, utilizadas para reducir la cantidad de información sin perder las características más relevantes.

La extracción de características permite simplificar la representación de la imagen y facilita tareas posteriores como la detección de objetos, el reconocimiento de patrones o la clasificación de imágenes.

3.2.3. Detección y reconocimiento de objetos

La detección y reconocimiento de objetos constituye una de las tareas centrales dentro de la visión por computadora. Esta área se enfoca en identificar automáticamente los objetos presentes dentro de una imagen o secuencia de video, así como determinar su ubicación dentro del espacio visual.

El desarrollo de técnicas de detección de objetos ha permitido que los sistemas inteligentes comprendan escenas complejas del mundo real. Gracias a estas técnicas, las máquinas pueden reconocer diferentes tipos de elementos presentes en su entorno, tales como personas, vehículos, señales de tránsito, animales, productos industriales o cualquier objeto previamente aprendido durante el entrenamiento del modelo.

Estas capacidades son fundamentales en múltiples aplicaciones tecnológicas, incluyendo vehículos autónomos, robótica industrial, sistemas de vigilancia inteligente, control de calidad automatizado y sistemas de asistencia para personas con discapacidad visual. En todos estos casos, la detección de objetos permite transformar información visual en conocimiento útil para la toma de decisiones automatizadas.

Concepto de detección de objetos

La detección de objetos es un proceso dentro de la visión por computadora que consiste en identificar y localizar múltiples objetos dentro de una imagen o video. A diferencia de otras tareas como la clasificación de imágenes, la detección de objetos no solo determina qué objetos están presentes, sino también su ubicación exacta dentro de la imagen.

De acuerdo con la literatura especializada, la detección de objetos consiste en identificar y ubicar múltiples objetos dentro de una imagen o video, clasificándolos y señalando su posición mediante cajas delimitadoras [33].

Estas cajas delimitadoras, conocidas como bounding boxes, son rectángulos que rodean cada objeto detectado y que indican su posición dentro de la imagen. Cada una de estas cajas se define mediante coordenadas que especifican su ubicación y tamaño dentro del plano de la imagen.

Entre las características principales de la detección de objetos se encuentran:

- Localización espacial: cada objeto detectado se representa mediante una caja delimitadora que indica su posición dentro de la imagen utilizando coordenadas (x, y, ancho, alto) [33].
- Clasificación de objetos: además de ubicar el objeto, el sistema asigna una etiqueta o clase que identifica el tipo de objeto detectado, por ejemplo “persona”, “vehículo” o “semaforo”.
- Detección simultánea de múltiples objetos: los algoritmos modernos pueden identificar varios objetos diferentes dentro de una misma imagen o fotograma.
- Procesamiento en tiempo real: en muchas aplicaciones, como vehículos autónomos o robótica móvil, la detección de objetos debe realizarse en tiempo real para permitir una respuesta inmediata ante cambios en el entorno.

Por ejemplo, en un sistema de conducción autónoma, un algoritmo de detección de objetos puede analizar continuamente la escena frente al vehículo e identificar elementos como automóviles, peatones, ciclistas y señales de tránsito, lo cual permite tomar decisiones de conducción seguras [33][32].

Debido a su importancia en la interpretación de escenas visuales, la detección de objetos representa uno de los pilares fundamentales de los sistemas modernos de visión por computadora.

Técnicas tradicionales de detección de objetos

Antes del desarrollo de los métodos basados en aprendizaje profundo, los sistemas de detección de objetos se basaban principalmente en técnicas clásicas de procesamiento de imágenes y reconocimiento de patrones. Estos métodos utilizaban características diseñadas manualmente para identificar patrones visuales dentro de las imágenes.

Entre las técnicas tradicionales más utilizadas se encuentran las siguientes:

Cascadas de clasificadores Haar (Viola–Jones)

El algoritmo Viola–Jones es uno de los métodos más conocidos para la detección de objetos, especialmente para la detección de rostros humanos. Este enfoque utiliza características Haar, que representan diferencias de intensidad entre regiones adyacentes de la imagen. Mediante el entrenamiento de múltiples clasificadores en cascada, el sistema puede identificar patrones característicos asociados con rostros u otros objetos[40].

Este método fue ampliamente utilizado en aplicaciones de reconocimiento facial y sistemas de vigilancia debido a su eficiencia computacional y su capacidad de procesamiento en tiempo real.

HOG combinado con SVM

Otra técnica ampliamente utilizada en detección de objetos es la combinación del descriptor HOG (Histogram of Oriented Gradients) con clasificadores SVM (Support Vector Machines). Este enfoque fue utilizado principalmente en la detección de peatones dentro de imágenes.

El descriptor HOG analiza la distribución de los gradientes de intensidad dentro de pequeñas regiones de la imagen, generando una representación numérica que describe la forma del objeto. Posteriormente, un clasificador SVM determina si la región analizada corresponde a un objeto específico o al fondo de la imagen[41].

Segmentación de regiones y análisis de contornos

Otros enfoques tradicionales se basaban en la segmentación de imágenes mediante técnicas de detección de bordes o agrupación de regiones con características similares. A partir de estas regiones segmentadas, se analizaban propiedades geométricas como tamaño, forma o relación de aspecto para determinar si correspondían a objetos de interés.

Comparación mediante plantillas

El método de template matching o comparación de plantillas consiste en comparar regiones de la imagen con ejemplos previamente almacenados de un objeto específico. Este método evalúa la similitud entre la imagen observada y las plantillas conocidas[32].

Aunque este enfoque es relativamente sencillo, presenta limitaciones importantes cuando existen variaciones en la escala, rotación o iluminación de los objetos.

En general, las técnicas tradicionales dependían fuertemente de características diseñadas manualmente por expertos. Como resultado, su capacidad para adaptarse a diferentes condiciones visuales era limitada. Debido a estas limitaciones, estos métodos han sido gradualmente reemplazados por técnicas basadas en aprendizaje profundo que permiten aprender automáticamente las características relevantes a partir de grandes conjuntos de datos [32].

Detección de objetos basada en aprendizaje profundo

El avance más significativo en la detección de objetos se produjo con la introducción de técnicas de aprendizaje profundo, particularmente mediante redes neuronales convolucionales (CNN). Estas redes permiten aprender automáticamente representaciones visuales complejas directamente a partir de los datos, eliminando la necesidad de diseñar manualmente las características.

Las CNN están especialmente diseñadas para procesar imágenes, ya que utilizan filtros convolucionales que detectan patrones locales como bordes, texturas o formas dentro de la imagen. A medida que la información avanza a través de las capas de la red, estos patrones simples se combinan para formar representaciones más complejas que permiten reconocer objetos completos.

Entre las principales características de los métodos basados en aprendizaje profundo se encuentran:

- Aprendizaje automático de características visuales a partir de grandes conjuntos de datos.
- Mayor capacidad para manejar variaciones en iluminación, escala, orientación y oclusión de objetos.
- Mejor rendimiento en términos de precisión comparado con métodos tradicionales.
- Posibilidad de procesar imágenes completas en lugar de analizar pequeñas regiones de manera independiente.

De acuerdo con estudios recientes, los sistemas modernos de visión por computadora emplean modelos de aprendizaje profundo basados en redes neuronales convolucionales para reconocer patrones complejos dentro de las imágenes [32].

Gracias a estos avances, la detección de objetos ha alcanzado niveles de precisión y velocidad que permiten su implementación en aplicaciones del mundo real.

Modelos modernos de detección de objetos (YOLO, SSD, Faster R-CNN)

Los algoritmos modernos de detección de objetos basados en aprendizaje profundo pueden clasificarse principalmente en dos categorías: detectores de dos etapas y detectores de una sola etapa.

Detectores de dos etapas

Los detectores de dos etapas funcionan mediante un proceso dividido en dos fases principales. En la primera etapa se generan propuestas de regiones dentro de la imagen que podrían contener objetos. Posteriormente, en la segunda etapa, cada una de estas regiones es analizada por un clasificador para determinar si contiene un objeto y cuál es su categoría.

Entre los modelos más conocidos de este tipo se encuentran:

- R-CNN
- Fast R-CNN
- Faster R-CNN

El modelo Faster R-CNN introdujo mejoras importantes al integrar una red de propuestas de regiones (Region Proposal Network) que permite generar regiones candidatas de manera más eficiente [33].

Estos modelos suelen ofrecer una alta precisión en la detección de objetos, aunque generalmente requieren mayor tiempo de procesamiento.

Detectores de una sola etapa

Los detectores de una sola etapa realizan la detección de objetos en un único paso dentro de la red neuronal, lo que permite procesar imágenes con mayor velocidad. Estos modelos combinan la extracción de características y la predicción de objetos dentro de una sola arquitectura.

Entre los detectores más utilizados se encuentran:

- YOLO (You Only Look Once)
- SSD (Single Shot MultiBox Detector)

El modelo YOLO divide la imagen en una cuadrícula y predice simultáneamente las cajas delimitadoras y las clases de los objetos presentes en cada región de la imagen [33].

Por su parte, el modelo SSD utiliza múltiples mapas de características con diferentes resoluciones para detectar objetos de distintos tamaños.

Estos modelos son ampliamente utilizados en aplicaciones que requieren procesamiento en tiempo real, como vehículos autónomos, drones, robots móviles y sistemas de vigilancia inteligente.

3.2.4. Aplicaciones de la visión por computadora en asistencia a la movilidad

La visión por computadora tiene un papel fundamental en el desarrollo de tecnologías orientadas a mejorar la movilidad y la seguridad de las personas. Gracias a la capacidad de interpretar información visual del entorno, los sistemas inteligentes pueden identificar obstáculos, analizar rutas y proporcionar asistencia durante la navegación.

Las aplicaciones en asistencia a la movilidad abarcan diversos ámbitos, incluyendo robótica autónoma, transporte inteligente y tecnologías de apoyo para personas con discapacidad visual. En todos estos casos, la visión por computadora permite transformar información visual en señales que facilitan la toma de decisiones o la orientación espacial.

Sistemas de detección de obstáculos

Los sistemas de detección de obstáculos utilizan cámaras y sensores para analizar continuamente el entorno y detectar objetos que puedan interferir con el desplazamiento de un dispositivo o persona. Estos sistemas son ampliamente utilizados en robots móviles, vehículos autónomos y drones.

Diversos estudios destacan que las cámaras permiten a los robots identificar y responder a obstáculos en tiempo real, facilitando así la planificación de rutas seguras durante la navegación [42].

Un ejemplo importante son los robots móviles autónomos (AMR) utilizados en almacenes y centros logísticos. Estos robots utilizan sistemas de visión para detectar estanterías, trabajadores u otros robots dentro de su entorno de trabajo. Con base en esta información, pueden modificar su trayectoria para evitar colisiones y optimizar su desplazamiento.

De manera similar, los vehículos autónomos utilizan cámaras y algoritmos de visión por computadora para identificar automóviles, peatones, ciclistas y señales de tránsito dentro del entorno urbano. Según Amazon Web Services, estos sistemas analizan imágenes capturadas por cámaras y utilizan inteligencia artificial para identificar elementos relevantes dentro de la carretera [34].

Sistemas de asistencia para personas con discapacidad visual

La visión por computadora también ha sido utilizada para desarrollar tecnologías que ayudan a mejorar la movilidad de personas con discapacidad visual. Estas soluciones combinan sensores, cámaras y algoritmos de inteligencia artificial para interpretar el entorno y proporcionar información al usuario mediante señales auditivas o táctiles.

Entre las tecnologías más relevantes se encuentran:

- **Bastones inteligentes**
  Los bastones inteligentes incorporan sensores ultrasónicos, cámaras o sensores LiDAR para detectar obstáculos cercanos. Cuando el sistema identifica un objeto en el camino del usuario, se genera una alerta mediante vibraciones o señales sonoras que advierten sobre la presencia del obstáculo.

- **Dispositivos portátiles con cámara**
  Existen dispositivos portátiles equipados con cámaras que permiten analizar el entorno visual del usuario. Estos dispositivos pueden reconocer objetos, leer texto mediante reconocimiento óptico de caracteres (OCR) o identificar señales de tránsito.

- **Aplicaciones móviles de asistencia visual** Algunas aplicaciones móviles utilizan la cámara del teléfono inteligente para analizar el entorno y proporcionar información auditiva al usuario. Estas aplicaciones pueden describir objetos, leer textos o identificar productos mediante reconocimiento visual.

Investigaciones recientes señalan que los sistemas de navegación asistida pueden ayudar a las personas con discapacidad visual a evitar obstáculos y recibir indicaciones precisas para desplazarse hacia un destino [37].

Gracias a estas tecnologías, las personas con discapacidad visual pueden mejorar significativamente su independencia y seguridad durante la movilidad.

Uso de visión por computadora en dispositivos portátiles inteligentes

Los avances en miniaturización de hardware y procesamiento de inteligencia artificial han permitido que los algoritmos de visión por computadora puedan ejecutarse directamente en dispositivos portátiles. Esto ha dado lugar al desarrollo de dispositivos inteligentes capaces de analizar el entorno visual en tiempo real.

Entre estos dispositivos destacan:

- **Smartphones**
  Los teléfonos inteligentes actuales cuentan con cámaras de alta resolución y procesadores capaces de ejecutar redes neuronales optimizadas para visión por computadora. Esto permite desarrollar aplicaciones de asistencia visual que utilizan la cámara del dispositivo para analizar el entorno y proporcionar retroalimentación al usuario.

- **Smart glasses**
  Las gafas inteligentes incorporan cámaras frontales y sistemas de procesamiento que permiten analizar continuamente la escena visual frente al usuario. Estas gafas pueden proporcionar instrucciones auditivas que ayudan a orientar al usuario durante la navegación.

- **Dispositivos vestibles**

  Otros dispositivos portátiles incluyen cámaras integradas en cinturones, mochilas o bastones inteligentes. Estos sistemas pueden capturar imágenes del entorno y procesarlas mediante algoritmos de inteligencia artificial para generar señales auditivas o vibraciones que guíen al usuario.

En conjunto, estas tecnologías demuestran el potencial de la visión por computadora para mejorar la movilidad y la autonomía de las personas. La combinación de cámaras, algoritmos de detección de objetos y sistemas de retroalimentación sensorial permite desarrollar soluciones innovadoras que facilitan la navegación segura en entornos urbanos y cotidianos.

3.3. Sensores y cámaras para percepción del entorno

Los sistemas de navegación asistida requieren la capacidad de percibir el entorno físico para detectar obstáculos, estimar distancias y reconocer elementos relevantes durante el desplazamiento del usuario. En dispositivos portables como gafas inteligentes, esta percepción del entorno debe realizarse mediante sensores compactos y de bajo consumo energético.

La electrónica utilizada en este tipo de sistemas generalmente combina sensores de distancia, cámaras y unidades de procesamiento capaces de analizar la información capturada en tiempo real. Estos componentes permiten construir modelos del entorno cercano y generar alertas o instrucciones de navegación para el usuario.

En el contexto del presente trabajo terminal, el sistema propuesto busca integrar sensores de detección de obstáculos junto con una cámara que permita realizar procesamiento visual mediante técnicas de inteligencia artificial. Por ello, resulta necesario analizar los distintos tipos de sensores disponibles, sus características técnicas y su viabilidad para ser integrados en un dispositivo

portable como unas gafas inteligentes.

En este capítulo se revisan los principales tipos de sensores de distancia utilizados en robótica y sistemas de percepción, así como las características de cámaras compactas empleadas en aplicaciones de visión por computadora. Asimismo, se analiza la factibilidad de utilizar plataformas comerciales de gafas inteligentes como base para el desarrollo del prototipo.

3.3.1. Sensores para detección de obstáculos

Importancia de los sensores en sistemas de navegación

Los sensores constituyen uno de los componentes fundamentales en los sistemas de navegación asistida, ya que permiten detectar objetos, medir distancias y obtener información del entorno físico. En aplicaciones de movilidad asistida, los sensores deben proporcionar mediciones confiables que permitan identificar obstáculos con suficiente anticipación para evitar colisiones.

En dispositivos portables, la selección del sensor se vuelve particularmente importante debido a las limitaciones de tamaño, peso, consumo energético y capacidad de procesamiento. Un sensor adecuado debe ofrecer un balance entre precisión, alcance y tamaño físico para poder integrarse en estructuras compactas como monturas de gafas.

Además, el sensor debe ser capaz de funcionar en diferentes condiciones ambientales, incluyendo variaciones de iluminación, presencia de ruido o interferencias, y distintos tipos de superficies.

Sensores ultrasónicos

Los sensores ultrasónicos son dispositivos que utilizan ondas sonoras de alta frecuencia para medir distancias. Su funcionamiento se basa en el principio de eco: el sensor emite un pulso ultrasónico que se propaga en el aire y rebota al encontrarse con un objeto. El sensor mide el tiempo que tarda el eco en regresar y calcula la distancia al obstáculo [43].

La distancia puede calcularse mediante la ecuación:

$$d = \frac{v \cdot t}{2}$$

donde d representa la distancia al objeto, v la velocidad del sonido en el aire y t el tiempo transcurrido entre la emisión y la recepción del eco [43].

Los sensores ultrasónicos presentan varias ventajas, entre ellas su bajo costo, facilidad de implementación y relativa precisión para distancias cortas y medias. Sin embargo, también presentan limitaciones importantes, como la sensibilidad a superficies absorbentes o irregulares y un ángulo de detección relativamente amplio que puede dificultar la identificación precisa de obstáculos.

Un ejemplo básico ampliamente utilizado en prototipos electrónicos es el sensor HC-SR04, el cual puede medir distancias aproximadas entre 2 cm y 4 m [43]. No obstante, para aplicaciones que demandan mayor precisión, mayor alcance y resistencia al ruido eléctrico y acústico, existen sensores ultrasónicos industriales con prestaciones considerablemente superiores.

Sensor ultrasónico MB1232 (I2CXL-MaxSonar-EZ3)

El sensor MB1232, perteneciente a la familia I2CXL-MaxSonar-EZ de MaxBotix, representa una alternativa de alto desempeño dentro de la categoría de sensores ultrasónicos [44]. A diferencia de los sensores básicos como el HC-SR04, el MB1232 incorpora algoritmos de procesamiento de señal en tiempo real que lo hacen especialmente adecuado para entornos ruidosos y aplicaciones de detección precisa de obstáculos.

El MB1232 opera a una frecuencia ultrasónica de 42 kHz y es capaz de medir distancias desde 20 cm hasta 765 cm (7.65 m) con una resolución de 1 cm [44]. Su tasa de actualización puede alcanzar hasta 40 Hz para objetos cercanos y 15 Hz para el rango completo, lo que permite una detección fluida de obstáculos en movimiento.

Entre sus características más destacadas para aplicaciones de navegación asistida se encuentran [44]:

- Interfaz de comunicación I2C, que permite controlar múltiples sensores mediante únicamente dos cables y conectar hasta 127 unidades sobre un mismo bus
- Calibración automática en tiempo real para compensar variaciones de voltaje, temperatura y ruido acústico o eléctrico del entorno
- Algoritmos de análisis de forma de onda y rechazo de ruido que producen lecturas prácticamente libres de interferencias
- Haz acústico estrecho con alta sensibilidad, adecuado para detectar objetos pequeños y medianos a distancias medias y largas
- Voltaje de operación de 3 V a 5.5 V con consumo de corriente promedio muy bajo, lo que facilita su uso en dispositivos alimentados por batería

- Tamaño físico inferior a un pulgada cúbica, con orificios de montaje integrados en la placa de circuito
- Rango de temperatura operacional de 0°C a 65°C

El bajo consumo energético y el tamaño compacto del MB1232 lo convierten en una opción especialmente adecuada para integrarse en dispositivos portables como gafas inteligentes. Además, su interfaz I2C simplifica la conexión con plataformas de procesamiento como Raspberry Pi o microcontroladores embebidos, reduciendo el número de pines necesarios y facilitando el uso simultáneo de múltiples sensores [44].

Sensores ToF (Time of Flight)

Los sensores Time of Flight (ToF) utilizan luz infrarroja para medir distancias mediante el cálculo del tiempo que tarda un pulso de luz en reflejarse en un objeto y regresar al sensor.

A diferencia de los sensores ultrasónicos, los sensores ToF ofrecen mayor precisión y una respuesta más rápida, lo que los hace especialmente útiles en aplicaciones de robótica y dispositivos móviles [45].

Entre sus principales características se encuentran:

- Alta precisión en mediciones de corta y media distancia
- Tamaño compacto
- Alta frecuencia de actualización
- Menor interferencia con el ruido ambiental

Sensores populares de este tipo incluyen el VL53L0X [46] y el VL53L1X [47], desarrollados por STMicroelectronics, los cuales pueden medir distancias de hasta aproximadamente 4 metros dependiendo de las condiciones de iluminación y reflectividad del objeto.

Sensores LiDAR compactos

Los sensores LiDAR (Light Detection and Ranging) utilizan pulsos de luz láser para medir distancias con alta precisión. Estos sensores son ampliamente utilizados en robótica autónoma, vehículos autónomos y sistemas de mapeo tridimensional.

Los sistemas LiDAR generan múltiples mediciones de distancia en diferentes direcciones, permitiendo construir representaciones tridimensionales del entorno.

Entre las ventajas de los sensores LiDAR se encuentran:

- Alta precisión
- Mayor alcance que sensores ultrasónicos o ToF
- Capacidad de generar mapas detallados del entorno

Sin embargo, los sensores LiDAR suelen presentar un mayor costo, mayor consumo energético y un tamaño físico mayor, lo que puede dificultar su integración en dispositivos como gafas inteligentes [48, 49].

Comparación entre sensores de distancia

La Tabla 3.1 presenta una comparación general entre los sensores más comunes utilizados para detección de obstáculos.

Tipo de sensor	Alcance típico	Tamaño	Precisión
Ultrasónico básico	2 cm – 4 m	Medio	Media
Ultrasónico avanzado (MB1232)	20 cm – 7.65 m	$14{,}4x15{,}5x15{,}5mm$	$1mm$
ToF	hasta 4 m	Muy pequeño	Alta
LiDAR compacto	10 m – 40 m	Medio	Muy alta

Tabla 3.1: Comparación entre diferentes tipos de sensores de distancia [43, 44, 47, 49].

Para dispositivos portables como gafas inteligentes, el sensor MB1232 representa la solución más equilibrada dentro de la categoría ultrasónica, gracias a su tamaño reducido, alcance de hasta 7.65 m, resolución centimétrica, interfaz I2C y robusta resistencia al ruido. Estas características lo hacen especialmente adecuado para la detección confiable de obstáculos en entornos urbanos con presencia de ruido acústico y eléctrico [44].

3.3.2. Cámaras para visión por computadora

Importancia de las cámaras en sistemas inteligentes

Las cámaras desempeñan un papel fundamental en los sistemas de visión por computadora, ya que permiten capturar imágenes del entorno que posteriormente pueden ser analizadas mediante algoritmos de inteligencia artificial.

En el caso de sistemas de navegación asistida, las cámaras pueden utilizarse para:

- Detectar objetos
- Reconocer señales o puntos de referencia
- Analizar el entorno urbano
- Complementar la información proporcionada por sensores de distancia

La integración de cámaras con algoritmos de visión por computadora permite construir sistemas capaces de interpretar el entorno visual y tomar decisiones basadas en esta información [50].

Características importantes en cámaras para dispositivos portables

Al seleccionar una cámara para un dispositivo portable, deben considerarse varios factores técnicos.

Resolución

La resolución determina la cantidad de detalle presente en la imagen capturada. Una mayor resolución permite mejorar la precisión de los algoritmos de detección de objetos, aunque también incrementa los requerimientos de procesamiento.

Tamaño físico

Para integrarse en unas gafas inteligentes, la cámara debe ser lo suficientemente pequeña para no afectar la ergonomía del dispositivo.

Consumo energético

El consumo energético es un factor crítico en dispositivos portables alimentados por baterías.

Campo de visión

El campo de visión (Field of View, FOV) determina el área del entorno que puede capturar la cámara. Para aplicaciones de navegación se suelen preferir cámaras con campos de visión amplios [51, 50].

Autofoco y rango dinámico

En aplicaciones de visión artificial para navegación asistida, el autofoco resulta fundamental para mantener la nitidez de los objetos detectados a distintas distancias. Asimismo, un rango dinámico elevado permite obtener imágenes utilizables tanto en interiores con poca iluminación como en exteriores con luz solar intensa.

Módulos de cámara compactos

Existen diversos módulos de cámara diseñados específicamente para sistemas embebidos.

Entre los más utilizados se encuentran:

- **Raspberry Pi Camera Module 3 (IMX708)** [52]
- **Arducam Mini Camera** [51]
- **OV2640** [53]
- **IMX219** [51]

Estos módulos ofrecen resoluciones que van desde 2 hasta 12 megapíxeles y se integran fácilmente con plataformas como Raspberry Pi, Jetson Nano o microcontroladores compatibles [50, 51].

Raspberry Pi Camera Module 3 (sensor IMX708)

El Raspberry Pi Camera Module 3 es la tercera generación del módulo de cámara oficial de Raspberry Pi, lanzado en 2023 [50]. Incorpora el sensor Sony IMX708, un sensor CMOS de back-illuminated y arquitectura apilada (stacked) de 12 megapíxeles con resolución máxima de 4608×2592 píxeles, lo que lo posiciona como uno de los módulos de cámara compactos de mayor resolución disponibles para plataformas embebidas.

El IMX708 incorpora un modo HDR (High Dynamic Range) que permite capturar imágenes de alta calidad en condiciones de iluminación variadas, lo cual resulta especialmente valioso en entornos de navegación urbana donde la iluminación cambia drásticamente entre espacios interiores y exteriores [52].

Entre las características técnicas más relevantes del Raspberry Pi Camera Module 3 se encuentran [50]:

- Sensor Sony IMX708 de 12 MP (4608×2592 píxeles), tipo back-illuminated stacked CMOS
- Sistema de autoenfoque por detección de fase (PDAF) combinado con detección por contraste (CDAF), que permite ajustar el foco de manera rápida y precisa sobre objetos a distintas distancias
- Modo HDR disponible para capturas de hasta 3 MP, útil en entornos con alta variación de iluminación
- Grabación de video Full HD a 50 fps, con tasas de fotogramas superiores disponibles a resoluciones menores
- Campo de visión diagonal de 75° en la versión estándar y de 120° en la versión Wide, esta última especialmente adecuada para navegación asistida al capturar un área del entorno más amplia
- Interfaz MIPI CSI-2 de 2 carriles, compatible con todos los modelos de Raspberry Pi equipados con conector CSI
- Factor de forma compacto, con dimensiones de placa iguales a las del Camera Module 2, lo que simplifica su integración en estructuras mecánicas de dispositivos portables
- Filtro de corte de infrarrojos integrado en la versión estándar; disponible también en versión NoIR para aplicaciones nocturnas con iluminación infrarroja

Su compatibilidad nativa con las bibliotecas `libcamera` y `Picamera2` de Raspberry Pi simplifica el desarrollo de aplicaciones de visión por computadora, eliminando la necesidad de controladores adicionales [50].

Modelo	Resolución	Tamaño	Uso común
OV2640 [53]	2 MP	Muy pequeño	IoT y microcontroladores
IMX219 [51]	8 MP	Pequeño	Raspberry Pi
IMX477 [50]	12 MP	Medio	Visión avanzada
IMX708 (Camera Module 3) [50]	12 MP	Pequeño	Visión avanzada, PDAF, HDR

Tabla 3.2: Comparación de módulos de cámara compactos.

3.3.3. Integración en gafas inteligentes

Diseño de hardware personalizado

Una opción para implementar el sistema consiste en diseñar un prototipo de gafas inteligentes desde cero. En este enfoque, los sensores, cámaras y procesadores se integran directamente en una estructura personalizada.

Este enfoque permite mayor flexibilidad en el diseño del sistema, ya que se pueden seleccionar los componentes electrónicos más adecuados para la aplicación. Sin embargo, también implica mayores desafíos de diseño mecánico, distribución de energía y miniaturización.

Uso de gafas inteligentes comerciales

Otra alternativa consiste en utilizar plataformas comerciales de gafas inteligentes como base para el prototipo. Entre las opciones disponibles actualmente destacan las gafas Ray-Ban Meta [54], las cuales incorporan cámaras, micrófonos y conectividad inalámbrica.

Estas gafas ofrecen ventajas importantes, como un diseño ergonómico optimizado y una integración electrónica ya probada [54]. No obstante, presentan ciertas limitaciones para proyectos de investigación, ya que su hardware y software suelen ser sistemas cerrados que restringen el acceso directo a los sensores y componentes internos.

Modificar este tipo de dispositivos puede resultar complejo debido a las restricciones del fabricante y a la dificultad de acceder a los circuitos internos sin afectar el funcionamiento del sistema [54].

Comparación de enfoques

La Tabla 3.3 muestra una comparación entre desarrollar un prototipo y modificar gafas comerciales.

Característica	Prototipo	Gafas comerciales
Control del hardware	Alto	Bajo
Facilidad de desarrollo	Media	Alta
Flexibilidad	Alta	Baja
Tiempo de desarrollo	Mayor	Menor
Acceso a sensores	Completo	Limitado

Tabla 3.3: Comparación entre prototipo y uso de gafas comerciales [54].

Para proyectos académicos y de investigación, el desarrollo de un prototipo, suele ser la opción más viable, ya que permite integrar sensores específicos como el MB1232 y el módulo de cámara IMX708, experimentar con diferentes configuraciones y adaptar el sistema a los objetivos del proyecto.

3.3.4. Análisis de viabilidad de plataformas de gafas inteligentes

Para implementar el sistema de navegación asistida propuesto, existen dos enfoques principales: desarrollar un prototipo de gafas inteligentes desde cero o utilizar plataformas comerciales existentes.

Entre las plataformas comerciales disponibles actualmente destacan dispositivos como las gafas Ray-Ban Meta [54], que integran cámaras, micrófonos y conectividad inalámbrica dentro de un diseño ergonómico.

Sin embargo, estas plataformas suelen utilizar sistemas cerrados que limitan el acceso directo a los sensores y componentes electrónicos internos, lo que dificulta su modificación para proyectos de investigación [54].

La Tabla 3.4 presenta una comparación entre ambos enfoques.

Característica	Prototipo	Ray-Ban Meta
Acceso al hardware	Completo	Muy limitado
Flexibilidad de sensores	Alta	Baja
Capacidad de modificación	Alta	Muy baja
Tiempo de desarrollo	Mayor	Menor
Costo de desarrollo	Medio	Alto
Uso en investigación	Adecuado	Limitado

Tabla 3.4: Comparación entre desarrollo y uso de gafas comerciales [54].

Para proyectos académicos y de investigación, el desarrollo de un prototipo suele ser la alternativa más adecuada, ya que permite seleccionar componentes específicos como el sensor ultrasónico MB1232 y el módulo de cámara Raspberry Pi Camera Module 3 con sensor IMX708, integrar diferentes módulos electrónicos y adaptar el sistema a los objetivos del proyecto.

3.4. Discapacidad visual

3.4.1. Concepto de discapacidad visual

La discapacidad visual se refiere a cualquier condición que limite o reduzca la capacidad de una persona para percibir su entorno mediante el sentido de la vista. Esta limitación puede presentarse en distintos grados y manifestarse a través de diversas alteraciones en el sistema visual, afectando funciones como la agudeza visual, el campo visual, la percepción de colores o la sensibilidad al contraste.

De acuerdo con la Organización Mundial de la Salud (OMS), la discapacidad visual "se produce cuando una afección ocular afecta al sistema visual y a sus funciones relacionadas con la visión" [55]. Esta definición incluye una amplia variedad de condiciones médicas que pueden afectar los ojos o las vías visuales del sistema nervioso central.

En términos generales, la discapacidad visual engloba todas las formas de pérdida de visión que no pueden ser corregidas completamente mediante lentes, cirugía u otros tratamientos médicos convencionales. Esto incluye desde una disminución leve en la capacidad visual hasta la pérdida total de la visión.

Las causas de la discapacidad visual pueden ser múltiples y abarcan tanto factores congénitos como adquiridos. Entre las causas más comunes se encuentran los errores refractivos no corregidos, cataratas, glaucoma, degeneración macular relacionada con la edad, retinopatía diabética, lesiones traumáticas y enfermedades neurológicas que afectan el procesamiento visual.

Además del aspecto médico, la discapacidad visual también debe entenderse desde una perspectiva social y funcional. No solo implica una alteración en el sistema visual, sino que también puede generar limitaciones significativas en la realización de actividades cotidianas como leer, desplazarse de forma segura, reconocer rostros, utilizar dispositivos tecnológicos o interactuar con el entorno urbano.

Por esta razón, actualmente se promueve un enfoque basado en la Clasificación Internacional del Funcionamiento, de la Discapacidad y de la Salud [56], que considera tanto las condiciones

médicas como los factores ambientales y sociales que influyen en la calidad de vida de las personas con discapacidad visual.

Definición de discapacidad visual según organismos internacionales

Diversos organismos internacionales han desarrollado definiciones y criterios para clasificar la discapacidad visual con el objetivo de facilitar su diagnóstico, tratamiento y seguimiento a nivel global. Entre las instituciones más relevantes que han establecido lineamientos en este ámbito se encuentran la Organización Mundial de la Salud (OMS), la International Agency for the Prevention of Blindness (IAPB) y diferentes organismos gubernamentales responsables de la salud pública.

Según la OMS, la discapacidad visual ocurre cuando una afección ocular afecta el funcionamiento del sistema visual de tal manera que la persona experimenta limitaciones significativas en su capacidad para realizar actividades que dependen de la visión [55]. Estas afecciones pueden afectar diferentes componentes del sistema visual, incluyendo la retina, el nervio óptico, la córnea o el cerebro.

Desde una perspectiva epidemiológica, la discapacidad visual representa un problema importante de salud pública. La OMS estima que en el mundo existen más de 2.200 millones de personas con algún tipo de deterioro visual, de las cuales aproximadamente 1.000 millones presentan afecciones que podrían haberse prevenido o tratado adecuadamente [55]. Estas cifras reflejan la magnitud global del problema y resaltan la importancia de implementar políticas de prevención, diagnóstico temprano y tratamiento oportuno.

En muchos países se utiliza el criterio establecido por la OMS para definir la denominada ceguera legal, la cual se considera cuando la agudeza visual en el mejor ojo es inferior a 0.05 (3/60) incluso después de aplicar corrección óptica [57]. Este criterio permite determinar el nivel de discapacidad visual que puede dar acceso a programas de apoyo, servicios de rehabilitación y beneficios sociales.

Asimismo, algunas legislaciones consideran que una persona presenta discapacidad visual cuando su agudeza visual es inferior a 0.3 (6/18) en el mejor ojo con corrección. Este enfoque reconoce que incluso niveles moderados de pérdida visual pueden generar dificultades significativas en la vida diaria [57].

En síntesis, los organismos internacionales coinciden en que la discapacidad visual debe definirse no solo en términos médicos, sino también en función de su impacto en la funcionalidad y en la capacidad de la persona para interactuar con su entorno.

Clasificación de la discapacidad visual

La clasificación de la discapacidad visual permite categorizar el grado de deterioro visual de una persona con base en criterios médicos estandarizados. Esta clasificación se basa principalmente en dos parámetros fundamentales:

- Agudeza visual (AV)
- Campo visual

La agudeza visual mide la capacidad de una persona para distinguir detalles finos a una distancia determinada, mientras que el campo visual representa el área total que puede observar una persona sin mover los ojos.

De acuerdo con los criterios establecidos por la OMS y la Clasificación Internacional del Funcionamiento, la discapacidad visual puede dividirse en diferentes niveles de severidad según la agudeza visual del mejor ojo con corrección óptica [57]:

- **Leve:** Agudeza visual entre 0.3 y 0.5 ($\geq$6/18). En este nivel, la persona puede realizar muchas actividades cotidianas, aunque puede experimentar dificultades en tareas que requieren precisión visual, como leer letras pequeñas o reconocer detalles a distancia.
- **Moderada:** Agudeza visual entre 0.1 y 0.3 ($<$6/18). Las personas con este grado de discapacidad visual presentan limitaciones más evidentes en actividades como leer, conducir o identificar objetos a distancia.
- **Severa:** Agudeza visual entre 0.05 y 0.1 ($<$6/60). En esta etapa la persona tiene una visión muy reducida y requiere ayudas ópticas o tecnológicas para realizar muchas actividades cotidianas [57].
- **Ceguera:** Agudeza visual inferior a 0.05 ($<$3/60) en el mejor ojo con corrección óptica [57]. En esta condición, la visión es extremadamente limitada o inexistente.

Además de la agudeza visual, el campo visual también puede utilizarse para determinar la presencia de discapacidad visual. Por ejemplo, una persona puede tener una agudeza visual relativamente buena, pero presentar un campo visual extremadamente reducido (visión en túnel), lo cual genera limitaciones similares a las de una persona con baja visión.

Estas clasificaciones permiten establecer criterios clínicos uniformes y facilitan la comparación de estadísticas epidemiológicas a nivel internacional.

3.4.2. Tipos de discapacidad visual

La discapacidad visual puede manifestarse de diferentes maneras dependiendo del tipo de alteración que afecte al sistema visual. En general, estas condiciones se clasifican en función del grado de pérdida visual y de la naturaleza de las limitaciones que generan en la vida cotidiana.

Ceguera total

La ceguera total representa la forma más severa de discapacidad visual. Se caracteriza por una pérdida prácticamente completa de la visión, lo que impide que la persona perciba formas, colores o movimientos en el entorno.

Desde un punto de vista clínico, la ceguera total se define generalmente como una agudeza visual inferior a 0.05 (3/60) en el mejor ojo, incluso después de aplicar corrección óptica [57]. También puede diagnosticarse cuando el campo visual se encuentra restringido a menos de 10 grados.

Las personas con ceguera total no pueden utilizar la visión como medio principal para orientarse o interactuar con su entorno. Por esta razón, dependen en gran medida de otros sentidos como el tacto y el oído para realizar actividades cotidianas.

Existen dos tipos principales de ceguera total:

- Ceguera congénita: Se presenta desde el nacimiento y puede ser causada por factores genéticos, infecciones durante el embarazo o malformaciones del sistema visual.
- Ceguera adquirida: Ocurre posteriormente debido a enfermedades o lesiones que afectan el sistema visual.

Las principales causas de ceguera incluyen enfermedades como:

- Retinopatía diabética
- Glaucoma avanzado
- Infecciones oculares graves
- Traumatismos oculares
- daños neurológicos en el nervio óptico

[55]

Debido a la ausencia de información visual, las personas con ceguera total suelen utilizar herramientas de apoyo como bastones blancos, perros guía o tecnologías de asistencia para desplazarse de manera segura.

Baja visión

La baja visión se refiere a una condición en la que una persona presenta una reducción significativa de su capacidad visual, pero aún conserva cierto grado de visión útil. A diferencia de la ceguera total, las personas con baja visión pueden percibir algunos elementos visuales como luces, sombras, formas grandes o colores intensos.

Generalmente se considera baja visión cuando la agudeza visual se encuentra entre 0.05 y 0.3 en el mejor ojo, incluso después de utilizar corrección óptica [57].

Las personas con baja visión pueden enfrentar dificultades para realizar tareas como:

- Leer textos
- Reconocer rostros
- Distinguir objetos a distancia
- Desplazarse en entornos con iluminación deficiente

[55]

Entre las causas más frecuentes de baja visión se encuentran:

- Degeneración macular
- Gglaucoma
- Cataratas avanzadas
- Daño retiniano
- Enfermedades genéticas o degenerativas

[55]

Aunque estas condiciones limitan significativamente la capacidad visual, muchas personas con baja visión pueden mantener un nivel considerable de independencia mediante el uso de ayudas ópticas, tecnológicas y programas de rehabilitación visual.

Deficiencias visuales parciales

Además de la reducción en la agudeza visual, existen otras alteraciones que afectan la calidad de la visión sin necesariamente provocar una pérdida completa de la capacidad visual. Estas condiciones se conocen como deficiencias visuales parciales.

- **Defectos del campo visual:** Algunas enfermedades o lesiones pueden provocar la pérdida parcial de la visión periférica o central. Por ejemplo, el glaucoma puede generar una condición conocida como visión en túnel, en la cual la persona pierde gradualmente su campo visual periférico [55].
- **Alteraciones en la percepción del color:** El daltonismo y otras alteraciones cromáticas afectan la capacidad para distinguir ciertos colores. Aunque estas condiciones no suelen provocar discapacidad severa, pueden generar dificultades en actividades que dependen del reconocimiento cromático [57].
- **Problemas de sensibilidad al contraste:** Algunas personas presentan dificultades para distinguir objetos cuando existe poca diferencia de brillo entre ellos. Esto puede complicar tareas como leer textos impresos o identificar escalones [57].

Estas deficiencias visuales pueden generar limitaciones significativas en situaciones reales, especialmente en entornos urbanos complejos donde la información visual es esencial para la orientación.

3.4.3. Impacto de la discapacidad visual en la movilidad

La discapacidad visual afecta profundamente la capacidad de una persona para desplazarse de manera segura y autónoma. La visión es uno de los principales sentidos utilizados para orientarse en el espacio, evitar obstáculos y planificar rutas de desplazamiento.

Cuando esta capacidad se ve limitada, las personas pueden experimentar dificultades significativas para realizar actividades cotidianas como caminar por la ciudad, utilizar transporte público o desplazarse en entornos desconocidos.

Barreras físicas en entornos urbanos

Las ciudades modernas presentan múltiples obstáculos que dificultan la movilidad de las personas con discapacidad visual. Entre los problemas más comunes:

- Aceras irregulares
- Escalones sin señalización
- Postes u objetos colocados en zonas de paso
- Falta de rampas accesibles
- Ausencia de pavimento táctil

Estas barreras pueden generar riesgos significativos de caídas o accidentes. Investigaciones recientes indican que muchas personas con discapacidad visual encuentran dificultades para desplazarse de manera segura e independiente debido a la presencia de estos obstáculos [58].

Dificultades de orientación y desplazamiento

La orientación espacial depende en gran medida de la información visual disponible. Cuando esta información no está presente, las personas deben recurrir a otras estrategias como:

- Referencias auditivas
- Memoria espacial
- Señales táctiles

Sin embargo, estas estrategias no siempre son suficientes en entornos urbanos complejos. Según estudios recientes, la falta de información visual puede provocar desorientación, incremento del tiempo de desplazamiento y mayor riesgo de accidentes [58].

Importancia de la accesibilidad urbana

La accesibilidad urbana busca diseñar espacios públicos que puedan ser utilizados por todas las personas, incluyendo aquellas con discapacidad visual. Ya existen ciertas medidas para personas con discapacidad como el pavimento táctil para guiar rutas peatonales, los semáforos con señales auditivas, señalización en braille o iluminación adecuada en calles y edificios.

Estas medidas permiten mejorar la independencia y seguridad de las personas con discapacidad visual al desplazarse en espacios públicos.

3.4.4. Tecnologías de asistencia para personas con discapacidad visual

Las tecnologías de asistencia han evolucionado significativamente en las últimas décadas. Actualmente existen múltiples herramientas diseñadas para ayudar a las personas con discapacidad visual a interactuar con su entorno y desplazarse de manera más segura.

Métodos tradicionales de asistencia (bastón, perro guía)

Los métodos tradicionales incluyen herramientas ampliamente utilizadas como el bastón blanco o el perro guía

El bastón permite detectar obstáculos cercanos mediante el contacto con el suelo, mientras que el perro guía puede ayudar a evitar obstáculos y seguir rutas conocidas.

Aunque estos métodos siguen siendo fundamentales, ofrecen información limitada sobre el entorno y requieren entrenamiento especializado [58].

Tecnologías digitales de asistencia

Los avances en tecnología han permitido el desarrollo de soluciones más avanzadas como:

- Sistemas de navegación basados en GPS
- Aplicaciones móviles de reconocimiento visual
- Dispositivos portátiles con cámaras

Estas tecnologías pueden identificar objetos, leer textos y proporcionar información auditiva al usuario sobre su entorno.

Dispositivos inteligentes para la movilidad asistida

Los dispositivos inteligentes combinan sensores, cámaras y algoritmos de inteligencia artificial para proporcionar asistencia avanzada durante la movilidad.

Entre los dispositivos más innovadores se encuentran las gafas inteligentes con cámaras, los bastones electrónicos con sensores, dispositivos vestibles con retroalimentación háptica.

Investigaciones recientes han demostrado que los sistemas que combinan visión artificial, señales auditivas y vibraciones pueden mejorar significativamente la capacidad de navegación de las personas con discapacidad visual [59].

3.5. Navegación digital en mapas

La navegación digital se ha convertido en una herramienta fundamental dentro de los sistemas modernos de movilidad y localización. En el contexto de aplicaciones inteligentes y dispositivos portátiles, la navegación digital permite integrar información geográfica, sistemas de posicionamiento y algoritmos de enrutamiento para asistir a los usuarios durante sus desplazamientos [60].

Los sistemas de navegación digital combinan diferentes tecnologías, como bases de datos cartográficas, sensores de posicionamiento y servicios de mapas en línea, con el objetivo de guiar a un usuario desde un punto de origen hasta un destino determinado. Estos sistemas son capaces de transformar información geográfica compleja en instrucciones comprensibles para el usuario, incluyendo indicaciones paso a paso, cálculo de rutas óptimas y alertas contextuales [61].

La integración de sistemas de información geográfica y servicios de mapas digitales ha permitido el desarrollo de plataformas modernas de navegación capaces de representar información espacial en tiempo real. Tecnologías como OpenStreetMap, Google Maps y los sistemas GIS facilitan el análisis y visualización de datos geográficos utilizados en aplicaciones de movilidad urbana y navegación asistida [62].

En aplicaciones de movilidad asistida y navegación inteligente, los sistemas digitales de mapas permiten integrar algoritmos de planificación de rutas, percepción del entorno y actualización dinámica de trayectorias. Esto resulta especialmente importante en sistemas portables orientados a personas con discapacidad visual, donde la navegación debe adaptarse continuamente a cambios en el entorno urbano [63].

En este capítulo se revisan los principales fundamentos teóricos relacionados con la navegación digital, incluyendo los sistemas de información geográfica, las tecnologías de posicionamiento global, los servicios de mapas digitales y sus aplicaciones dentro de la movilidad urbana. Asimismo, se analizan los componentes principales de un sistema de navegación digital y su relevancia para el desarrollo de soluciones de asistencia, particularmente en dispositivos portables orientados a mejorar la accesibilidad de personas con discapacidad visual.

3.5.1. Sistemas de navegación digital

Concepto de navegación digital

La navegación digital se refiere al uso de mapas electrónicos y tecnologías de posicionamiento para determinar la ubicación de un usuario y proporcionar instrucciones de ruta hacia un destino específico. Este tipo de sistemas integra información geográfica con algoritmos de cálculo de rutas para generar instrucciones de desplazamiento optimizadas.[64]

En términos generales, un sistema de navegación digital recibe como entrada la ubicación actual del usuario, obtenida mediante tecnologías de posicionamiento como GPS o sensores de dispositivos móviles, así como un destino seleccionado por el usuario. Posteriormente, el sistema calcula una ruta óptima dentro de una red de calles o caminos utilizando algoritmos de búsqueda de rutas, como el cálculo de la ruta más corta o la ruta más rápida.[64]

La navegación digital se refiere al uso de mapas electrónicos y tecnologías de posicionamiento, como GPS, Wi-Fi y sensores inerciales, para determinar la ubicación de un usuario y proporcionar guía de rutas. Estos sistemas transforman información geográfica y espacial en instrucciones de navegación comprensibles para el usuario, permitiendo calcular trayectorias óptimas entre un origen y un destino dentro de redes urbanas o entornos interiores. En la práctica, un sistema de navegación digital utiliza la posición actual del usuario —obtenida mediante GNSS o técnicas de posicionamiento en interiores— junto con un destino deseado para generar rutas y asistir el desplazamiento en tiempo real [64, 65].

Entre los conceptos fundamentales que componen la navegación digital se encuentran:

- **Positioning**: Determining real-time coordinates (latitude/longitude) via GPS or alternative systems.
- **Routing**: Calculating a path on a map from origin to destination using shortest-path or fastest-route algorithms.
- **User interface**: Presenting map views and instructions (visual or auditory) to the user.

Los sistemas de navegación digital suelen combinar estos elementos para asistir desplazamientos a pie, en automóvil, bicicleta o transporte público. En dispositivos móviles y wearables, los datos de navegación se obtienen mediante APIs de servicios de mapas o mediante mapas almacenados localmente, actualizándose de manera continua conforme el usuario se desplaza.[64]

En el caso de prototipos portables, la navegación debe integrar la información proveniente de

sensores como GPS o cámaras con servicios de mapas digitales, equilibrando la capacidad de respuesta en tiempo real con las limitaciones de energía y conectividad del dispositivo.

Componentes de un sistema de navegación

Un sistema completo de navegación digital está compuesto por varios módulos que interactúan entre sí para ofrecer orientación al usuario [61].

Positioning Module

Utiliza receptores GPS/GNSS y en algunos casos sensores inerciales o redes Wi-Fi para estimar la ubicación actual del dispositivo. Una limitación importante es que el GPS funciona de forma óptima en exteriores, pero puede presentar fallas en interiores. Por esta razón, algunos sistemas incorporan sensores IMU o balizas BLE para mejorar la continuidad del posicionamiento [66, **?**].

Map Database

Consiste en la base de datos cartográfica que contiene información sobre calles, intersecciones, puntos de interés y otros elementos geográficos. Estos datos pueden ser proporcionados por servicios como Google Maps, OpenStreetMap o Mapbox. Dependiendo del diseño del sistema, los mapas pueden almacenarse localmente o solicitarse dinámicamente a través de una API [67, 64].

Routing Engine

Es el módulo encargado de calcular la mejor ruta dentro de la red vial. Puede ejecutarse en servidores remotos, como ocurre con servicios en la nube (por ejemplo Google Directions API) [64], o directamente en el dispositivo mediante motores de enrutamiento como OSRM o GraphHopper [68].

User Interface / Output

Este componente presenta las instrucciones al usuario mediante interfaces visuales, auditivas o hápticas. En dispositivos portables, las instrucciones suelen comunicarse mediante audio o vibraciones [69].

Sensors for Context

En sistemas avanzados, especialmente en dispositivos de asistencia, se incorporan sensores adicionales como cámaras o sensores de proximidad para detectar obstáculos. Estos sensores permiten modificar la navegación en tiempo real, ajustando las instrucciones o generando alertas cuando se detecta un obstáculo cercano [60, 69].

Cada uno de estos componentes debe optimizarse para su implementación en dispositivos portables, considerando restricciones como el consumo energético, la capacidad de procesamiento y el ancho de banda disponible.

3.5.2. Sistemas de información geográfica (SIG)

Definición de los sistemas de información geográfica

Un Sistema de Información Geográfica (SIG) es un marco de software diseñado para capturar, almacenar, analizar y visualizar datos geográficos [70]. Estos sistemas permiten integrar múltiples fuentes de información espacial, como imágenes satelitales, redes de transporte y puntos de interés, organizándolos mediante su ubicación en la superficie terrestre.

According to Esri, a GIS "integrates many types of data" and organizes them into map visualizations and 3D scenes [70]. Dentro de los sistemas de navegación, los SIG proporcionan la infraestructura necesaria para la gestión de bases de datos cartográficas y el análisis espacial requerido para calcular rutas o identificar ubicaciones cercanas.

Entre las funciones principales de un SIG se encuentran:

- Integración de múltiples capas de información geográfica.
- Análisis espacial de datos.
- Visualización de mapas en dos y tres dimensiones.
- Análisis de redes para cálculo de rutas.

Estas capacidades permiten que los sistemas de navegación digital procesen información geográfica compleja y generen rutas optimizadas para diferentes medios de transporte.[70]

Representación digital de mapas

Los mapas digitales representan características geográficas mediante estructuras de datos digitales. Existen dos formatos principales para representar mapas en sistemas computacionales:

Raster maps

Son representaciones basadas en una cuadrícula de píxeles, similares a una imagen. Este tipo de mapas se utiliza comúnmente en imágenes satelitales o en mapas renderizados previamente.

Strengths: simple to display on any device.

Weaknesses: lack flexibility (can't change style on the fly) [71].

Vector maps

Están compuestos por primitivas geométricas como puntos, líneas y polígonos, a las cuales se les asocian atributos. Esto permite modificar dinámicamente el estilo visual de los mapas [71].

Por ejemplo, las calles pueden representarse mediante líneas vectoriales que pueden cambiar de grosor o color dependiendo del nivel de zoom o del estilo del mapa.

La mayoría de los servicios modernos utilizan un sistema basado en *tiles*. En este modelo, el mapa se divide en pequeñas secciones cuadradas que se cargan dinámicamente conforme el usuario navega por el mapa.

Geoapify explica que:

"Map tiles are the building blocks of a digital map… generated based on GIS data, rendered by a map server, and delivered on demand" [71].

Los *raster tiles* son imágenes estáticas, mientras que los *vector tiles* contienen información geográfica codificada que puede renderizarse dinámicamente [71]. Para dispositivos portables, los vector tiles suelen ser preferibles debido a su menor consumo de datos y mayor flexibilidad [71].

3.5.3. Tecnologías utilizadas en navegación digital

Sistema de posicionamiento global (GPS)

El Sistema de Posicionamiento Global (GPS) es la tecnología de posicionamiento satelital más utilizada a nivel mundial. Este sistema permite determinar la ubicación de un dispositivo con una precisión aproximada de entre 1 y 5 metros bajo condiciones normales.[66]

Sin embargo, el GPS presenta algunas limitaciones, como la pérdida de señal en zonas urbanas densas o en interiores [66]. Para mejorar la precisión y disponibilidad del posicionamiento, los receptores modernos utilizan múltiples constelaciones satelitales.

Entre los principales sistemas GNSS se encuentran:

Sistema	Operador	Inicio	Precisión aproximada
GPS	Estados Unidos	1978	∼5 m
GLONASS	Rusia	1982	∼5–10 m
Galileo	Unión Europea	2016	∼1 m
BeiDou	China	2020	∼3–5 m

Tabla 3.5: Principales sistemas GNSS utilizados en navegación digital [66].

Los receptores multi-GNSS pueden utilizar simultáneamente varios sistemas satelitales, lo que incrementa el número de satélites visibles y mejora la precisión del posicionamiento [66].

Servicios de mapas digitales

Existen diversas plataformas que ofrecen servicios de mapas digitales, entre las que destacan Google Maps, OpenStreetMap, Mapbox y HERE.

Google Maps

Ofrece una base de datos cartográfica altamente detallada, con información actualizada sobre puntos de interés, tráfico y transporte público. Proporciona múltiples APIs para integración en aplicaciones móviles y web [64].

OpenStreetMap

Es un proyecto colaborativo de código abierto que permite crear y editar mapas libremente. Su principal ventaja es que los datos pueden utilizarse sin costo y pueden adaptarse a diferentes aplicaciones [62].

Mapbox y HERE

Son plataformas comerciales que combinan datos abiertos con información propietaria, ofreciendo herramientas avanzadas de personalización de mapas.

La elección del servicio de mapas depende de factores como el costo, la disponibilidad de APIs, la posibilidad de uso offline y la capacidad de personalización.

Integración de APIs de navegación

Las aplicaciones modernas de navegación suelen integrar múltiples APIs para proporcionar funcionalidades completas.

Directions API

Permite calcular rutas entre dos puntos y obtener instrucciones paso a paso [64].

Geocoding / Places API

Convierte direcciones o nombres de lugares en coordenadas geográficas [64].

Transit APIs

Proporcionan información sobre rutas de transporte público y horarios [72].

En el caso de sistemas portables, como las gafas inteligentes propuestas en este trabajo, estas APIs pueden utilizarse para solicitar rutas utilizando las coordenadas actuales del usuario y el destino seleccionado. El sistema recibe una serie de puntos intermedios y genera instrucciones de navegación que pueden comunicarse mediante audio o vibración.

3.5.4. Aplicaciones de la navegación digital en movilidad urbana

Navegación peatonal

La navegación peatonal está diseñada específicamente para usuarios que se desplazan caminando. Este tipo de navegación considera elementos como aceras, cruces peatonales y senderos que no forman parte de las redes vehiculares.

Las aplicaciones modernas de mapas ofrecen modos de navegación peatonal que proporcionan instrucciones paso a paso adaptadas a la velocidad de desplazamiento de una persona. Además, pueden incorporar información sobre escaleras, elevadores o accesos peatonales [64].

En dispositivos de asistencia, como las gafas inteligentes, la navegación peatonal puede complementarse con sensores que detecten obstáculos, permitiendo adaptar las instrucciones en tiempo real para mejorar la seguridad del usuario.

Navegación en transporte público

La navegación basada en transporte público combina segmentos de caminata con trayectos en autobuses, trenes o metro. Para ello, los sistemas de navegación integran datos provenientes de fuentes como el estándar GTFS (General Transit Feed Specification), que describe rutas, horarios y paradas del transporte público [72].

Las plataformas de mapas pueden utilizar esta información para generar rutas multimodales que incluyan caminata y transporte público, así como estimaciones de tiempo de llegada.

En sistemas de asistencia personal, estas funciones permiten informar al usuario cuándo abordar un vehículo, cuándo descender y cuál es la mejor ruta para llegar a su destino final.

Sistemas de navegación asistida

Los sistemas de navegación asistida representan una evolución de los sistemas de navegación tradicionales, al integrar sensores adicionales y algoritmos inteligentes para adaptarse al entorno del usuario [37].

En el contexto de dispositivos portables, estos sistemas pueden combinar navegación digital con técnicas de visión por computadora para detectar obstáculos o reconocer elementos del entorno urbano. De esta forma, el sistema no solo proporciona instrucciones de ruta, sino que también ofrece información contextual que mejora la seguridad y autonomía del usuario [37].

Este enfoque es particularmente relevante en aplicaciones de asistencia para personas con discapacidad visual, donde la navegación digital puede integrarse con sensores de proximidad y procesamiento de imágenes para generar alertas sobre obstáculos cercanos o cambios en el entorno [37].

3.6. Ray casting

El ray casting es una técnica ampliamente utilizada en gráficos por computadora, robótica y sistemas de percepción espacial para determinar la interacción entre rayos proyectados y objetos dentro de un entorno [73]. En aplicaciones de navegación y mapeo, esta técnica permite estimar distancias, detectar obstáculos y construir representaciones del entorno a partir de sensores.

Dentro del contexto de sistemas de asistencia y navegación inteligente, como el prototipo de gafas inteligentes propuesto en este trabajo terminal, el ray casting puede utilizarse para modelar el entorno cercano del usuario y detectar espacios transitables. Al proyectar rayos desde la posición del usuario hacia diferentes direcciones, es posible identificar obstáculos y actualizar mapas locales que faciliten la toma de decisiones de navegación.

En este capítulo se presentan los fundamentos teóricos del ray casting, su relación con otras técnicas como el ray tracing, así como sus principales aplicaciones en gráficos por computadora, simulación y robótica. Finalmente, se analiza su utilización en sistemas de mapeo y navegación en

tiempo real.

3.6.1. Fundamentos del ray casting

Definición de ray casting

El ray casting es una técnica computacional que consiste en proyectar rayos desde un punto de origen hacia diferentes direcciones dentro de un entorno para detectar la primera intersección con un objeto u obstáculo. Este método permite calcular distancias, verificar visibilidad y estimar la estructura espacial de un entorno [73].

A diferencia de técnicas más complejas de renderizado, el ray casting no calcula reflexiones ni interacciones de luz complejas. Su principal objetivo es determinar si un rayo intersecta con un objeto y, en caso afirmativo, identificar la distancia al primer punto de contacto [73].

Debido a su simplicidad computacional, el ray casting es ampliamente utilizado en sistemas que requieren procesamiento en tiempo real, como motores de videojuegos, simuladores de sensores y algoritmos de mapeo robótico [74].

En sistemas de navegación, esta técnica puede emplearse para simular sensores de distancia, detectar obstáculos en mapas digitales y construir representaciones espaciales del entorno [75].

Principios matemáticos del trazado de rayos

Desde el punto de vista matemático, un rayo puede representarse como una función paramétrica que describe la trayectoria de un punto en el espacio a partir de un origen y una dirección.

Sea un punto de origen (x_0, y_0) y un ángulo de dirección θ. La trayectoria del rayo puede describirse mediante las ecuaciones:

$$x = x_0 + t\cos(\theta)$$

$$y = y_0 + t\sin(\theta)$$

donde t representa la distancia recorrida a lo largo del rayo.

En sistemas discretos como los mapas de cuadrícula utilizados en robótica y navegación, el

espacio se divide en celdas. El algoritmo de ray casting recorre progresivamente estas celdas a lo largo de la trayectoria del rayo hasta encontrar la primera celda ocupada o alcanzar una distancia máxima [75].

Para implementar este proceso de forma eficiente se utilizan algoritmos de recorrido de líneas en cuadrículas, como el algoritmo de Bresenham [76] o el método Digital Differential Analyzer (DDA) [77]. Estos algoritmos permiten identificar las celdas atravesadas por el rayo utilizando operaciones aritméticas simples.

En aplicaciones de mapeo y navegación, el ray casting puede utilizarse para simular sensores como LIDAR o cámaras de profundidad, proyectando múltiples rayos en distintas direcciones para reconstruir la geometría del entorno [75].

3.6.2. Diferencias entre ray casting y ray tracing

Definición de ray tracing

El ray tracing es una técnica de renderizado utilizada principalmente en gráficos por computadora para generar imágenes realistas mediante la simulación del comportamiento físico de la luz.

A diferencia del ray casting, el ray tracing sigue múltiples trayectorias de rayos después de cada interacción con un objeto, considerando fenómenos como reflexión, refracción y dispersión de la luz [73]. Esto permite generar efectos visuales complejos, como sombras suaves, reflejos y transparencia.

Sin embargo, debido a la gran cantidad de cálculos requeridos para simular estas interacciones, el ray tracing presenta un costo computacional considerablemente mayor que el ray casting [73].

Comparación entre ray casting y ray tracing

Las principales diferencias entre ambas técnicas pueden observarse en la Tabla 3.6.

Aspecto	Ray Casting	Ray Tracing
Propósito	Detección de visibilidad y distancia	Renderizado realista
Profundidad del rayo	Una sola intersección	Múltiples reflexiones
Complejidad computacional	Baja	Alta
Latencia	Baja, apta para tiempo real	Alta, requiere GPU
Aplicaciones	Robótica, simulación de sensores	Gráficos fotorrealistas

Tabla 3.6: Comparación entre ray casting y ray tracing [73].

Como puede observarse, el ray casting resulta significativamente más eficiente desde el punto de vista computacional, ya que cada rayo se procesa una sola vez hasta encontrar el primer obstáculo [73].

Ventajas del ray casting en sistemas en tiempo real

Debido a su simplicidad algorítmica, el ray casting es particularmente adecuado para sistemas que requieren procesamiento en tiempo real [73].

Entre sus principales ventajas se encuentran:

- Bajo costo computacional.
- Fácil implementación en sistemas embebidos.
- Capacidad de ejecutarse en CPU sin necesidad de GPU.
- Compatibilidad con mapas discretos basados en cuadrículas [75].

Estas características lo convierten en una técnica ideal para aplicaciones de robótica móvil, sistemas de navegación autónoma y dispositivos portables con recursos computacionales limitados.

3.6.3. Aplicaciones del ray casting

Ray casting en gráficos por computadora

En el ámbito de los gráficos por computadora, el ray casting se ha utilizado históricamente para determinar la visibilidad de objetos dentro de una escena tridimensional [73].

Uno de los usos más conocidos fue en los primeros motores gráficos de videojuegos en primera persona, donde el ray casting se utilizaba para calcular la distancia entre el jugador y las paredes del entorno [74]. A partir de esta información, el sistema generaba una representación tridimensional simulada utilizando un entorno bidimensional.

Aunque en la actualidad muchos motores gráficos utilizan técnicas más avanzadas como ray tracing o rasterización acelerada por GPU, el ray casting sigue siendo relevante para cálculos rápidos de visibilidad y detección de colisiones [73].

Ray casting en simulación y videojuegos

En simulación y videojuegos, el ray casting se utiliza para diversas tareas relacionadas con la interacción entre objetos y el entorno [74].

Entre sus aplicaciones se incluyen:

- Detección de colisiones.
- Simulación de sensores virtuales.
- Cálculo de visibilidad entre personajes.
- Sistemas de iluminación simplificados.

La capacidad de proyectar rayos en múltiples direcciones permite simular sensores de distancia o verificar si existe una línea de visión directa entre dos puntos del entorno [73].

Ray casting en robótica y percepción espacial

En robótica móvil, el ray casting se emplea frecuentemente para modelar sensores de distancia y actualizar mapas del entorno [75].

Un uso común es la simulación de sensores LIDAR dentro de mapas de ocupación. En este contexto, se proyectan múltiples rayos desde la posición del robot hacia distintas direcciones. Cada rayo se detiene cuando detecta una celda ocupada, permitiendo estimar la distancia a los obstáculos cercanos [75].

Esta técnica forma parte de algoritmos de mapeo y localización utilizados en sistemas de navegación autónoma y en métodos de SLAM (Simultaneous Localization and Mapping) [78].

3.6.4. Uso del ray casting en mapeo y navegación en tiempo real

Construcción de mapas de ocupación

Los mapas de ocupación representan el entorno mediante una cuadrícula de celdas donde cada celda almacena la probabilidad de estar ocupada por un obstáculo.

Este concepto fue introducido por Elfes en 1989 [75] y constituye uno de los métodos más utilizados en robótica para representar entornos desconocidos.

Cada celda de la cuadrícula puede contener uno de tres estados principales:

- Libre
- Ocupada
- Desconocida

El ray casting permite actualizar estas celdas mediante la proyección de rayos que atraviesan el mapa. Las celdas recorridas por el rayo se marcan como libres, mientras que la celda donde el rayo se detiene se marca como ocupada [75]. Para recorrer las celdas de forma eficiente se emplean algoritmos como el de Bresenham [76] o el DDA [77].

Este proceso se repite continuamente a medida que el sistema recibe nuevos datos de sensores.

Detección de espacios transitables

Una de las aplicaciones más importantes del ray casting en sistemas de navegación es la detección de espacios transitables [75].

Al proyectar rayos en diferentes direcciones desde la posición del usuario o robot, es posible identificar áreas libres de obstáculos dentro del entorno. Esta información puede utilizarse para:

- Identificar rutas seguras
- Evitar colisiones
- Ajustar la trayectoria de navegación

En entornos urbanos o interiores, esta capacidad resulta fundamental para garantizar una navegación segura y eficiente [78].

Aplicación en sistemas de navegación asistida

En sistemas de asistencia para movilidad, el ray casting puede utilizarse para complementar la información proporcionada por mapas digitales.

En el caso del prototipo de gafas inteligentes propuesto en este trabajo, la técnica de ray casting puede emplearse para analizar el entorno inmediato del usuario y detectar obstáculos cercanos. Esta información puede integrarse con datos de navegación provenientes de servicios de mapas digitales [79] para generar instrucciones más seguras y contextualizadas.

Por ejemplo, si el sistema detecta un obstáculo en la trayectoria prevista, puede ajustar las instrucciones de navegación o emitir una alerta para que el usuario modifique su dirección de movimiento.

De esta forma, la integración del ray casting con sensores y sistemas de navegación digital permite construir soluciones de asistencia más robustas, capaces de mejorar la autonomía y seguridad de los usuarios durante sus desplazamientos [75, 78].

Capítulo 4

Marco metodológico

4.1. Enfoque metodológico

El presente trabajo se desarrolla bajo un enfoque aplicado, orientado al diseño e implementación de un prototipo funcional de un sistema de navegación asistida para personas con discapacidad visual. Este tipo de enfoque es adecuado cuando el objetivo principal no es generar conocimiento teórico nuevo, sino producir una solución técnica concreta que pueda ser evaluada y validada en condiciones reales [80].

Dado que el sistema involucra múltiples componentes tecnológicos que deben integrarse de forma coherente, se adopta una **metodología de desarrollo incremental** [81]. Este modelo permite construir y validar cada módulo del sistema de manera progresiva, reduciendo el riesgo de fallos en la integración final y facilitando la detección temprana de problemas en etapas intermedias. A diferencia de un enfoque en cascada, el desarrollo incremental permite retroalimentar el diseño de etapas anteriores conforme avanza la implementación, lo cual es especialmente relevante en proyectos con componentes de percepción e inteligencia artificial, donde el comportamiento del sistema depende fuertemente de la interacción entre módulos [82].

El enfoque combina técnicas de inteligencia artificial, procesamiento de imágenes, planificación de rutas y sistemas de interacción humano-máquina, con el objetivo de construir una solución integral capaz de operar en tiempo real en entornos urbanos y espacios interiores.

4.2. El sistema

El sistema propuesto se basa en una arquitectura modular que permite la integración de múltiples tecnologías para la percepción, representación y navegación del entorno. Este diseño facilita el desarrollo independiente de cada componente, así como su posterior integración en un sistema funcional unificado [83].

La arquitectura se divide en los siguientes módulos principales:

- Adquisición de datos
- Percepción del entorno
- Representación espacial
- Planeación de rutas
- Interacción con el usuario

Cada módulo cumple una función específica dentro del sistema, permitiendo transformar la información capturada del entorno en instrucciones comprensibles para el usuario final.

4.3. Metodología de desarrollo

El desarrollo del sistema se lleva a cabo mediante un enfoque incremental dividido en etapas, permitiendo validar cada componente de forma independiente antes de su integración final. Esta estrategia ha sido ampliamente utilizada en el desarrollo de sistemas embebidos y de asistencia tecnológica, donde la complejidad de los componentes hace inviable una validación conjunta sin haber verificado primero el comportamiento individual de cada módulo [82].

Las etapas principales son:

1. **Adquisición y preprocesamiento de datos:** Captura de información visual y de proximidad del entorno mediante los sensores del sistema. Se aplican técnicas de preprocesamiento para normalizar y acondicionar los datos antes de ser procesados por los módulos posteriores.
2. **Percepción visual del entorno:** Implementación de un módulo de visión por computadora capaz de identificar elementos relevantes del entorno en tiempo real, tales como obstáculos, personas y estructuras de interés.

3. **Inferencia contextual:** Incorporación de un modelo de razonamiento probabilístico para inferir relaciones entre los objetos detectados y mejorar la comprensión del entorno más allá de la detección individual de elementos.
4. **Construcción del mapa:** Generación de una representación espacial del entorno que permita distinguir zonas libres de obstáculos y zonas ocupadas, habilitando la navegación segura del usuario.
5. **Planeación de rutas:** Implementación de un algoritmo de búsqueda de caminos óptimos sobre el mapa generado, permitiendo calcular trayectorias que eviten obstáculos y se adapten a cambios dinámicos en el entorno.
6. **Integración con navegación global:** En entornos exteriores, el sistema se apoya en servicios de cartografía digital para obtener rutas a nivel de ciudad, las cuales son refinadas localmente mediante el módulo de planeación de rutas.
7. **Interfaz de usuario:** Desarrollo de un módulo de generación de lenguaje natural que traduce las instrucciones de navegación en mensajes auditivos claros y comprensibles para el usuario.

4.4. Descripción de módulos

4.4.1. Módulos de percepción

Los módulos de percepción se encargan de adquirir e interpretar la información proveniente del entorno. Esto incluye el procesamiento de las imágenes capturadas por la cámara del dispositivo, la interpretación visual del entorno para detectar objetos y estructuras relevantes para la navegación, y la lectura de los sensores de proximidad que complementan la información visual con datos de distancia. En conjunto, estos submódulos proporcionan al sistema una representación multisensorial del espacio inmediato que rodea al usuario.

4.4.2. Módulos de representación

Los módulos de representación tienen como objetivo construir y mantener un modelo del entorno a partir de la información percibida. Para ello, se implementan mecanismos de localización y mapeo simultáneo que permiten estimar la posición del usuario dentro del espacio, así como técnicas de proyección espacial para generar una representación del entorno en forma de malla de ocupación. Esta representación distingue entre zonas libres y zonas ocupadas, y sirve como base para el módulo de planeación de rutas.

4.4.3. Módulo de navegación y rutas

Este módulo se encarga de calcular y gestionar la trayectoria que debe seguir el usuario para llegar a su destino. Incluye un modelo general de navegación que integra la información local del mapa con rutas a nivel global, un submódulo de dirección que genera instrucciones de movimiento paso a paso, un submódulo de integración con servicios de cartografía para la navegación en exteriores, y un módulo de alerta de obstáculos que detecta y notifica la presencia de elementos que puedan representar un riesgo en la trayectoria calculada.

4.4.4. Módulos de interacción

Los módulos de interacción constituyen la interfaz entre el sistema y el usuario. Comprenden un módulo de reconocimiento de voz que permite al usuario comunicarse con el sistema de forma natural, un módulo de procesamiento de lenguaje natural que interpreta las instrucciones recibidas y genera respuestas adecuadas, y un módulo de salida que transmite las instrucciones de navegación y alertas al usuario mediante síntesis de voz. Esta combinación permite una interacción fluida y accesible sin requerir el uso de interfaces visuales.

4.5. Integración del sistema

La integración del sistema se realiza mediante la conexión secuencial y coordinada de los módulos descritos, permitiendo el flujo continuo de información desde la adquisición de datos hasta la generación de instrucciones para el usuario. El diseño modular facilita esta integración, ya que cada componente expone interfaces bien definidas que permiten sustituir o actualizar implementaciones internas sin afectar al resto del sistema [83].

El sistema opera en tiempo real, procesando continuamente la información del entorno y actualizando la ruta en función de los cambios detectados. Esta integración permite combinar la navegación global y local, proporcionando una solución adaptable a distintos tipos de entornos.

4.6. Pruebas y validación

Para validar el funcionamiento del sistema, se realizarán pruebas en escenarios controlados y entornos reales, siguiendo una estrategia de validación incremental acorde con la metodología de desarrollo adoptada [81]. Los aspectos a evaluar son:

- Precisión en la detección de objetos y obstáculos
- Capacidad de evitar obstáculos de forma segura
- Eficiencia y corrección en la generación de rutas
- Tiempo de respuesta del sistema en condiciones de operación real
- Claridad y comprensibilidad de las instrucciones auditivas

Los resultados obtenidos permitirán evaluar el desempeño global del sistema e identificar áreas de mejora en cada uno de sus componentes antes de considerarlo apto para uso en entornos no controlados.

Capítulo 5

Requerimientos del sistema

El presente capítulo describe los requerimientos que el sistema debe satisfacer para constituir una solución válida al problema de navegación asistida para personas con discapacidad visual. Estos requerimientos se derivan de las necesidades del usuario final, de las restricciones del entorno de operación y de las características de la arquitectura modular definida en el capítulo anterior. Sirven como base directa para el capítulo de análisis, en el cual se justifica la selección de tecnologías a partir de los criterios aquí establecidos.

Los requerimientos se organizan en dos categorías: funcionales, que describen qué debe hacer el sistema, y no funcionales, que definen las restricciones de calidad bajo las cuales debe operar.

5.1. Requerimientos Funcionales

Los requerimientos funcionales se agrupan conforme a la arquitectura modular del sistema, siguiendo la misma organización de módulos descrita en el marco metodológico.

RF – Módulos de interacción

RF-01 **Captura continua de voz con detección de actividad.** El sistema debe capturar la entrada de audio del usuario de forma continua mediante el micrófono del dispositivo y detectar automáticamente los segmentos en los que hay señal de voz activa, descartando los periodos de silencio para optimizar el procesamiento.

RF-02 **Filtrado y normalización de audio.** El sistema debe aplicar supresión de ruido ambiental y normalización de nivel sobre la señal de audio antes de entregarla al módulo de procesa-

miento de lenguaje natural, con el fin de reducir errores de reconocimiento en entornos con ruido de fondo.

RF-03 Reconocimiento de voz e interpretación de intención. El sistema debe transcribir la señal de voz a texto, clasificar la intención del usuario —indicar un destino, solicitar descripción del entorno, controlar la sesión— y extraer las entidades relevantes para ejecutar la acción correspondiente.

RF-04 Síntesis y reproducción de instrucciones por voz. El sistema debe convertir en audio las instrucciones de navegación, las alertas y las respuestas generadas por el sistema, reproduciéndolas al usuario mediante síntesis de voz con lenguaje natural, frases cortas y referencias espaciales claras.

RF-05 Control de sesión por voz. El sistema debe reconocer comandos de voz para iniciar, pausar, reanudar y detener la sesión de navegación, sin requerir ningún tipo de interacción física o visual por parte del usuario.

RF-06 Exposición completa de funciones a través de voz. El módulo de interacción debe exponer la totalidad de las funciones del sistema mediante comandos de voz, de forma que ninguna función quede accesible exclusivamente por medios táctiles o visuales. Esto incluye iniciar y detener la navegación, indicar destinos, solicitar descripciones del entorno y pedir rutas alternativas.

RF – Módulos de percepción

RF-07 Captura continua de imágenes y preprocesamiento. El sistema debe adquirir fotogramas de forma continua mediante la cámara del dispositivo y aplicar operaciones de preprocesamiento —redimensionamiento, normalización e igualación de contraste— para estandarizar la entrada al módulo de visión.

RF-08 Detección de objetos con localización espacial. El módulo de visión debe detectar los objetos relevantes para la navegación presentes en cada fotograma —incluyendo al menos personas, obstáculos estáticos, puertas, escaleras y desniveles— generando para cada uno una región de interés con su nivel de confianza asociado.

RF-09 Clasificación de objetos y estimación de nivel de riesgo. El sistema debe asignar una categoría semántica a cada objeto detectado y determinar su nivel de riesgo para la trayectoria actual, distinguiendo entre elementos que representan un peligro inmediato y elementos que forman parte del entorno navegable sin representar una amenaza.

RF-10 **Estimación de distancia a objetos detectados.** El sistema debe estimar la distancia entre el usuario y cada objeto detectado a partir de la información visual disponible y de la representación espacial del entorno, con el fin de determinar la urgencia de la alerta correspondiente.

RF-11 **Validación cruzada con sensores de proximidad.** El sistema debe contrastar la información visual con las mediciones de los sensores de proximidad del dispositivo para confirmar o descartar la presencia de obstáculos, reduciendo la tasa de falsas alarmas antes de notificar al módulo de navegación.

RF-12 **Generación y emisión de alerta de obstáculo.** Al confirmar la presencia de un obstáculo relevante, el sistema debe notificar al usuario por voz de forma inmediata, especificando la naturaleza del obstáculo y su posición relativa, y enviar las coordenadas y el nivel de riesgo al módulo de navegación para desencadenar el recálculo de ruta si corresponde.

RF-13 **Inferencia de relaciones contextuales entre objetos.** El sistema debe inferir relaciones espaciales y contextuales entre los elementos detectados —como el estado de una puerta o el grado de obstrucción de un pasillo— para enriquecer la representación del entorno más allá de la detección individual de objetos.

RF – Módulos de representación

RF-14 **Estimación continua de la posición del usuario.** El sistema debe mantener y actualizar de forma continua una estimación de la posición y orientación del usuario dentro del entorno, integrando el movimiento detectado con la información perceptual disponible.

RF-15 **Construcción incremental del mapa del entorno.** El sistema debe construir y actualizar progresivamente un mapa del entorno a medida que el usuario se desplaza, sin requerir un mapa previo del espacio como condición para operar.

RF-16 **Representación de zonas libres y ocupadas.** El mapa generado debe distinguir explícitamente entre zonas navegables y zonas ocupadas por obstáculos, proporcionando al módulo de planeación una estructura de datos adecuada para calcular trayectorias seguras.

RF-17 **Actualización dinámica del mapa.** El mapa del entorno debe actualizarse en respuesta a los cambios detectados, reflejando la aparición o desaparición de obstáculos de forma que la representación espacial corresponda siempre al estado actual del entorno. La frecuencia mínima de esta actualización se establece en RNF-03.

RF – Módulo de navegación y rutas

RF-18 Cálculo de ruta óptima local. El módulo de navegación debe calcular una trayectoria que conecte la posición actual del usuario con su destino, minimizando el costo del recorrido y garantizando que la ruta propuesta evite todas las zonas ocupadas registradas en el mapa.

RF-19 Actualización periódica de la ruta durante la navegación. El sistema debe recalcular y actualizar la ruta activa de forma periódica durante una sesión de navegación, incorporando los cambios detectados en el mapa para mantener la trayectoria vigente coherente con el estado actual del entorno. La frecuencia mínima de este recálculo se establece en RNF-04.

RF-20 Recálculo de ruta ante obstáculos confirmados. Cuando el módulo de percepción confirme la presencia de un obstáculo en la trayectoria activa, el módulo de navegación debe generar una ruta alternativa de forma inmediata, sin esperar al ciclo periódico de actualización establecido en RNF-04.

RF-21 Generación de instrucciones de movimiento. El módulo de dirección debe traducir cada punto de la ruta calculada en una instrucción de movimiento discreta —avanzar, girar a la derecha, girar a la izquierda, detenerse— expresada en términos de distancia y ángulo comprensibles, y entregarla al módulo de salida para su emisión por voz.

RF-22 Integración con cartografía global en exteriores. En entornos exteriores, el sistema debe obtener una ruta a nivel de ciudad a través de un servicio de cartografía digital, integrando esa información con la navegación local para guiar al usuario desde su ubicación actual hasta un destino arbitrario.

RF-23 Solicitud de ruta alternativa por voz. El usuario debe poder solicitar por voz una ruta alternativa en cualquier momento de la navegación. El sistema debe analizar su viabilidad y aplicarla, o notificar al usuario si no existe una alternativa disponible.

5.2. Requerimientos No Funcionales

Los requerimientos no funcionales establecen los criterios de calidad bajo los cuales el sistema debe operar. Son determinantes en la selección de tecnologías, ya que imponen restricciones concretas sobre el rendimiento, la precisión, la portabilidad y la seguridad de cada módulo.

RNF – Rendimiento

RNF-01 Latencia de detección y alerta de obstáculos. El sistema debe detectar, clasificar y emitir la alerta de un obstáculo en un tiempo máximo de 2 segundos desde la captura del fotograma en el que éste aparece, para garantizar que la notificación al usuario sea útil antes de que la situación represente un riesgo.

RNF-02 Latencia de reconocimiento de voz. El sistema debe procesar e interpretar un comando de voz en menos de 500 ms desde que el usuario termina de hablar, de forma que la respuesta o acción resultante no interrumpa el flujo natural de la interacción.

RNF-03 Frecuencia de actualización del mapa. La representación espacial del entorno debe actualizarse a una frecuencia mínima de 10 Hz, de modo que los cambios en el entorno queden reflejados en el mapa sin un desfase perceptible entre la posición real del usuario y su posición estimada en el mapa.

RNF-04 Frecuencia de actualización de la ruta activa. La ruta de navegación debe recalcularse al menos una vez cada diez segundos durante una sesión activa, garantizando que la trayectoria incorpore los cambios recientes del entorno sin imponer una carga computacional continua al sistema. Este valor constituye la restricción temporal asociada al comportamiento funcional descrito en RF-19.

RNF-05 Eficiencia computacional en hardware limitado. El sistema debe poder ejecutarse en hardware de propósito general con recursos limitados, como los disponibles en dispositivos móviles o sistemas embebidos de bajo costo —por ejemplo, una Raspberry Pi 4 o equivalente— cumpliendo los tiempos de respuesta establecidos sin requerir infraestructura de cómputo en la nube para su operación básica.

RNF – Precisión y confiabilidad

RNF-06 Precisión en la detección de obstáculos. El módulo de visión debe alcanzar una tasa de detección correcta mínima del 90 % bajo condiciones de iluminación normales, manteniendo la tasa de falsas alarmas por debajo del 5 % para evitar interrupciones innecesarias durante la navegación.

RNF-07 Tasa de reconocimiento correcto de comandos de voz. El módulo de procesamiento de lenguaje natural debe interpretar correctamente al menos el 92 % de los comandos emitidos en condiciones de ruido ambiental de hasta 60 dB.

RNF-08 Disponibilidad y degradación controlada. El sistema debe permanecer operativo durante toda la sesión de navegación sin reinicios no solicitados. Ante el fallo de un módulo no crítico,

debe degradarse de forma controlada, notificando al usuario la pérdida de funcionalidad y manteniendo operativas las funciones restantes.

RNF-09 Comportamiento conservador ante incertidumbre. Ante cualquier situación de baja confianza en la información del entorno —como detección con umbral bajo o pérdida de señal de posicionamiento— el sistema debe optar por la acción más conservadora: advertir al usuario y sugerir detenerse hasta recuperar información confiable. Es preferible una alerta de precaución adicional que omitir un obstáculo real.

RNF – Usabilidad y accesibilidad

RNF-10 Interfaz exclusivamente por voz. El sistema debe poder operarse en su totalidad mediante comandos de voz, sin requerir interacción táctil o visual por parte del usuario durante la navegación. Esta restricción aplica a la experiencia completa del usuario y complementa lo establecido en RF-06, que define la responsabilidad funcional del módulo de interacción.

RNF-11 Claridad y comprensibilidad de las instrucciones. Las instrucciones generadas por el módulo de salida deben utilizar lenguaje natural, frases cortas y referencias espaciales concretas —izquierda, derecha, metros— de forma que sean comprensibles sin ambigüedad para un usuario sin conocimientos técnicos y sin entrenamiento previo en el uso del sistema.

RNF-12 Retroalimentación proactiva ante eventos relevantes. El sistema debe emitir retroalimentación auditiva de forma proactiva ante eventos significativos —detección de un obstáculo, cambio de ruta, pérdida de señal— sin esperar a que el usuario lo solicite explícitamente.

RNF – Portabilidad y mantenibilidad

RNF-13 Operación autónoma sin conexión a internet. Los módulos de percepción, representación, planeación local e interacción deben funcionar de forma autónoma sin requerir conexión a internet. La navegación global en exteriores puede depender de conectividad, pero el sistema debe degradar su funcionamiento de forma controlada en ausencia de conexión, manteniendo operativas las funciones locales.

RNF-14 Modularidad e independencia entre componentes. Los módulos de voz, visión, representación y navegación deben estar desacoplados e intercomunicados mediante interfaces bien definidas, de modo que cualquiera de ellos pueda ser modificado o sustituido sin afectar el funcionamiento de los demás, siempre que se respeten las interfaces establecidas.

RNF-15 **Facilidad de actualización de modelos.** La arquitectura debe permitir sustituir los modelos de percepción o inferencia por versiones mejoradas sin requerir cambios estructurales en el resto del sistema, de forma que el desempeño pueda actualizarse conforme avance el estado del arte en las tecnologías empleadas.

Capítulo 6

Análisis y diseño

6.1. Casos de Uso

En esta sección se describen los casos de uso del sistema propuesto, los cuales representan las principales interacciones entre el usuario y la solución de navegación asistida desarrollada. Los casos de uso permiten identificar las funcionalidades clave del sistema desde una perspectiva externa, enfocándose en las acciones que el usuario puede realizar y las respuestas que el sistema proporciona.

El sistema está diseñado para asistir a personas con discapacidad visual durante su desplazamiento en entornos urbanos o interiores, mediante la generación de instrucciones auditivas, la detección de obstáculos y la adaptación dinámica de rutas. En este contexto, los casos de uso modelan el flujo de interacción necesario para lograr una navegación segura y eficiente.

Los actores principales identificados son el **usuario**, quien interactúa mediante comandos de voz, y el **sistema**, el cual procesa la información del entorno y genera las respuestas correspondientes. A partir de esta interacción, se definen los distintos escenarios funcionales que permiten cumplir con los objetivos del sistema. Los casos de uso considerados son los siguientes:

Activar el sistema

Este caso de uso permite iniciar el funcionamiento del sistema de navegación mediante la activación del dispositivo. El usuario presiona el botón de activación, el sistema se inicia, se activan los sensores, cámara y el modulo de reconocimiento de voz, el sistema queda listo para recibir instrucciones y el resultado esperado es que el sistema queda preparado para comenzar la navegación.

Detener la navegación

Permite finalizar el proceso de navegación y detener las instrucciones auditivas emitidas por el sistema. El usuario solicita detener la navegación, el sistema cancela la ruta activa y la navegación finaliza correctamente.

Indicar destino

El usuario proporciona al sistema el lugar al que desea dirigirse. El usuario menciona el destino mediante comandos de voz, el sistema procesa la información recibida, identifica el destino solicitado, el sistema procede a la confirmación del destino. Así, el sistema obtiene correctamente el destino deseado.

Confirmar destino

El sistema valida que el destino interpretado sea correcto antes de iniciar la navegación. El sistema comunica el destino detectado, el usuario confirma la información, si la información es correcta, empieza la navegación.

Emitir instrucciones de navegación

El sistema proporciona instrucciones auditivas durante el recorrido hacia el destino seleccionado. El sistema calcula la ruta, se generan instrucciones de navegación y el usuario recibe indicaciones auditivas. El resultado esperado es que el usuario recibe orientación continua durante el trayecto.

Detectar y alertar obstáculos

El sistema detecta obstáculos durante el desplazamiento y emite alertas preventivas al usuario. Los sensores y cámaras monitorean el entorno, el sistema identifica posibles obstáculos, se genera una alerta auditiva y el resultado esperado es que el usuario evite colisiones.

Recalcular ruta

Permite recalcular una nueva ruta cuando el camino original presenta inconvenientes. El sistema detecta un problema en la ruta actual, se calcula una ruta alternativa o se espera el guardado de una ruta nueva y el resultado es la navegación continúa mediante una ruta alternativa segura.

Confirmar nueva ruta

El usuario valida si desea continuar mediante la nueva ruta propuesta por el sistema. El sistema informa la existencia de una nueva ruta, el usuario acepta o rechaza la alternativa; si acepta, el sistema actualiza la navegación, el resultado es la ruta alternativa confirmada y guardada.

Finalizar navegación

El sistema informa al usuario que ha llegado correctamente al destino seleccionado. El sistema detecta la llegada al destino, se emite una notificación auditiva y finaliza el proceso de navegación.

6.1.1. Diagrama de casos de uso

El diagrama de casos de uso mostrado en la Figura 6.1 representa de manera gráfica las interacciones entre el usuario y el sistema, así como la relación entre los distintos casos de uso identificados.

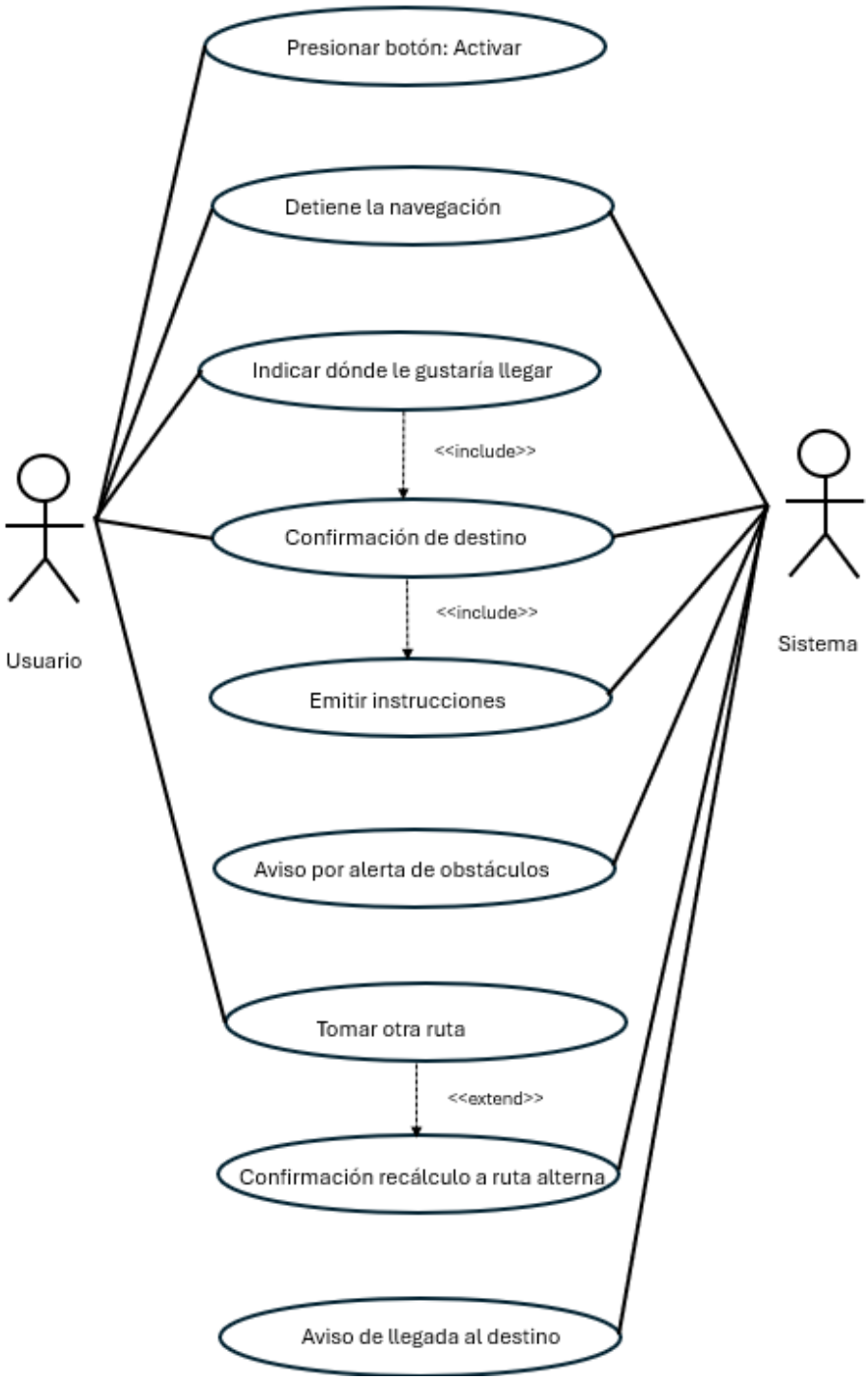


Figura 6.1: Diagrama de casos de uso del sistema de navegación asistida

Este modelo permite identificar de forma clara las funcionalidades del sistema, así como las dependencias y comportamientos alternativos que pueden surgir durante su operación.
El actor principal es el Usuario, quien interactúa directamente con el sistema mediante distintas funcionalidades. Por su parte, el Sistema actúa como el ente que procesa la información, toma decisiones y proporciona retroalimentación.

El usuario tiene la capacidad de activar el sistema “Presionar botón: Activar”. Este caso de uso representa la acción inicial que habilita la interacción con el sistema. Asimismo, el usuario puede detener la navegación en cualquier momento a través del caso de uso “Detiene la navegación”, lo que evidencia que el sistema debe permitir el control total sobre su ejecución.

“Indicar dónde le gustaría llegar”, donde el usuario proporciona el destino deseado. Este caso de uso describe una interacción directa en la que el sistema debe ser capaz de recibir y procesar la entrada del usuario. Asociado a este, se encuentra el caso de uso “Confirmación de destino”, el cual corresponde a una responsabilidad del sistema: validar y confirmar la información recibida. La relación «include» entre ambos indica que la confirmación es un comportamiento obligatorio que forma parte del servicio principal de captura de destino.

Una vez que el destino ha sido definido, el sistema ofrece la funcionalidad de “Emitir instrucciones”, que consiste en proporcionar indicaciones al usuario para llegar a su destino. Este caso de uso también está relacionado mediante «include», lo que indica que depende de la confirmación previa del destino para ejecutarse correctamente.

Durante la interacción, el sistema incluye la capacidad de “Aviso por alerta de obstáculos”, lo cual representa una funcionalidad autónoma del sistema orientada a la seguridad del usuario. En este caso, el usuario no necesariamente inicia la acción, sino que el sistema actúa de manera proactiva notificando eventos relevantes del entorno.

“Tomar otra ruta”, está asociado a una relación «extend» con “Confirmación recálculo a ruta alterna”. Esta relación indica que se trata de una funcionalidad opcional que solo se activa bajo ciertas condiciones, como la detección de obstáculos o cambios en el entorno. En este contexto, el sistema asume la responsabilidad de recalcular una nueva ruta, mientras que el usuario participa de forma indirecta al recibir la confirmación de dicho ajuste.

El caso de uso “Aviso de llegada al destino”, representa la notificación al usuario de que se ha cumplido el objetivo de la navegación. Este caso de uso corresponde completamente al sistema, quien debe identificar el cumplimiento de la meta y comunicarlo oportunamente.

6.2. Diagramas de flujo

Con el objetivo de representar de manera detallada la secuencia operativa interna de este sistema, se emplea un diagrama de flujo, el cual describe los procesos, decisiones y ciclos que se ejecutan desde la activación del sistema hasta la llegada al destino.

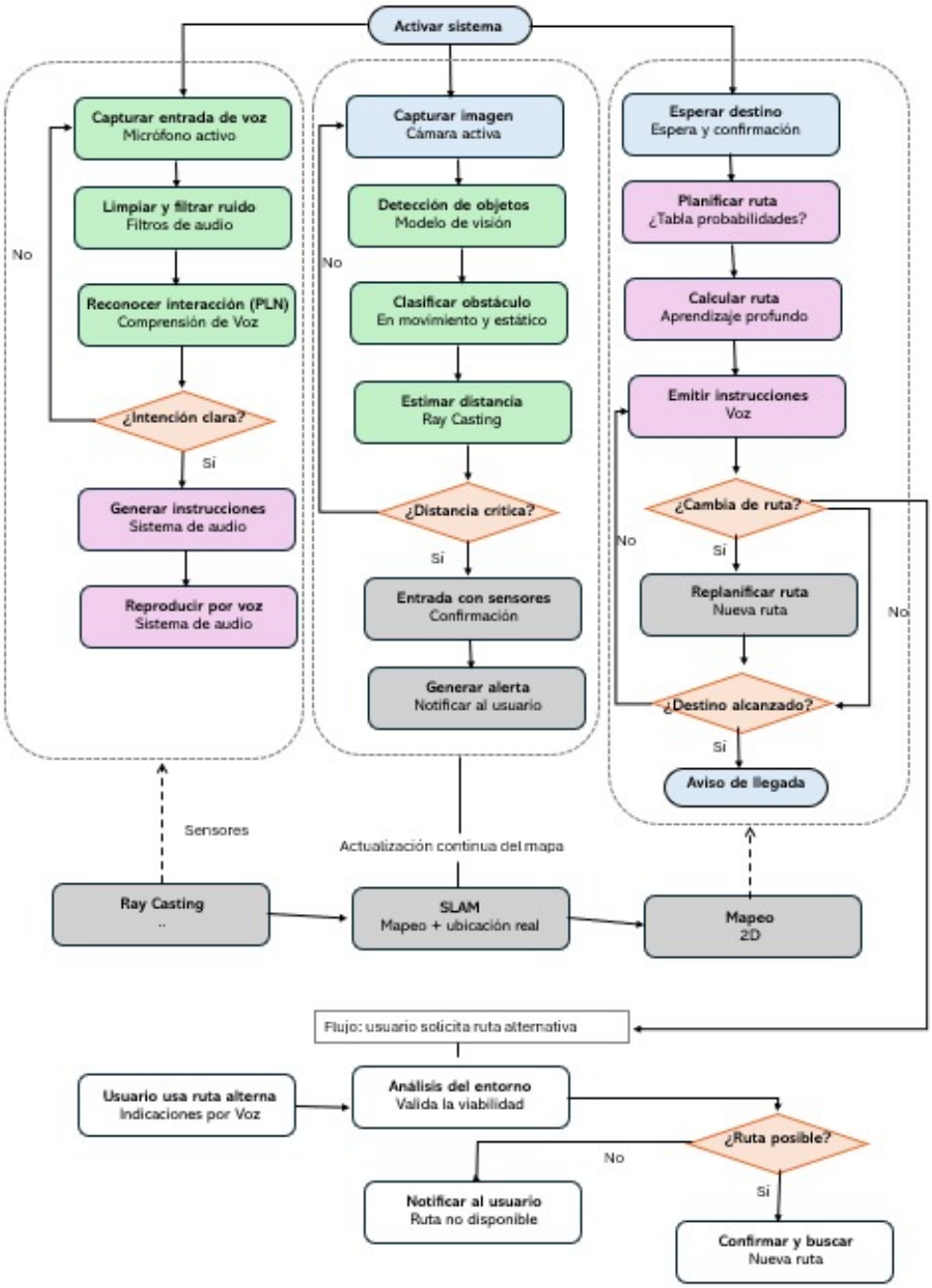


Figura 6.2: Diagrama de flujo global del sistema

Módulo 1 - Voz El sistema comienza capturando la entrada del usuario mediante el bloque "Capturar entrada de voz", donde se utiliza el micrófono para recibir comandos. Posteriormente, la señal pasa por el proceso de "Limpiar y filtrar ruido", en el cual se aplican técnicas de procesamiento de audio para mejorar la calidad de la señal.

Una vez depurada, la información es interpretada en "Reconocer interacción (PLN)", donde se emplean modelos de procesamiento de lenguaje natural para comprender la intención del usuario.

Se presenta una decisión clave: “¿Intención clara?”.
Si la respuesta es afirmativa, el sistema procede a “Generar instrucciones” y posteriormente a “Reproducir por voz”, proporcionando retroalimentación auditiva al usuario. En caso contrario, el sistema puede requerir nueva entrada o repetir el proceso de interpretación.
Este módulo es fundamental, ya que permite una interacción natural y accesible entre el usuario y el sistema.

Módulo 2 - Visión
De manera paralela, el sistema activa la cámara en el bloque “Capturar imagen”, obteniendo información visual del entorno. Esta información es procesada en “Detección de objetos”, donde un modelo de visión identifica elementos relevantes.
Posteriormente, en “Clasificar obstáculo”, el sistema determina si los objetos detectados representan un riesgo, diferenciando entre obstáculos móviles y estáticos.
El siguiente paso es “Estimar distancia”, donde se calcula la proximidad de los objetos mediante técnicas como Ray Casting.
Esto conduce a la decisión “¿Distancia crítica?”:
Si se detecta un riesgo cercano, el sistema ejecuta “Entrada con sensores” para validar la información y posteriormente “Generar alerta”, notificando al usuario. Si no existe riesgo, el sistema continúa monitoreando el entorno sin interrumpir la navegación.
Este módulo garantiza la seguridad del usuario mediante análisis en tiempo real del entorno.

Módulo 3 - Ruta En este módulo, el sistema puede encontrarse inicialmente en el estado “Esperar destino”, hasta recibir la confirmación del usuario. Una vez definido, se ejecuta “Planificar ruta”, donde se evalúan posibles trayectorias.
Posteriormente, en “Calcular ruta”, se emplean algoritmos de aprendizaje profundo para determinar la mejor opción disponible. El resultado se comunica mediante “Emitir instrucciones”, guiando al usuario durante su desplazamiento.
Durante la navegación, el sistema evalúa continuamente la condición “¿Cambia de ruta?”:
Si no hay cambios, el sistema sigue emitiendo instrucciones. Si ocurre un evento (por ejemplo, un obstáculo), se activa “Replanificar ruta”, generando una nueva trayectoria.
Finalmente, se evalúa “¿Destino alcanzado?”:
Si la respuesta es afirmativa, el sistema ejecuta “Aviso de llegada”, concluyendo el proceso. En caso contrario, el ciclo de navegación continúa.

Sensores y mapeo Se presenta: SLAM (Mapeo y ubicación en tiempo real) Mapeo 2D Ray Casting
Estos elementos permiten la actualización continua del entorno, proporcionando información clave a los módulos de visión y planificación. Gracias a este sistema, el modelo puede tomar decisiones informadas en tiempo real.

Se contempla un flujo adicional cuando el usuario decide cambiar la ruta. Este proceso inicia en "Usuario usa ruta alterna", seguido de un "Análisis del entorno" para validar la viabilidad.
Se evalúa la decisión "¿Ruta posible?":
Si la ruta es viable, el sistema procede a "Confirmar y buscar nueva ruta". En caso contrario, se ejecuta "Notificar al usuario ruta no disponible", informando la imposibilidad del cambio.

6.2.1. Flujo de percepción

Para los modulos que ayudan al sistema a percibir el mundo como el de Visión y el de los sensores, el flujo es: El proceso interno del sistema para la detección, análisis y validación de obstáculos, integrando información proveniente de visión por computadora y sensores físicos. Su objetivo principal es garantizar que las decisiones de navegación se basen en información confiable, reduciendo falsos positivos y aumentando la seguridad del usuario.

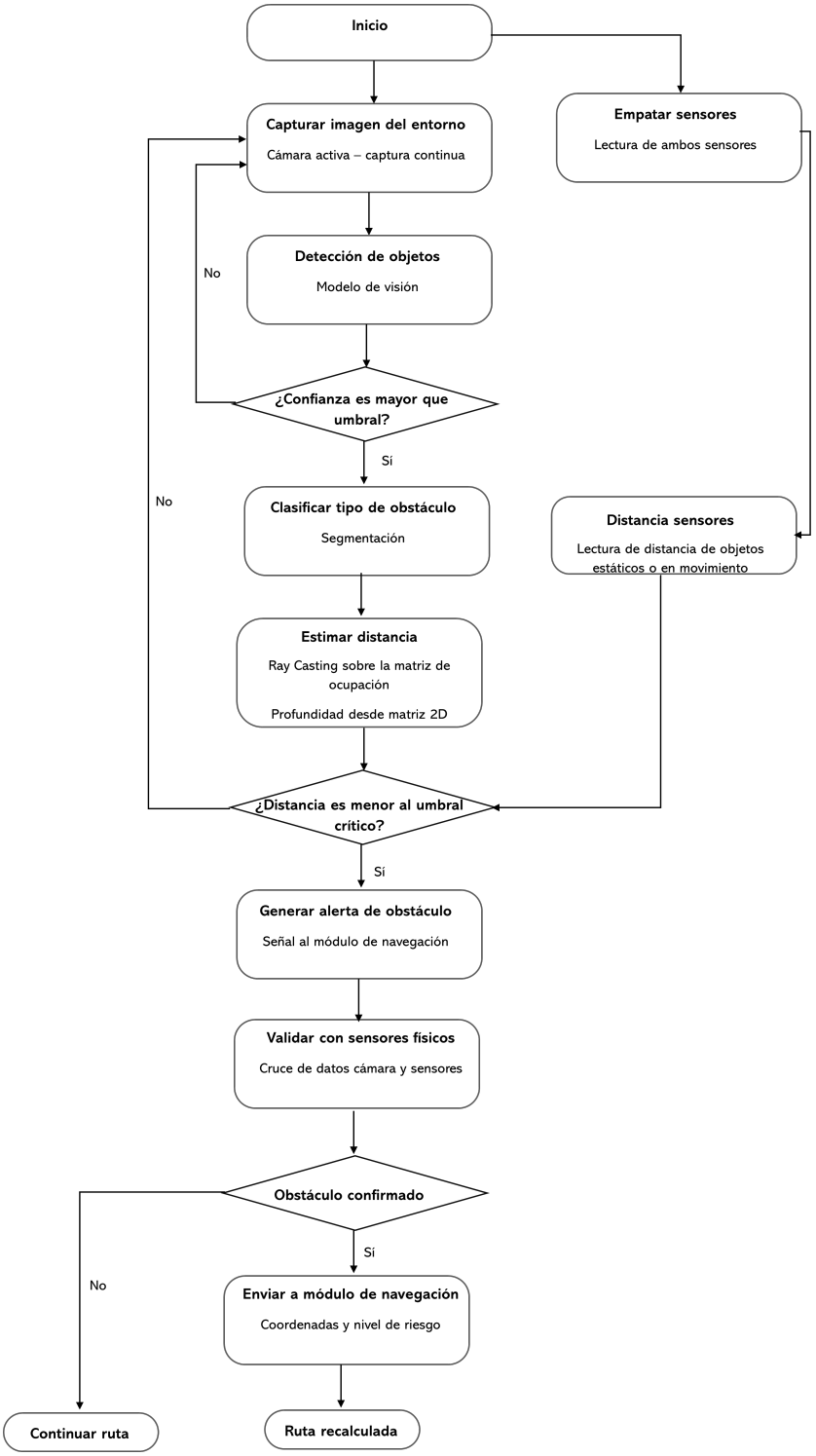


Figura 6.3: Diagrama de flujo del Modulo de Vision y Modulo sensorial

El flujo inicia a partir de dos procesos fundamentales: Captura de imagen mediante una cámara activa y la entrada de los sensores, donde se realiza la lectura simultánea de los sensores físicos para sincronizar la información.

Una vez capturada la imagen, el sistema ejecuta el proceso de “Detección de objetos”, utilizando un modelo de visión por computadora para identificar elementos presentes en el entorno.

Posteriormente, se evalúa la condición “¿Confianza es mayor que umbral?”, la cual determina si la detección es suficientemente confiable: Si la confianza no supera el umbral, el sistema descarta la detección y regresa al proceso de captura. Si la confianza es adecuada, el flujo continúa con el análisis del objeto detectado.

Cuando la detección es válida, el sistema procede a “Clasificar tipo de obstáculo”, diferenciando entre objetos estáticos y dinámicos mediante técnicas de segmentación. “Estimar distancia”, utiliza Ray Casting sobre los objetos.

Los sensores físicos aportan información mediante el bloque “Distancia sensores”, proporcionando una medición adicional independiente de la visión. Con la información obtenida, se evalúa la condición “¿Distancia es menor al umbral crítico?”: Si no se encuentra dentro de un rango de riesgo, el sistema continúa monitoreando sin generar intervención. Si la distancia es crítica, se activa el proceso de respuesta.

En este caso, el sistema ejecuta “Generar alerta de obstáculo”, enviando una señal al módulo de navegación para advertir la presencia de un posible riesgo.

“Validar con sensores físicos”, donde se cruzan los datos obtenidos por la cámara y los sensores. A continuación, se evalúa la decisión “Obstáculo confirmado”: Si no se confirma, el sistema descarta la alerta y continúa con la navegación normal.

Una vez validado el obstáculo se ejecuta “Enviar a módulo de navegación”, proporcionando las coordenadas del obstáculo y nivel de riesgo.

Con esta información, el sistema de navegación puede tomar decisiones informadas, como ajustar la trayectoria y recalcular la ruta.

Esto se refleja en el resultado “Ruta recalculada”. En caso de que no se confirme el obstáculo, el flujo concluye en “Continuar ruta”, manteniendo la trayectoria actual.

6.2.2. Flujo de Navegación

En este diagrama se explica el flujo de la navegación que se llevará a cabo.

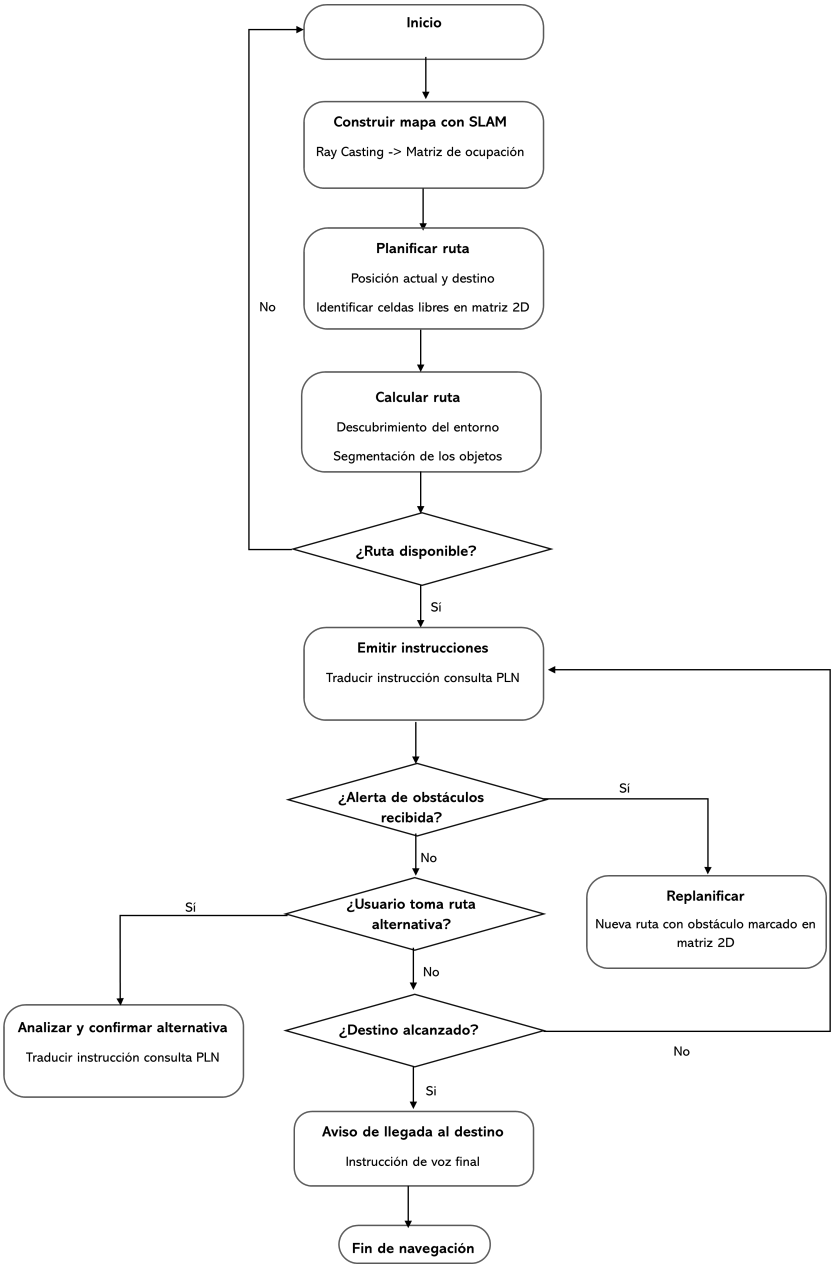


Figura 6.4: Diagrama de flujo del Modulo de Visión y Modulo sensorial

El flujo inicia en el bloque Inicio y avanza hacia la construcción del entorno espacial mediante SLAM (Simultaneous Localization and Mapping), donde el sistema aplica Ray Casting para generar una matriz de ocupación 2D.

Con el mapa listo, el sistema ejecuta la planificación de ruta, identificando la posición actual, el destino y las celdas libres en la cuadrícula.

A partir de esto, calcula la ruta mediante descubrimiento del entorno y segmentación de objetos. Si no existe una ruta disponible, el flujo regresa a planificar nuevamente. Cuando sí hay una ruta,

se evalúa la condición ¿Ruta disponible? y el sistema procede a emitir instrucciones traducidas mediante procesamiento de lenguaje natural (PLN).

Desde ese punto se abre un ciclo de navegación activa: Si llega una alerta de obstáculo, el sistema pasa a Replanificar, marcando el obstáculo en la matriz 2D y generando una nueva ruta, para luego volver a emitir instrucciones.

Si no hay alerta pero el usuario toma una ruta alternativa, el sistema la analiza y confirma mediante PLN.

Si no hay alerta y el usuario sigue la ruta, se evalúa si el destino fue alcanzado: si no, el ciclo regresa a emitir instrucciones; si sí, se emite un aviso de llegada por voz y el flujo concluye en Fin de navegación.

6.2.3. Flujo de la Interacción

Aquí se muestra como se comporta el flujo de los módulos de Reconocimiento de Voz, PLN y salida al usuario, permitiendo la interacción natural entre el usuario y el sistema de navegación asistida.

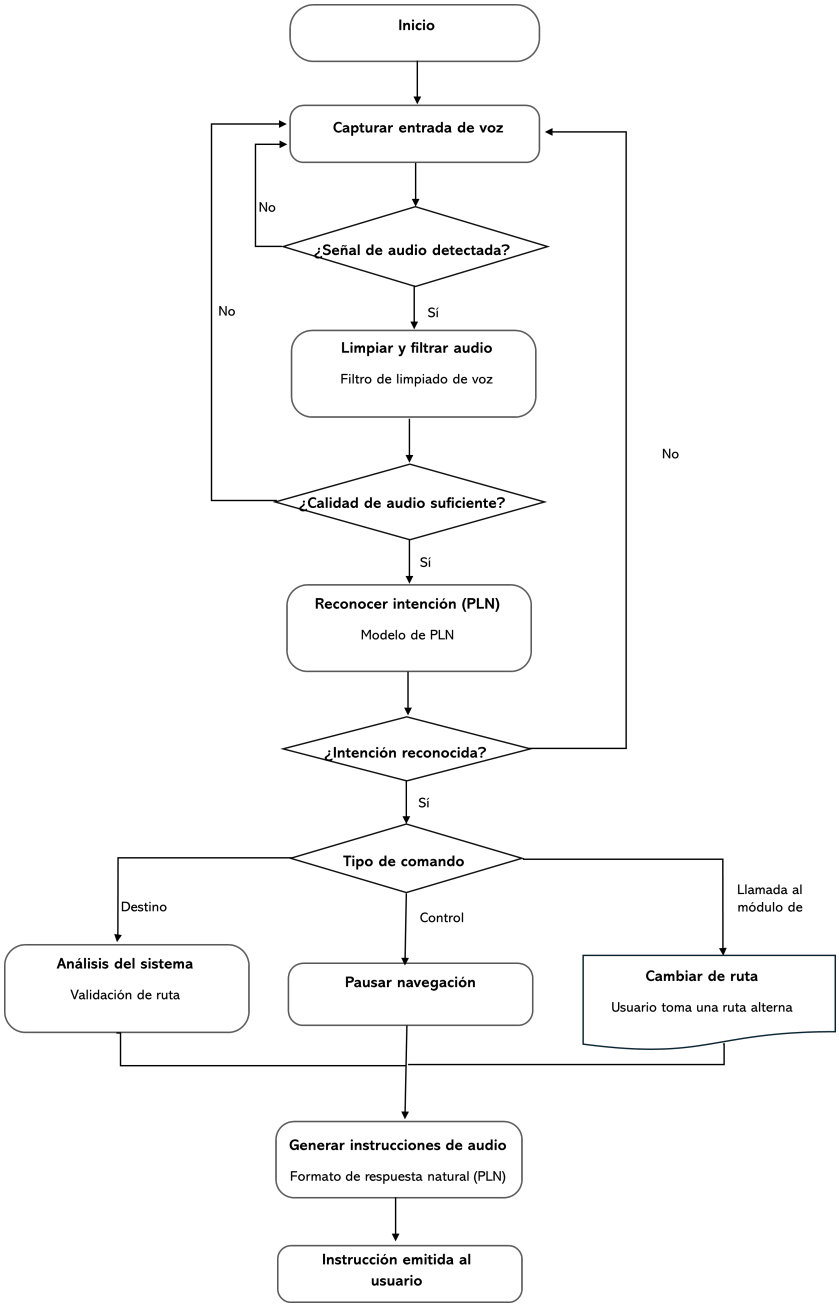


Figura 6.5: Diagrama de flujo del Modulo de Lenguaje Natural y Reconocimiento de voz

El proceso inicia en el bloque “Inicio”, seguido de “Capturar entrada de voz”, donde el sistema activa el micrófono para adquirir señales de audio de manera continua. Se evalúa la condición “¿Señal de audio detectada?”: Si no se detecta señal, el sistema permanece en estado de espera y continúa monitoreando la entrada de audio. Si se detecta señal, el flujo avanza al procesamiento de la misma.

Una vez detectada la señal, el sistema ejecuta el bloque “Limpiar y filtrar audio”, donde se aplican técnicas de reducción de ruido y mejora de calidad. Se evalúa “¿Calidad de audio suficiente?”:

Si la calidad no es adecuada, el sistema descarta la señal y regresa a la captura, evitando interpretaciones erróneas. Si la calidad es suficiente, el flujo continúa hacia el análisis semántico. Este paso es fundamental para garantizar la confiabilidad del reconocimiento.

El sistema procesa la señal mediante el bloque “Reconocer intención (PLN)”, utilizando un modelo de procesamiento de lenguaje natural para interpretar el contenido del mensaje y se evalúa la condición “¿Intención reconocida?”: Si no se reconoce la intención, el sistema vuelve al estado de captura, solicitando implícitamente una nueva entrada. Si la intención es reconocida, el flujo continúa hacia la clasificación del comando.

Una vez comprendida la intención, el sistema clasifica la solicitud en el bloque “Tipo de comando”, distinguiendo tres puntos principales:

1. Comandos de destino: Se dirigen al bloque “Análisis del sistema”, donde se realiza la validación de la ruta hacia el destino solicitado. Este proceso implica verificar la viabilidad del trayecto y preparar la navegación.

2. Comandos de control: Incluyen acciones como “Pausar navegación”, permitiendo al usuario modificar el estado del sistema en tiempo real.

3. Cambio de ruta: Corresponde al bloque “Cambiar de ruta”, donde el usuario solicita una alternativa. Este caso implica la comunicación con el módulo de navegación para recalcular el trayecto.

Independientemente del tipo de comando, el sistema procede al bloque “Generar instrucciones de audio”, donde se construye una respuesta en lenguaje natural mediante técnicas de PLN.

Finalmente, se ejecuta “Instrucción emitida al usuario”, proporcionando retroalimentación auditiva clara y comprensible.

6.3. Arquitectura

A continuación se describe la arquitectura del sistema propuesto, la cual define los módulos que lo componen, su función dentro del flujo de procesamiento y la manera en que interactúan entre sí para lograr una navegación asistida coherente y en tiempo real.

6.3.1. Diagrama de arquitectura

El siguiente diagrama presenta la arquitectura modular del sistema de navegación asistida, mostrando cómo interactúan sus tres fuentes de entrada principales: la voz del usuario, la cámara y los sensores físicos.

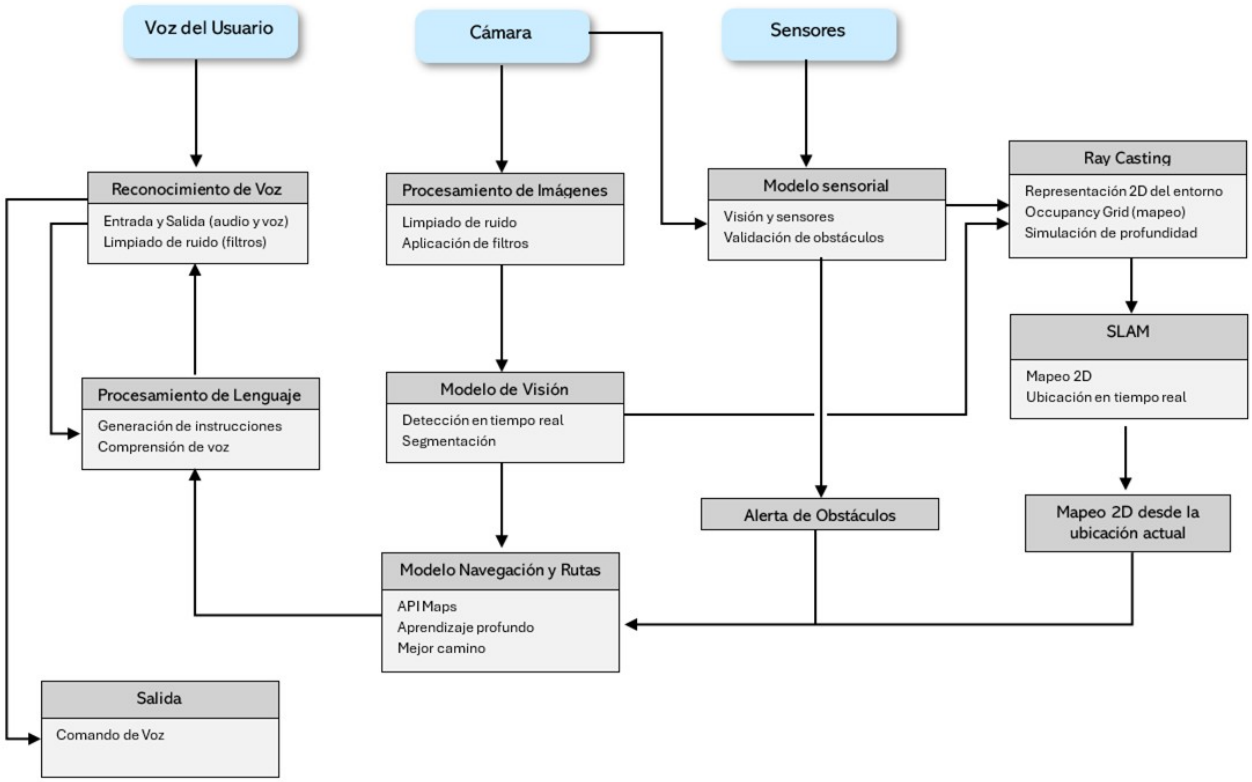


Figura 6.6: Diagrama de arquitectura general del sistema

El sistema tiene tres entradas principales: voz, cámara y sensores. La cámara pasa por procesamiento de imagen y luego por el modelo de visión, cuya tarea clave es la segmentación: detectar objetos y extraer sus coordenadas en el espacio (básicamente "hay un obstáculo en X, Y"). Esas coordenadas son una de las dos entradas más importantes al módulo de navegación.
La entrada dada por el mapeo es resultante de la cadena generada por SLAM donde se obtiene un mapa en 2D con ubicación actual, el flujo se basa en los sensores y el Ray Casting que construye un Occupancy Grid, es decir, una rejilla que indica las celdas libres y ocupadas. Esa matriz 2D es literalmente el terreno sobre el que se calcula la ruta.
El módulo de navegación y rutas recibe 3 entradas:

- La matriz 2D con el mapa del entorno
- Las coordenadas de objetos que detectó el modelo de visión (la segmentación)

- Múltiples rutas basadas en la probabilidad de pesos que le asignan costo o preferencia a distintos caminos, así el sistema no solo busca el camino más corto geométricamente sino el más probable o seguro según la posibilidad de caminos que ha evaluado previamente el modelo.

Con esas tres entradas se calcula la mejor ruta.

Los sensores son alarmas en las que si detectan un obstáculo inesperado, generan una alerta que va directo al módulo de navegación para forzar un recalculado de la ruta, esto con el fin de evitar obstáculos. Esto es independiente del flujo normal, es una interrupción reactiva.
La salida es dada con la ruta calculada, el sistema extrae las instrucciones paso a paso y las va entregando como salida de voz en tiempo real apoyado en la salida dada por el procesamiento de lenguaje que representa el sistema, también actualiza su estado interno conforme avanza, para dar las instrucciones adecuadas.

6.4. Análisis de Requerimientos

El presente capítulo tiene como objetivo establecer el puente entre los requerimientos definidos en el capítulo anterior y las decisiones tecnológicas que dan forma a la implementación del sistema. Cada tecnología elegida responde a uno o más requerimientos concretos y se justifica con base en los fundamentos teóricos estudiados en el marco teórico. El análisis se organiza por agrupación de módulos, siguiendo la misma estructura arquitectónica del sistema.

Es importante señalar que la selección no se basa únicamente en la capacidad funcional de cada tecnología, sino también en su viabilidad dentro de las restricciones establecidas por los requerimientos no funcionales: operación en hardware embebido de recursos limitados (RNF-05), operación autónoma sin conexión a internet (RNF-13) y latencias de respuesta acotadas (RNF-01, RNF-02).

6.4.1. Módulos de percepción

Los módulos de percepción son el punto de entrada de toda la información que el sistema procesa. Su función es transformar señales físicas —fotogramas de cámara y lecturas de sensores— en representaciones computacionales útiles para los módulos posteriores. Los requerimientos que gobiernan esta capa son especialmente exigentes en términos de latencia (RNF-01), precisión (RNF-06) y eficiencia computacional (RNF-05), lo que reduce significativamente el espacio de opciones tecnológicas viables.

Preprocesamiento de imágenes: OpenCV y NumPy

El requerimiento RF-07 establece que el sistema debe adquirir fotogramas de forma continua y aplicar operaciones de preprocesamiento antes de alimentar el módulo de visión. Según la literatura, el preprocesamiento transforma los datos visuales sin procesar en un formato adecuado para el análisis posterior, eliminando distorsiones y resaltando las características relevantes [38]. Operaciones como redimensionamiento, normalización e igualación de contraste son necesarias porque un modelo de detección produce resultados más estables cuando recibe entradas homogéneas, reduciendo la variabilidad innecesaria debida a condiciones de iluminación o escala.

Para esta etapa se elige **OpenCV** (`cv2`) en combinación con **NumPy**. OpenCV es una biblioteca de código abierto especializada en visión por computadora que implementa de forma eficiente las operaciones de preprocesamiento requeridas, y NumPy proporciona la estructura de datos matricial sobre la que se representan las imágenes como arreglos numéricos. Ambas bibliotecas están ampliamente documentadas, tienen una implementación madura y pueden ejecutarse sin aceleración GPU, lo cual es indispensable dado que el hardware objetivo es una plataforma embebida (RNF-05). Además, al operar localmente sin dependencia de servicios externos, se garantiza el cumplimiento de RNF-13.

Detección de objetos: YOLOv8

El requerimiento central de los módulos de percepción es RF-08, que exige detectar objetos relevantes para la navegación en cada fotograma con su localización espacial y nivel de confianza. Este requerimiento se complementa con RF-09, que requiere asignar a cada objeto una categoría semántica y un nivel de riesgo. Dado que el ciclo completo desde la captura hasta la alerta no debe superar los 300 ms (RNF-01) y que la tasa de detección correcta debe ser de al menos el 90 % (RNF-06), se necesita un modelo que combine velocidad y precisión.

Como se estudió en el marco teórico, los detectores de una sola etapa como YOLO realizan la extracción de características y la predicción de objetos en un único paso, lo que les permite procesar imágenes con mayor velocidad que los detectores de dos etapas como Faster R-CNN [33]. YOLO divide la imagen en una cuadrícula y predice simultáneamente las cajas delimitadoras y las clases presentes en cada región [33], lo que lo convierte en la arquitectura más adecuada para una aplicación con restricciones de latencia en tiempo real.

Se elige específicamente **YOLOv8** en su implementación de **Ultralytics**, ya que esta versión introduce mejoras arquitectónicas respecto a versiones anteriores, mantiene compatibilidad con exportación en formato ONNX para optimización en plataformas embebidas, y ofrece una API de uso directo en Python que simplifica la integración. Además, su entrenamiento puede realizarse

sobre conjuntos de datos especializados en objetos relevantes para la navegación peatonal, lo que permite ajustar el modelo a los casos de uso específicos del sistema (personas, puertas, escaleras, obstáculos estáticos).

Detección de proximidad: sensores ultrasónicos

El requerimiento RF-10 exige estimar la distancia entre el usuario y los objetos detectados, y el requerimiento RF-11 establece que esta estimación debe validarse con mediciones de sensores físicos para reducir falsas alarmas. La validación cruzada entre visión y sensores tiene especial relevancia en situaciones donde la iluminación es baja o cuando un objeto no se captura con suficiente claridad en la cámara.

Como se expone en el marco teórico, los sensores ultrasónicos miden distancias mediante el tiempo de vuelo de un pulso sonoro y ofrecen una lectura sencilla y económica para cortas y medias distancias [43]. Si bien los sensores ToF ofrecen mayor precisión y menor tamaño [45], los sensores ultrasónicos como el HC-SR04 resultan suficientes para validar la presencia o ausencia de obstáculos cercanos dentro de un rango de operación de 2 cm a 4 m [43], que cubre los casos críticos de navegación peatonal. Su implementación mediante el protocolo I2C sobre la plataforma embebida es directa mediante bibliotecas como `smbus2` o `RPi.GPIO`, sin requerir hardware adicional ni procesamiento significativo.

Inferencia contextual y fusión sensorial

El requerimiento RF-13 exige que el sistema infiera relaciones contextuales entre objetos detectados, como el estado de apertura de una puerta o el grado de obstrucción de un pasillo, para enriquecer la representación del entorno más allá de la detección individual. El requerimiento RF-12 establece que el sistema solo debe emitir una alerta al *confirmar* un obstáculo, no al simplemente detectarlo, lo cual requiere un mecanismo de validación probabilística.

Para esta función se adopta un modelo de **inferencia probabilística basada en reglas**, implementado en Python. Este enfoque corresponde al aprendizaje supervisado descrito en el marco teórico, donde la inferencia probabilística es la base para clasificación [29]. La elección de un modelo probabilístico ligero, en lugar de una red neuronal compleja, responde directamente a RNF-05: la inferencia contextual debe ejecutarse en hardware embebido de recursos limitados sin introducir latencia adicional significativa al ciclo de percepción. La fusión de la salida del módulo de visión con la lectura de sensores se realiza mediante umbrales de confianza configurables, garantizando el comportamiento conservador requerido por RNF-09.

6.4.2. Módulos de representación

Los módulos de representación transforman la información perceptual en una estructura espacial que el módulo de navegación puede utilizar para planificar rutas. El mapa generado debe distinguir zonas libres de zonas ocupadas (RF-16), actualizarse a una frecuencia mínima de 10 Hz (RNF-03) y construirse de forma incremental sin necesidad de un mapa previo (RF-15).

Representación del entorno: occupancy grid con NumPy

Como se estudió en el marco teórico, los mapas de ocupación representan el entorno mediante una cuadrícula de celdas donde cada celda almacena la probabilidad de estar ocupada por un obstáculo [75]. Este concepto introducido por Elfes en 1989 es uno de los métodos más utilizados en robótica móvil para representar entornos desconocidos, ya que cada celda puede clasificarse como libre, ocupada o desconocida [75], lo cual corresponde exactamente a la representación requerida por RF-16.

La *occupancy grid* se implementa como una matriz bidimensional gestionada con **NumPy**, aprovechando sus operaciones vectorizadas para actualizar múltiples celdas en un solo paso. Esta elección garantiza eficiencia computacional (RNF-05) y evita la necesidad de dependencias externas para el manejo matricial. La representación en formato *log-odds* permite actualizar probabilidades de ocupación de forma incremental y numéricamente estable, acumulando observaciones a lo largo del tiempo.

Estimación de posición: SLAM simplificado con EKF

El requerimiento RF-14 exige mantener una estimación continua de la posición y orientación del usuario integrando movimiento y percepción, y el requerimiento RF-15 añade que esta estimación debe construirse de forma incremental sin mapa previo. Estos dos requerimientos describen conjuntamente el problema clásico de *Simultaneous Localization and Mapping* (SLAM), cuyo objetivo es estimar simultáneamente la pose del agente y el mapa del entorno a partir únicamente de sus propias observaciones [78].

Dado que el hardware objetivo es una plataforma embebida con recursos limitados (RNF-05), una implementación completa de SLAM como ORB-SLAM o LiDAR-SLAM sería inviable: estos sistemas requieren procesamiento intensivo de nubes de puntos o de descriptores visuales que excede la capacidad de una Raspberry Pi 4 operando en tiempo real. Por ello se opta por un **SLAM simplificado orientado a mapeo local**, apoyado en un **Filtro de Kalman Extendido** (EKF) implementado en Python con NumPy. Esta elección equilibra la necesidad de localización continua

(RF-14) con la restricción de recursos (RNF-05).

Representación del estado. La pose del usuario se modela como un vector de estado tridimensional:

$$\mathbf{x}_t = [x_t,\ y_t,\ \theta_t]^T$$

donde (x_t, y_t) es la posición en el plano 2D y θ_t es la orientación. Esta representación es suficiente para el dominio de navegación peatonal en superficies planas y evita la complejidad computacional de una pose en 6 grados de libertad.

Modelo de movimiento (etapa de predicción). En cada ciclo, la pose se propaga hacia adelante usando los incrementos de odometría $(\Delta d,\ \Delta\theta)$ estimados a partir del desplazamiento del usuario:

$$\mathbf{x}_t^- = \mathbf{x}_{t-1} + \begin{bmatrix} \Delta d \cos(\theta_{t-1}) \\ \Delta d \sin(\theta_{t-1}) \\ \Delta\theta \end{bmatrix}$$

Este modelo de movimiento unicycle es no lineal en θ, lo que justifica el uso del EKF en lugar del filtro de Kalman estándar: el EKF linealiza localmente las funciones no lineales mediante jacobianos, haciendo manejable la propagación de incertidumbre sin abandonar el marco probabilístico [22]. La covarianza del estado se propaga como:

$$\mathbf{P}_t^- = \mathbf{F}\,\mathbf{P}_{t-1}\,\mathbf{F}^T + \mathbf{Q}$$

donde $\mathbf{F}$ es el jacobiano del modelo de movimiento respecto al estado y $\mathbf{Q}$ es la covarianza del ruido de proceso, que modela la incertidumbre acumulada por la odometría aproximada.[22].

Corrección con mediciones (etapa de actualización). Una vez obtenida la predicción, el EKF la corrige con las mediciones reales provenientes de los sensores de proximidad. La ganancia de Kalman $\mathbf{K}$ determina cuánto peso se otorga a la medición frente a la predicción:

$$\mathbf{K} = \mathbf{P}_t^-\mathbf{H}^T\big(\mathbf{H}\mathbf{P}_t^-\mathbf{H}^T + \mathbf{R}\big)^{-1}$$

La pose y la covarianza se actualizan a partir de la innovación, es decir, la diferencia entre la medición real $\mathbf{z}_t$ y la medición esperada $h(\mathbf{x}_t^-)$ según el modelo de observación $h(\cdot)$:

$$\mathbf{x}_t = \mathbf{x}_t^- + \mathbf{K}\big(\mathbf{z}_t - h(\mathbf{x}_t^-)\big)$$

$$\mathbf{P}_t = (\mathbf{I} - \mathbf{K}\mathbf{H})\,\mathbf{P}_t^-$$

donde $\mathbf{H}$ es el jacobiano de $h(\cdot)$ respecto al estado y $\mathbf{R}$ es la covarianza del ruido de medición de los sensores. Cuando la innovación es pequeña (la predicción coincide bien con la medición) $\mathbf{K}$ se acerca a cero y se confía más en el modelo; cuando es grande, $\mathbf{K}$ crece y la medición tiene mayor peso sobre la estimación final. Este mecanismo autorregulado es lo que hace al EKF robusto ante la acumulación de error de odometría [22].

Actualización del mapa en log-odds. En paralelo a la estimación de la pose, el mapa de ocupación se actualiza de forma incremental usando una representación en log-odds. Cada celda c almacena el logaritmo de la razón de probabilidades de estar ocupada frente a estar libre:

$$l_t(c) = l_{t-1}(c) + \log \frac{p(\mathbf{z}_t \mid c = \text{occ})}{p(\mathbf{z}_t \mid c = \text{free})}$$

Esta representación tiene dos ventajas directas que justifican su elección. Primero, la actualización es aditiva e incremental: cada nueva medición simplemente suma un término al valor acumulado de la celda, sin necesidad de recalcular probabilidades completas. Segundo, trabajar en escala logarítmica evita problemas numéricos de underflow cuando las probabilidades se acercan a cero o a uno tras muchas observaciones, lo que es frecuente en celdas cercanas al usuario [22]. Los valores de log-odds se saturan en un rango $[l_{\text{mín}},\, l_{\text{máx}}]$ para limitar el tiempo que el sistema tarda en corregir una celda marcada erróneamente, garantizando que el mapa siga siendo actualizable ante cambios dinámicos del entorno (RF-17).

Actualización del mapa: ray casting con algoritmo de Bresenham

El requerimiento RF-17 establece que el mapa debe actualizarse en respuesta a cambios detectados en el entorno, y RNF-03 fija una frecuencia mínima de 10 Hz para esta actualización. El mecanismo de actualización debe ser computacionalmente ligero para no comprometer la latencia global del sistema.

Como se explica en el marco teórico, el ray casting permite proyectar rayos desde la posición estimada del usuario para actualizar las celdas del mapa: las celdas recorridas por el rayo se marcan como libres y la celda donde el rayo se detiene se marca como ocupada [75]. Para recorrer las celdas de forma eficiente se utiliza el **algoritmo de Bresenham** [76], que permite identificar las celdas atravesadas por un rayo mediante operaciones aritméticas simples, sin multiplicaciones ni divisiones en punto flotante. Como se señala en el marco teórico, el ray casting presenta un bajo costo computacional, puede ejecutarse en CPU sin necesidad de GPU y es compatible con mapas discretos basados en cuadrículas [73], lo que lo convierte en la técnica más adecuada para cum-

plir RNF-03 y RNF-05 simultáneamente, a diferencia del ray tracing, que requiere procesamiento significativamente mayor [73].

6.4.3. Módulo de navegación y rutas

El módulo de navegación es el núcleo decisional del sistema. Debe calcular rutas óptimas sobre el mapa local (RF-18), actualizarlas periódicamente (RF-19), recalcularlas de forma inmediata ante obstáculos confirmados (RF-20), traducirlas en instrucciones de movimiento (RF-21), integrarse con navegación global en exteriores (RF-22) y gestionar la seguridad del usuario como prioridad (RNF-09).

Navegación local: aprendizaje por refuerzo profundo (DRL)

A diferencia de los algoritmos clásicos de búsqueda de caminos como A* o Dijkstra, que requieren un mapa completamente conocido y estático para operar de forma óptima [27, 26], la navegación en entornos peatonales reales implica incertidumbre, cambios dinámicos y necesidad de adaptación continua. El requerimiento RF-19 establece que la ruta se recalcula periódicamente durante la navegación, pero RF-20 exige recálculo inmediato ante un obstáculo confirmado: esta combinación describe un agente que debe tomar decisiones secuenciales bajo incertidumbre, que es exactamente el dominio del aprendizaje por refuerzo.

Como se estudió en el marco teórico, el aprendizaje por refuerzo es un enfoque en el que un agente aprende a tomar decisiones mediante la interacción con su entorno, recibiendo recompensas o penalizaciones, con el objetivo de aprender una política que maximice la recompensa acumulada [29]. Al combinarlo con redes neuronales profundas, el agente es capaz de aprender representaciones complejas del entorno directamente desde los datos de los sensores, lo que lo hace especialmente adecuado para espacios de estado continuos y entornos con obstáculos dinámicos como los que enfrenta el sistema [29].

Se propone el uso de **aprendizaje por refuerzo profundo** (DRL) para la navegación local, con un espacio de acciones discreto compuesto por las cuatro decisiones posibles del sistema: avanzar, girar a la izquierda, girar a la derecha y detenerse. El estado del entorno se representa como un vector compacto que incluye las distancias al obstáculo más cercano en cada dirección, la posición relativa al siguiente waypoint y los niveles de riesgo detectados por el módulo de percepción. La selección específica del algoritmo de entrenamiento, el entorno de simulación y la estrategia de transferencia al hardware real se definirán en la segunda etapa del trabajo terminal, una vez validado el comportamiento del resto de los módulos del sistema.

Submódulo de dirección: traducción de acciones

El requerimiento RF-21 exige que la acción elegida por el modelo se traduzca en una instrucción discreta y comprensible para el usuario. Dado que el espacio de acciones del modelo de RL es ya discreto (avanzar, girar izquierda, girar derecha, detenerse), esta traducción puede implementarse mediante **reglas determinísticas simples** en Python: cada índice de acción se mapea directamente a una instrucción de texto con distancia y ángulo aproximados. La complejidad del sistema se concentra en el modelo de decisión; la capa de traducción no requiere aprendizaje ni módulos adicionales. Esto garantiza latencia baja y comportamiento predecible, alineándose con RNF-09.

Módulo de alerta de obstáculos: capa de seguridad por umbrales

El requerimiento RF-12 establece que al confirmar un obstáculo el sistema debe notificar al usuario de forma inmediata y desencadenar el recálculo de ruta. El requerimiento RNF-09 añade que ante baja confianza el sistema debe optar por la acción más conservadora. Estos requerimientos describen una capa de seguridad independiente del modelo de RL, que actúa como interruptor prioritario.

Esta capa se implementa mediante **umbrales determinísticos** sobre la distancia estimada y el nivel de riesgo asignado por el módulo de percepción. Si la distancia al obstáculo confirmado cae por debajo de un umbral de alerta, el sistema emite la alerta por voz y notifica al módulo de navegación para recalcular, independientemente de la decisión del modelo de RL. Este diseño garantiza el comportamiento conservador exigido por RNF-09 sin depender del umbral de confianza del modelo, que podría variar en entornos poco estructurados.

Navegación global en exteriores: Google Maps Directions API

El requerimiento RF-22 establece que en entornos exteriores el sistema debe obtener rutas a nivel de ciudad a través de un servicio de cartografía digital. Como se estudió en el marco teórico, las APIs de navegación como la Directions API permiten calcular rutas entre dos puntos y obtener instrucciones paso a paso [64], y los sistemas de navegación digital transforman información geográfica en instrucciones comprensibles que guían al usuario hacia su destino [64].

Se elige **Google Maps Directions API** porque ofrece cobertura global, soporte para navegación peatonal, instrucciones paso a paso estructuradas y una integración directa mediante la biblioteca `requests` en Python mediante intercambio de JSON. Esta API opera únicamente cuando hay conexión a internet disponible, lo cual es coherente con lo establecido en RNF-13: la navegación global puede depender de conectividad, pero el sistema debe degradarse de forma controlada en su

ausencia, manteniendo operativos los módulos locales de percepción, representación e interacción. Cuando no hay conexión disponible, el módulo simplemente deja de emitir waypoints globales y el sistema continúa operando con navegación local.

6.4.4. Módulos de interacción

Los módulos de interacción son la interfaz entre el sistema y el usuario. Dado que el sistema está orientado a personas con discapacidad visual, toda la interacción debe realizarse exclusivamente por voz (RF-06, RNF-10), sin requerir interfaz visual en ningún momento. Los requerimientos imponen restricciones de latencia de 500 ms para el reconocimiento de comandos (RNF-02) y una tasa de reconocimiento correcto de al menos el 92 % en entornos con hasta 60 dB de ruido ambiental (RNF-07). Dado que el sistema debe operar sin conexión a internet (RNF-13), todos los componentes de interacción deben funcionar de forma local.

Captura y preprocesamiento de audio: PyAudio

El requerimiento RF-01 establece que el sistema debe capturar audio de forma continua y detectar automáticamente segmentos de voz activa. El requerimiento RF-02 añade que debe aplicarse supresión de ruido antes de entregar la señal al módulo de reconocimiento.

Se utiliza **PyAudio** como interfaz de captura de audio, ya que proporciona acceso de bajo nivel al micrófono del dispositivo en Python y es compatible con la plataforma embebida objetivo. La detección de actividad de voz (VAD) se implementa mediante un umbral de energía sobre el nivel de la señal capturada, descartando segmentos de silencio para reducir el volumen de datos enviados al módulo de reconocimiento. El preprocesamiento de reducción de ruido se aplica mediante filtros sobre el flujo de audio antes de pasarlo al reconocedor, cumpliendo RF-02 sin introducir dependencias de servicios en la nube.

Reconocimiento de voz: Vosk

El requerimiento RF-03 exige transcribir la señal de voz a texto e interpretar la intención del usuario. Las restricciones RNF-13 (sin internet) y RNF-02 (latencia $<$ 500 ms) descartan motores de reconocimiento basados en la nube como Google Speech o Whisper en su versión en línea, ya que dependen de conectividad e introducen latencia de red variable.

Se elige **Vosk** como motor de reconocimiento automático de voz (ASR) porque opera completamente de forma local (*offline*), ofrece modelos en español de tamaño reducido compatibles con

hardware embebido de recursos limitados, y su API en Python permite integrar reconocimiento continuo en tiempo real con baja latencia. Vosk cumple directamente RNF-13 al no requerir internet, y su ejecución local garantiza una latencia de reconocimiento dentro de los 500 ms establecidos por RNF-02 en condiciones normales de operación.

Procesamiento de lenguaje natural: spaCy y clasificación por reglas

Una vez transcrito el texto, el sistema debe interpretar la intención del usuario (RF-03) y reconocer comandos para controlar la sesión como pausar, reanudar o detener la navegación (RF-05). Los comandos del sistema son un conjunto cerrado y predecible de intenciones (indicar destino, solicitar descripción, controlar sesión), lo que permite implementar un clasificador de intenciones sin necesidad de un modelo de lenguaje grande.

Una vez transcrito el texto, el sistema debe interpretar la intención del usuario (RF-03) y reconocer comandos para controlar la sesión como pausar, reanudar o detener la navegación (RF-05). Los comandos del sistema son un conjunto cerrado y predecible de intenciones (indicar destino, solicitar descripción, controlar sesión), lo que permite implementar un clasificador de intenciones sin necesidad de un modelo de lenguaje grande. El texto transcrito se normaliza previamente mediante tokenización y lematización con spaCy, eliminando palabras vacías para reducir el espacio de representación. Se emplea **spaCy** para el análisis morfológico y la extracción de entidades del texto transcrito, combinado con un clasificador de intenciones basado en reglas y similitud de patrones implementado en Python. Este enfoque opera localmente sin internet (RNF-13) y su ejecución es prácticamente instantánea, lo que contribuye a mantener la latencia global dentro de RNF-02. Si la complejidad de los comandos crece en iteraciones futuras del sistema, la modularidad de esta capa (RNF-14) permite sustituir el clasificador SVM por un modelo más sofisticado sin afectar el resto del sistema.

Para la etapa de clasificación de intenciones se considera como esquema de vectorización. Esta representación pondera cada término en función de su frecuencia en el comando y su rareza relativa en el corpus de entrenamiento, de modo que las palabras clave con mayor poder discriminativo, como los verbos de acción o los nombres de destino, reciben mayor peso que términos genéricos. Dado que el conjunto de intenciones del sistema es cerrado y el vocabulario de comandos es acotado y predecible, TF-IDF captura la señal suficiente para separar las clases de forma efectiva sin incurrir en el costo computacional de representaciones más complejas.

Síntesis de voz: pyttsx3

El requerimiento RF-04 establece que el sistema debe convertir en audio las instrucciones de navegación, alertas y respuestas, reproduciéndolas mediante síntesis de voz con lenguaje natural y frases cortas. El requerimiento RNF-11 añade que las instrucciones deben ser claras y usar referencias espaciales concretas, y RNF-12 exige que la retroalimentación se emita de forma proactiva ante eventos relevantes sin esperar solicitud del usuario.

Se elige **pyttsx3** como motor de *Text-to-Speech* (TTS) porque opera completamente de forma local sin depender de servicios en línea (RNF-13), es compatible con la plataforma embebida objetivo mediante ALSA/PulseAudio, y permite ajustar la velocidad de habla y el volumen para mejorar la comprensibilidad de las instrucciones (RNF-11). Su integración en Python es directa y añade una latencia de síntesis baja para textos cortos, lo que permite la retroalimentación proactiva requerida por RNF-12 ante eventos como detección de obstáculos o cambios de ruta.

6.5. Selección y justificación de hardware principal

6.5.1. Comparación de sensores de distancia para dispositivos portables

La selección del sensor de distancia representa un aspecto crítico en el diseño de sistemas de navegación asistida. En dispositivos portables como gafas inteligentes, los sensores deben cumplir con requisitos estrictos de tamaño reducido, bajo consumo energético y suficiente alcance para detectar obstáculos con anticipación.

Existen diversos tipos de sensores que pueden emplearse para este propósito, entre los que destacan sensores ultrasónicos básicos, sensores ultrasónicos industriales de alta precisión, sensores Time-of-Flight (ToF) y sensores LiDAR compactos. Cada tecnología presenta ventajas y limitaciones que deben considerarse en función del entorno de operación y las restricciones del dispositivo.

La Tabla 6.1 muestra una comparación entre algunos sensores comerciales ampliamente utilizados en sistemas embebidos y robótica.

Sensor	Tipo	Alcance	Precisión	Tamaño	Interfaz
HC-SR04 [43]	Ultrasónico	2 cm – 4 m	$\pm 3mm$	$45x20x15mm$	Trigger/Echo
MB1232 [44]	Ultrasónico	20 cm – 7.65 m	$\pm 1mm$	$14{,}4x15{,}5x15{,}5mm$	I2C
VL53L0X [46]	ToF	hasta 2 m	$\pm 3\,\%$	$4{,}4x2{,}4x1{,}0mm$	I2C
VL53L1X [47]	ToF	hasta 4 m	$\pm 20mm$	$4{,}9x2{,}5x1{,}5mm$	I2C
TF-Mini [48]	LiDAR	hasta 12 m	$\pm 6cm$	$42x15x16mm$	UART/I2C
Lidar Lite v3 [49]	LiDAR	hasta 40 m	$\pm 25cm$	$40x48x20mm$	I2C/PWM

Tabla 6.1: Comparación de sensores de distancia utilizados en sistemas embebidos [43, 44, 46, 47, 48, 49].

Como puede observarse, los sensores ultrasónicos básicos como el HC-SR04 [43] presentan un costo bajo pero su tamaño físico y precisión limitada los hacen menos adecuados para dispositivos portables en entornos ruidosos. El sensor MB1232 [44] resuelve estas limitaciones manteniendo el principio ultrasónico: ofrece un alcance de hasta 7.65 m, resolución de 1 cm, interfaz I2C, calibración automática en tiempo real y un tamaño inferior a una pulgada cúbica, lo que lo convierte en la opción más adecuada para el presente trabajo terminal.

En contraste, sensores ToF como el VL53L1X [47] ofrecen alta precisión pero a alcances más limitados (hasta 4 m), y los sensores LiDAR compactos como el TF-Mini [48] proporcionan mayor alcance pero con un consumo energético y costo superiores.

En el contexto del sistema de gafas inteligentes para navegación asistida desarrollado en este trabajo, el MB1232 representa la solución idónea, ya que combina el mayor alcance de la categoría ultrasónica compacta, alta tolerancia al ruido acústico y eléctrico de entornos urbanos, bajo consumo energético y la interfaz I2C requerida para su integración con la plataforma Raspberry Pi [44].

6.5.2. Comparación de módulos de cámara para sistemas embebidos

Las cámaras utilizadas en sistemas de visión por computadora deben proporcionar imágenes con suficiente resolución y calidad para permitir la detección y reconocimiento de objetos mediante algoritmos de inteligencia artificial.

En dispositivos portables, los módulos de cámara deben además cumplir con restricciones de tamaño y consumo energético [52].

La Tabla 6.2 presenta una comparación entre algunos módulos de cámara ampliamente utilizados en plataformas embebidas.

Modelo	Sensor	Resolución	Tamaño	Interfaz	Uso común
OV2640 [53]	OmniVision	2 MP	$21 \times 12{,}5mm$	SPI/DVP	IoT, ESP32-CAM
OV5647 [50]	OmniVision	5 MP	$25 \times 24mm$	CSI	Raspberry Pi
IMX219 [51]	Sony	8 MP	$25 \times 24mm$	CSI	Raspberry Pi v2
IMX477 [50]	Sony	12 MP	$38 \times 38mm$	CSI	Visión avanzada
IMX708 [52]	Sony	12 MP	$25 \times 24mm$	MIPI CSI-2	Raspberry Pi v3
ArducamMini [51]	Variado	2–12 MP	$21 \times 12{,}5mm$	SPI/I2C	Sistemas embebidos

Tabla 6.2: Comparación de módulos de cámara para aplicaciones embebidas [53, 50, 52, 51].

Entre los módulos presentados, el Raspberry Pi Camera Module 3 con sensor IMX708 [52] destaca como la opción más adecuada para el presente trabajo terminal. Su sensor Sony de 12 MP con tecnología back-illuminated stacked CMOS ofrece imágenes de alta resolución y calidad incluso en condiciones de baja iluminación, mientras que el sistema de autoenfoque por detección de fase (PDAF) garantiza nitidez sobre objetos a distintas distancias sin intervención manual. El modo HDR resulta especialmente valioso para mantener la calidad de imagen en entornos urbanos con variaciones abruptas de iluminación, como transiciones entre interiores y exteriores. Adicionalmente, su compatibilidad nativa con Raspberry Pi y soporte completo por parte de la biblioteca `libcamera` simplifican considerablemente el desarrollo del sistema de visión por computadora basado en inteligencia artificial propuesto en este trabajo [52].

6.5.3. Resumen de decisiones tecnológicas

La Tabla 6.3 resume las tecnologías seleccionadas para cada módulo del sistema, los requerimientos que motivan cada elección y los fundamentos teóricos que la respaldan.

Módulo	**Tecnología**	**Requerimientos**	**Fundamento teórico**
Preprocesamiento	OpenCV + NumPy	RF-07, RNF-05, RNF-13	[38]
Detección de objetos	YOLOv8 (Ultralytics)	RF-08, RF-09, RNF-01, RNF-06	[33]
Proximidad sensorial	Sensores ultrasónicos + smbus2	RF-10, RF-11	[43]
Fusión e inferencia	Clasificación por reglas probabilísticas en Python	RF-11, RF-12, RF-13, RNF-09	[29]
Mapa de ocupación	Occupancy grid + NumPy	RF-15, RF-16, RNF-03, RNF-05	[75]
Localización	SLAM simplificado + EKF	RF-14, RF-15, RNF-05	[22]
Actualización del mapa	Ray casting (Bresenham)	RF-17, RNF-03	[75, 76]
Navegación local	RL con PPO (SB3 + PyTorch)	RF-18, RF-19, RF-20, RNF-04	[29]
Alerta de obstáculos	Umbrales deterministicos	RF-12, RF-20, RNF-09	—
Instrucciones	Reglas de dirección en Python	RF-21, RNF-11	—
Navegación global	Google Maps Directions API	RF-22, RF-23, RNF-13	[64]
Captura de audio	PyAudio + VAD	RF-01, RF-02, RNF-02	—
Reconocimiento de voz	Vosk (offline ASR)	RF-03, RF-05, RNF-07, RNF-13	—
PLN e intención	spaCy + reglas	RF-03, RF-05, RNF-02	—
Síntesis de voz	pyttsx3 (offline TTS)	RF-04, RNF-11, RNF-12, RNF-13	—

Tabla 6.3: Resumen de tecnologías seleccionadas por módulo con sus requerimientos asociados.

Capítulo 7

Implementación de los módulos

El presente capítulo describe la implementación de cada módulo del sistema de navegación asistida, detallando su diseño interno, sus entradas y salidas, los algoritmos empleados y su pseudocódigo correspondiente. La organización sigue la arquitectura modular definida en el marco metodológico y justificada en el análisis de requerimientos, cubriendo los cuatro grupos de módulos: percepción, representación, navegación y rutas, e interacción.

Las tecnologías asignadas a cada módulo y los requerimientos que motivan cada elección se resumen en la Tabla 6.3 del capítulo anterior.

7.1. Módulos de percepción

Los módulos de percepción transforman las señales físicas del entorno, fotogramas de cámara y lecturas de sensores de proximidad en información estructurada que los módulos de representación y navegación pueden procesar. Su flujo interno se ilustró en la Figura 6.3.

7.1.1. Procesamiento de imágenes

Descripción: Este módulo prepara cada fotograma capturado por la cámara para que pueda ser procesado de forma eficiente y estable por el módulo de visión. Su objetivo es reducir la variabilidad innecesaria en la imagen, estandarizar las condiciones de entrada y eliminar artefactos visuales que puedan afectar la detección.

Entradas:

- Fotograma capturado por la cámara (imagen en bruto)

Salidas:

- Imagen preprocesada lista para el módulo de visión

Procesos realizados:

- Redimensionamiento al tamaño de entrada requerido por el modelo de visión
- Normalización de valores de píxel al rango $[0, 1]$
- Igualación de histograma para compensar variaciones de iluminación
- Filtrado gaussiano para reducción de ruido de alta frecuencia

Tecnologías utilizadas: OpenCV, NumPy

Se puede encontrar el pseudocódigo utilizado para el procesamiento de imágenes en el apéndice A.2

7.1.2. Módulo de visión

Descripción: Este módulo aplica un modelo de detección de objetos sobre la imagen preprocesada para identificar los elementos relevantes del entorno en tiempo real. Por cada objeto detectado genera una región de interés (bounding box), su categoría semántica y un nivel de confianza. Además, asigna a cada objeto un nivel de riesgo para la navegación, distinguiendo entre elementos de riesgo inmediato y elementos que forman parte del entorno navegable sin representar una amenaza.

Entradas:

- Imagen preprocesada (proveniente del módulo de procesamiento de imágenes)

Salidas:

- Lista de objetos detectados, cada uno con:
  - Clase semántica (persona, puerta, escalera, obstáculo, pared, etc.)
  - Región de interés (coordenadas de la caja delimitadora)

- Nivel de confianza de la detección (valor entre 0 y 1)
- Nivel de riesgo asignado (alto, medio, bajo)

Modelo empleado: YOLOv8 (Ultralytics). Se elige por su arquitectura de una sola etapa, que realiza extracción de características y predicción de objetos en un único paso, lo que permite cumplir la restricción de latencia de 300 ms establecida en RNF-01.

Tabla de niveles de riesgo:

Clase de objeto	Nivel de riesgo
Persona (en movimiento)	Alto
Escalera / desnivel	Alto
Obstáculo estático	Medio
Puerta (cerrada)	Medio
Puerta (abierta)	Bajo
Pared / superficie	Bajo

Tabla 7.1: Clasificación de riesgo por tipo de objeto detectado

Pseudocódigo del módulo de visión

Algoritmo 1: Pseudocódigo del módulo de visión

ENTRADA:
imagen_lista ▷ imagen preprocesada

SALIDA:
lista_detecciones ▷ lista de objetos con clase, caja, confianza y riesgo

Ejecutar el modelo de detección sobre la imagen
resultados_brutos ← modelo_deteccion.inferir(imagen_lista)

PROCESO
Filtrar resultados por umbral mínimo de confianza (umbral = 0.5)
lista_detecciones ← []

PARA cada resultado EN resultados_brutos **hacer:**
    SI resultado.confianza >= umbral_confianza **entonces:**
        clase ← resultado.clase

caja ← resultado.caja_delimitadora
conf ← resultado.confianza
riesgo ← tabla_riesgo[clase] ▷ Asignar nivel de riesgo según tabla de clasificación
agregar {clase, caja, conf, riesgo} a lista_detecciones
Fin SI
Fin PARA

Ordenar lista_detecciones por nivel de riesgo (mayor primero)

RETORNAR lista_detecciones

7.1.3. Módulo sensorial

Descripción: Este módulo combina las detecciones visuales con las lecturas de los sensores de proximidad para confirmar o descartar la presencia de obstáculos antes de notificar al módulo de navegación. La validación cruzada entre visión y sensores compensa las limitaciones individuales de cada fuente: errores de detección en condiciones de baja iluminación o lecturas ruidosas de los sensores. Solo los obstáculos confirmados por ambas fuentes disparan una alerta.

Entradas:

- Lista de objetos detectados (módulo de visión): clase, caja, confianza, riesgo
- Lecturas de distancia de los sensores de proximidad (en centímetros)

Salidas:

- Lista de obstáculos confirmados con: clase, distancia estimada, nivel de riesgo
- Señal de alerta (si algún obstáculo supera el umbral crítico)

Enfoque de fusión: Se utiliza un esquema de inferencia basado en reglas con validación cruzada espacial. La confianza final de cada obstáculo combina la confianza del modelo de visión con la consistencia de la lectura del sensor correspondiente:

$$\text{confianza_final} = \alpha \cdot \text{confianza_visual} + (1 - \alpha) \cdot \text{consistencia_sensor}$$

donde $\alpha \in [0, 1]$ es un parámetro configurable que pondera el peso de cada fuente, y consistencia_sensor es un valor normalizado que indica qué tan coherente es la distancia medida por el sensor con la posición estimada del objeto en la imagen.

Pseudocódigo del módulo sensorial

Algoritmo 2: Pseudocódigo del módulo sensorial

ENTRADA:
lista_detecciones ▷ objetos detectados por visión
lecturas_sensores ▷ arreglo de distancias por dirección (cm)

SALIDA:
obstaculos_confirmados ▷ lista de obstáculos validados
alerta_activa ▷ verdadero si hay riesgo crítico

Inicializar obstaculos_confirmados ← []
alerta_activa ← falso

PROCESO
PARA cada objeto EN lista_detecciones **hacer:**
    Estimar la dirección del objeto según su posición
    en la imagen (izquierda, centro, derecha)

    Seleccionar la lectura del sensor correspondiente
    a esa dirección
    distancia_sensor ← lecturas_sensores[direccion]

    Calcular consistencia entre visión y sensor
    SI distancia_sensor <distancia_maxima_sensor **entonces:**
        consistencia ← $1 - |distancia_visual - distancia_sensor|$
            / distancia_maxima_sensor
    SINO
        consistencia ← 0.0 ▷ sensor no detecta nada en esa dirección
    Fin SI

    Calcular confianza final
    confianza_final ← alfa * objeto.confianza
        + (1 - alfa) * consistencia

```
    SI confianza_final >= umbral_confirmacion entonces:
        distancia_final ← promedio(distancia_visual, distancia_sensor)
        agregar {objeto.clase, distancia_final,
         objeto.riesgo, confianza_final}
         a obstaculos_confirmados

        SI distancia_final <umbral_critico entonces:
            alerta_activa ← verdadero
        Fin SI
    Fin SI
Fin PARA

Ordenar obstaculos_confirmados por distancia (menor primero)

RETORNAR obstaculos_confirmados, alerta_activa
```

7.2. Módulos de representación

Los módulos de representación transforman la información perceptual en una estructura espacial que el módulo de navegación puede usar para planificar rutas. Este grupo de módulos opera de forma integrada: el ray casting actualiza el mapa de ocupación y el SLAM simplificado mantiene la estimación de la posición del usuario dentro de ese mapa.

El pseudocódigo general de la interacción entre los módulos de representación se pueden ver en el apéndice A.3

7.2.1. SLAM simplificado

Descripción: Este módulo estima en tiempo real la *pose* del usuario (posición y orientación) y mantiene sincronizada esa estimación con el mapa local del entorno. Se adopta un enfoque ligero inspirado en SLAM, adecuado para ejecución sobre hardware embebido con recursos limitados, integrando odometría aproximada y mediciones de sensores mediante un Filtro de Kalman Extendido simplificado.

Representación del estado: La pose del usuario se representa como el vector $\mathbf{x}_t =$

$[x_t,\ y_t,\ \theta_t]^T$, donde (x_t, y_t) es la posición en el plano 2D y θ_t la orientación. La derivación completa del modelo se presenta en el capítulo de análisis.

Entradas:

- Incrementos de movimiento estimados (odometría: Δd, $\Delta\theta$)
- Lecturas de sensores de proximidad
- Mapa de ocupación actual (reticula en log-odds)

Salidas:

- Estimación actualizada de la pose $\mathbf{x}_t$
- Mapa de ocupación actualizado

Modelo de movimiento: La predicción de la pose en cada ciclo se obtiene aplicando los incrementos de odometría $(\Delta d,\ \Delta\theta)$ sobre el estado anterior mediante el modelo unicycle no lineal descrito en el capítulo de análisis.

Corrección con EKF: La pose predicha se ajusta con las mediciones reales de los sensores siguiendo el esquema predictor-corrector del EKF detallado en el capítulo de análisis. El proceso calcula primero la innovación (diferencia entre la medición real y la esperada según el modelo de observación) y a partir de ella obtiene la ganancia de Kalman, que pondera automáticamente cuánto corregir la predicción en función de la incertidumbre relativa entre el modelo y los sensores. Con esa ganancia se actualiza tanto la pose estimada como su covarianza, reduciendo el error acumulado por la odometría aproximada. En paralelo, el mapa de ocupación se actualiza celda a celda en escala log-odds, sumando el aporte de cada nueva observación de forma incremental y saturando los valores en $[l_{\text{mín}},\ l_{\text{máx}}]$ para mantener el mapa corregible ante cambios dinámicos del entorno.

Actualización del mapa (log-odds): Una vez corregida la pose, cada celda del mapa de ocupación se actualiza sumando el aporte de la medición actual a su valor acumulado en escala log-odds, tal como se deriva en el capítulo de análisis. Los valores se saturan en el rango $[l_{\text{mín}},\ l_{\text{máx}}]$ para garantizar que el mapa pueda corregirse ante cambios dinámicos del entorno. Este proceso es ejecutado por el módulo de Ray Casting a partir de la pose estimada, y su pseudocódigo se detalla en la subsección siguiente.

Pseudocódigo del SLAM simplificado

Algoritmo 3: Pseudocódigo del módulo SLAM

ENTRADA:

pose_anterior ▷ [x, y, theta] del ciclo previo
covarianza_anterior ▷ matriz de covarianza del estado
odometria ▷ (delta_d, delta_theta) del movimiento actual
mediciones ▷ lecturas de sensores de proximidad
reticula ▷ mapa de ocupación en log-odds

SALIDA:

pose_estimada ▷ nueva pose corregida
covarianza_nueva ▷ covarianza actualizada
reticula ▷ mapa actualizado

— ETAPA 1: PREDICCIÓN —

Calcular pose predicha con modelo de movimiento
pose_pred.x ← pose_anterior.x
+ odometria.delta_d * coseno(pose_anterior.theta)
pose_pred.y ← pose_anterior.y
+ odometria.delta_d * seno(pose_anterior.theta)
pose_pred.theta ← pose_anterior.theta + odometria.delta_theta

Calcular jacobiano del modelo de movimiento (F)
F ← jacobiano_movimiento(pose_anterior, odometria)

Propagar covarianza
covarianza_pred ← F * covarianza_anterior * F^T + Q
▷ Q: covarianza de ruido del modelo de movimiento

— ETAPA 2: CORRECCIÓN —

Calcular medición esperada desde la pose predicha
medicion_esperada ← modelo_observacion(pose_pred, reticula)

Calcular innovación (diferencia entre real y esperado)
innovacion ← mediciones - medicion_esperada

Calcular jacobiano del modelo de observación (H)
H ← jacobiano_observacion(pose_pred)

Calcular ganancia de Kalman
S ← H * covarianza_pred * H^T + R
    // R: covarianza de ruido de medición
K ← covarianza_pred * H^T * inversa(S)

Corregir pose y covarianza
pose_estimada ← pose_pred + K * innovacion
covarianza_nueva ← (I - K * H) * covarianza_pred

— ETAPA 3: ACTUALIZACIÓN DEL MAPA (Ray Casting) —

PARA cada rayo i desde la pose estimada **hacer:**
    llamar al módulo de Ray Casting con:
        pose_estimada, angulo_rayo[i], distancia_sensor[i]
    actualizar reticula con las celdas resultantes
Fin PARA

Saturar valores de log-odds para estabilidad numérica
(valores fuera del rango [lo_min, lo_max] se recortan)

RETORNAR pose_estimada, covarianza_nueva, reticula

7.2.2. Ray Casting

Descripción: Este módulo proyecta rayos desde la posición estimada del usuario en distintas direcciones para actualizar el mapa de ocupación. Las celdas que el rayo atraviesa se marcan como libres y la celda donde el rayo impacta se marca como ocupada. El resultado es un mapa bidimensional que refleja el estado del entorno en tiempo real.

Entradas:

- Pose estimada del usuario (posición y orientación)
- Ángulos de emisión de los rayos

- Distancias de impacto (provenientes de sensores o del módulo de visión)
- Mapa de ocupación actual en formato log-odds

Salidas:

- Mapa de ocupación actualizado

Representación de celdas: El entorno se modela como una reticula bidimensional donde cada celda almacena un valor en escala log-odds que representa su probabilidad de estar ocupada:

$$c = \begin{cases} l > 0 & \text{celda probablemente ocupada} \\ l = 0 & \text{celda desconocida} \\ l < 0 & \text{celda probablemente libre} \end{cases}$$

Algoritmo de recorrido: Se utiliza el algoritmo de Bresenham para identificar de forma eficiente las celdas que atraviesa cada rayo, sin necesidad de operaciones en punto flotante para el recorrido de la línea.

El pseudocódigo completo del modulo se encuentra en el apéndice A.1

7.3. Módulo de navegación y rutas

Este módulo constituye el núcleo decisional del sistema. Combina navegación global a nivel de ciudad con navegación local reactiva mediante un modelo de aprendizaje por refuerzo. El flujo interno del módulo se ilustró en la Figura 6.4.

7.3.1. Modelo de navegación y rutas

Descripción general: El módulo de navegación adopta un enfoque híbrido. Para trayectos largos en exteriores se obtiene una ruta de alto nivel mediante un servicio de cartografía digital. La navegación local, que es la que guía al usuario paso a paso en tiempo real, se resuelve mediante un agente de aprendizaje por refuerzo (RL) entrenado para tomar decisiones de movimiento discretas sobre el mapa de ocupación.

¿Por qué aprendizaje por refuerzo para la navegación local? Los algoritmos de búsqueda de caminos clásicos, como A*, calculan rutas óptimas sobre mapas estáticos y completamente

conocidos. Sin embargo, el entorno peatonal real cambia constantemente: aparecen personas, se abren puertas, se bloquean pasillos. El agente de RL aprende una *política* de navegación que le permite seleccionar la mejor acción en función del estado actual del entorno, sin depender de un mapa completamente conocido de antemano. Esto lo hace inherentemente adaptable a cambios dinámicos.

Representación del estado del entorno: El agente recibe en cada paso un vector de estado compacto:

$$\mathbf{s}_t = [\, d_{\text{front}},\ d_{\text{left}},\ d_{\text{right}},\ \phi_{\text{goal}},\ d_{\text{goal}},\ r_{\text{front}},\ r_{\text{left}},\ r_{\text{right}} \,]$$

donde:

- $d_{\text{front}}, d_{\text{left}}, d_{\text{right}}$: distancias al obstáculo más cercano en cada dirección, extraídas del mapa de ocupación
- ϕ_{goal}: ángulo relativo entre la orientación actual y la dirección al próximo waypoint
- d_{goal}: distancia al próximo waypoint
- $r_{\text{front}}, r_{\text{left}}, r_{\text{right}}$: nivel de riesgo máximo detectado en cada dirección (0=libre, 1=bajo, 2=medio, 3=alto)

Espacio de acciones: El agente produce en cada paso una de cuatro acciones discretas:

Índice	Acción
0	Avanzar
1	Girar a la izquierda
2	Girar a la derecha
3	Detenerse

Tabla 7.2: Espacio de acciones discretas del agente de RL

Función de recompensa: La recompensa guía el aprendizaje del agente durante el entrenamiento en simulación:

$$r_t = \begin{cases} +100 & \text{si llega al destino} \\ -100 & \text{si colisiona con un obstáculo} \\ +\Delta d_{\text{goal}} & \text{si se acerca al destino (progreso)} \\ -\Delta d_{\text{goal}} & \text{si se aleja del destino} \\ -0{,}1 & \text{penalización por paso (incentiva eficiencia)} \\ -5 \cdot \text{riesgo} & \text{penalización por proximidad a obstáculos de alto riesgo} \end{cases}$$

Arquitectura del modelo: Se utiliza una red neuronal de tipo actor-crítico entrenada con el algoritmo PPO (*Proximal Policy Optimization*). El actor produce una distribución de probabilidad sobre las acciones y el crítico estima el valor del estado actual. El entrenamiento se realiza en un entorno de simulación construido con Gymnasium, y la política resultante se transfiere al hardware real en modo de inferencia.

Pseudocódigo del módulo de navegación (modo inferencia)

Algoritmo 4: Pseudocódigo del módulo de navegación (modo inferencia)

ENTRADA:

mapa_local	▷ reticula de ocupación actualizada
pose_actual	▷ [x, y, theta] del usuario
waypoint_global	▷ siguiente punto de la ruta global (si existe)
obstaculos	▷ lista de obstáculos confirmados
modelo_rl	▷ modelo de RL entrenado (solo inferencia)

SALIDA:

accion	▷ índice de la acción seleccionada (0-3)
instruccion_texto	▷ instrucción legible para el módulo de salida

PROCESO

```
Construir el vector de estado
d_front ← distancia_libre(mapa_local, pose_actual, "frente")
d_left ← distancia_libre(mapa_local, pose_actual, ïzquierda")
d_right ← distancia_libre(mapa_local, pose_actual, "derecha")
phi_goal ← angulo_relativo(pose_actual, waypoint_global)
d_goal ← distancia_euclidiana(pose_actual, waypoint_global)
riesgos ← nivel_riesgo_maximo(obstaculos)

estado ← [d_front, d_left, d_right,
          phi_goal, d_goal,
          riesgos]

Evaluar la política del agente
accion ← modelo_rl.seleccionar_accion(estado)
    // El modelo devuelve el índice de la acción con mayor probabilidad

Traducir acción a instrucción de texto
```

SI accion == 0 **entonces:**
    instruccion_texto ← .ªvanzar"
Fin SI

SI accion == 1 **entonces:**
    instruccion_texto ← "Girar a la izquierda"
Fin SI

SI accion == 2 **entonces:**
    instruccion_texto ← "Girar a la derecha"
Fin SI

SI accion == 3 **entonces:**
    instruccion_texto ← "Detenerse"
Fin SI

RETORNAR accion, instruccion_texto

7.3.2. Submódulo de dirección

Descripción: Este submódulo traduce la acción producida por el modelo de navegación en una instrucción de movimiento concreta y comprensible para el usuario, incorporando referencias de distancia y ángulo cuando están disponibles. Se alimenta de la pose actual (SLAM) para contextualizar la instrucción dentro del recorrido total.

Entradas:

- Acción seleccionada por el modelo de RL
- Pose actual del usuario
- Waypoint actual y distancia restante

Salidas:

- Instrucción de navegación en texto natural (para el módulo de salida)

Pseudocódigo del submódulo de dirección

Algoritmo 5: Pseudocódigo del submódulo de dirección

ENTRADA:

accion ▷ índice de acción (0-3)

distancia_meta ▷ distancia al siguiente waypoint (metros)

angulo_giro ▷ ángulo de giro estimado (grados, si aplica)

SALIDA:

instruccion ▷ cadena de texto para síntesis de voz

PROCESO

```
Construir instrucción según la acción
SI accion == 0 entonces:
    instruccion ← .^vance - redondear(distancia_meta) + "metros"
Fin SI

SI accion == 1 entonces:
    SI angulo_giro <= 45 entonces:
        instruccion ← "Gire ligeramente a la izquierda"
    SINO
        instruccion ← "Gire a la izquierda - angulo_giro + "grados"
    Fin SI
Fin SI

SI accion == 2 entonces:
    SI angulo_giro <= 45 entonces:
        instruccion ← "Gire ligeramente a la derecha"
    SINO
        instruccion ← "Gire a la derecha - angulo_giro + "grados"
    Fin SI
Fin SI

SI accion == 3 entonces:
    instruccion ← "Deténgase"
Fin SI

SI distancia_meta <5 metros entonces:
```

23: agregar al inicio: .ᴱˢtá cerca de su destino. "
24: **Fin SI**

25: **RETORNAR** instruccion

7.3.3. Submódulo de MAPS

Descripción: Este submódulo gestiona la navegación global en entornos exteriores mediante la integración con el servicio de cartografía digital. Recibe un destino del módulo de interacción, obtiene la ruta global en forma de waypoints y los entrega al módulo de navegación local para guiar al usuario paso a paso.

Entradas:

- Destino del usuario (texto o coordenadas)
- Posición actual del usuario (coordenadas GPS)

Salidas:

- Lista de waypoints (coordenadas) de la ruta global
- Instrucciones textuales de alto nivel (calles, giros, distancias)

Nota de operación: Este submódulo solo opera cuando hay conexión a internet disponible. En ausencia de conexión, el sistema notifica al usuario y continúa operando con navegación local únicamente (RNF-13).

Pseudocódigo del submódulo de MAPS

Algoritmo 6: Pseudocódigo del submódulo de MAPS

ENTRADA:
destino_texto ▷ destino indicado por el usuario ("la farmacia", etc.)
posicion_actual ▷ coordenadas GPS actuales del dispositivo

SALIDA:
lista_waypoints ▷ coordenadas de la ruta
instrucciones_voz ▷ instrucciones de alto nivel en texto

PROCESO

```
Verificar disponibilidad de conexión a internet
SI sin_internet entonces:
    retornar error "Sin conexión: navegación solo en modo local"
Fin SI

Geocodificar el destino a coordenadas
coordenadas_destino ← geocodificar(destino_texto)
SI geocodificacion_fallida entonces:
    retornar error "Destino no encontrado"
Fin SI

Solicitar ruta peatonal al servicio de cartografía
respuesta ← servicio_mapas.calcular_ruta(
            origen = posicion_actual,
            destino = coordenadas_destino,
            modo = "peatonal")

Extraer waypoints e instrucciones de la respuesta
lista_waypoints ← extraer_coordenadas(respuesta)
instrucciones_voz ← extraer_instrucciones_texto(respuesta)

Publicar waypoints para el módulo de navegación local

RETORNAR lista_waypoints, instrucciones_voz
```

7.3.4. Módulo de alerta de obstáculos

Descripción: Este módulo actúa como una capa de seguridad independiente del modelo de navegación. Evalúa los obstáculos confirmados y, si alguno supera el umbral crítico de distancia, interrumpe la navegación con una alerta inmediata y notifica al módulo de navegación para recalcular la ruta. Su comportamiento es determinístico y conservador: ante cualquier duda, prioriza la seguridad del usuario.

Entradas:

- Lista de obstáculos confirmados (clase, distancia, riesgo)
- Posición actual del usuario

Salidas:

- Alerta auditiva con prioridad y descripción
- Señal al módulo de navegación para recalcular ruta

Pseudocódigo del módulo de alerta de obstáculos

Algoritmo 7: Pseudocódigo del módulo de alerta de obstáculos

ENTRADA:
obstaculos_confirmados ▷ lista de obstáculos validados
posicion_usuario ▷ pose actual

SALIDA:
alerta ▷ {texto, prioridad}
recalcular_ruta ▷ verdadero si se debe redirigir

PROCESO

```
recalcular_ruta ← falso
cola_alertas ← []

PARA cada obstaculo EN obstaculos_confirmados hacer:
    SI obstaculo.distancia <umbral_critico (ej. 80 cm) entonces:
        texto ← .Atención: - obstaculo.clase
         + .a - obstaculo.distancia + çm. Deténgase."
        prioridad ← alta
        recalcular_ruta ← verdadero
    SINO SI obstaculo.distancia <umbral_precaucion (ej. 150 cm) entonces:
        texto ← "Precaución: - obstaculo.clase + çerca"
        prioridad ← media
    SINO
        texto ← obstaculo.clase + "detectado, sin riesgo inmediato"
        prioridad ← baja
    Fin SI
```

15: agregar {texto, prioridad}
a cola_alertas
16: **Fin PARA**

17: Ordenar cola_alertas por prioridad (alta primero)

18: **SI** cola_alertas no está vacía **entonces:**
19: alerta ← cola_alertas[primera]
20: enviar alerta al módulo de salida
21: **Fin SI**

22: **RETORNAR** alerta, recalcular_ruta

7.4. Módulos de interacción

Los módulos de interacción constituyen la interfaz entre el usuario y el sistema. Toda la comunicación se realiza exclusivamente por voz, sin requerir interfaz visual. El flujo de estos módulos se ilustró en la Figura 6.5.

7.4.1. Módulo de reconocimiento de voz

Descripción: Este módulo captura la señal de audio del micrófono de forma continua, detecta los segmentos donde hay voz activa, aplica reducción de ruido sobre esos segmentos y los transcribe a texto mediante un motor de reconocimiento automático de voz local.

Entradas:

- Flujo de audio continuo del micrófono

Salidas:

- Texto transcrito del comando del usuario

Algoritmo 1: Detección de actividad de voz (VAD)

La detección de actividad de voz separa los segmentos de audio que contienen voz de los periodos de silencio o ruido de fondo, evitando procesar señal irrelevante. El algoritmo opera sobre bloques de 512 muestras y mantiene un umbral de energía doble: un umbral de inicio que activa la captura cuando se detecta voz, y un umbral de fin que espera una pausa máxima de 700 ms antes de cerrar el segmento. Mientras la energía del bloque supera el umbral de fin, el contador de silencio se reinicia, lo que permite capturar comandos con pausas internas sin partirlos en segmentos separados.

El pseudocódigo completo de este algoritmo se encuentra en el Apéndice A.4.

Algoritmo 2: Reducción de ruido (Filtro de Wiener)

Sobre cada segmento de voz detectado se aplica un filtro espectral que estima el ruido de fondo y lo elimina, mejorando la calidad de la señal antes de transcribirla.

La señal limpia se obtiene como $Y(f) = G(f) \cdot X(f)$ y se recupera en el dominio del tiempo mediante la transformada inversa de Fourier. La ganancia se acota a un mínimo de cero para evitar amplificar frecuencias donde el ruido supera a la señal útil. El ruido de fondo se estima a partir de los primeros 100 ms del buffer donde no hay voz activa, lo que permite adaptar el filtro a las condiciones acústicas del entorno en cada sesión.

El pseudocódigo y definiciónes matematicas completas de este algoritmo se encuentra en el Apéndice A.4.

Algoritmo 3: Transcripción de voz a texto (ASR)

Una vez limpio el segmento de audio, se entrega al motor de reconocimiento automático de voz (ASR), que opera de forma completamente local sin requerir conexión a internet, cumpliendo con RNF-13. El motor carga un modelo de idioma en español y procesa el audio devolviendo la transcripción con mayor probabilidad junto con su nivel de confianza. Si este nivel no alcanza el umbral mínimo configurado, el segmento se descarta y el sistema solicita al usuario que repita el comando.

Algoritmo 8: Pseudocódigo ASR

ENTRADA:

audio_limpio ▷ audio preprocesado y sin ruido

SALIDA:

texto ▷ transcripción del comando del usuario

PROCESO

Inicializar reconocedor local con modelo de idioma en español
reconocedor ← cargar_modelo_asr(."pañol")

Alimentar el audio al reconocedor
resultado ← reconocedor.procesar(audio_limpio)

Extraer el texto con mayor probabilidad
texto ← resultado.transcripcion_principal

SI resultado.confianza $<$ umbral_confianza_asr **entonces:**
    texto ← ▷ señal descartada, se solicitará repetición
Fin SI

RETORNAR texto

7.4.2. Módulo de PLN

Descripción: Este módulo recibe el texto transcrito y determina la intención del usuario. El sistema maneja un conjunto cerrado y predecible de intenciones: indicar un destino, controlar la sesión (pausar, reanudar, detener) y solicitar una ruta alternativa. Para cada intención reconocida genera la acción correspondiente y la envía al módulo del sistema que debe ejecutarla.

Entradas:

- Texto transcrito del comando del usuario

Salidas:

- Intención clasificada y entidades extraídas (ej. destino)
- Acción a ejecutar en el sistema

Procesamiento lingüístico: El texto se normaliza mediante tokenización y lematización para reducir la variabilidad morfológica: “llévame”, “ir a” y “quiero ir” deben mapearse a la misma

intención. La vectorización TF-IDF pondera cada término según su frecuencia en el comando y su rareza relativa en el corpus, dando más peso a las palabras con mayor poder discriminativo (verbos de acción, nombres de destino).

Pseudocódigo del módulo de PLN

Algoritmo 9: Pseudocódigo del módulo de PLN

ENTRADA:

texto_crudo ▷ texto transcrito por el módulo ASR

SALIDA:

intencion ▷ clase de intención ("destino", "pausa",
// reanudar", "detener", ruta_alternativa")

entidades ▷ información extraída (ej. nombre del destino)

accion_sistema ▷ señal al módulo correspondiente

PROCESO

Normalizar el texto

texto_normalizado ← convertir_a_minusculas(texto_crudo)

tokens ← tokenizar(texto_normalizado)

lemmas ← lematizar(tokens) ▷ "llévame>"llevar"

tokens_filtrados ← eliminar_palabras_vacias(lemmas)

Vectorizar con TF-IDF

vector_tfidf ← calcular_tfidf(tokens_filtrados, vocabulario_entrenado)

Clasificar la intención

intencion ← clasificador_intenciones.predecir(vector_tfidf)
// Clases posibles: "destino", "pausa", reanudar",
// "detener", ruta_alternativa", "desconocido"

Extraer entidades según la intención

SI intencion == "destino" **entonces:**

    entidades.destino ← extraer_lugar(tokens_filtrados)

Fin SI

SI intencion == "desconocido" **entonces:**

    accion_sistema ← solicitar_repeticion_al_usuario

```
        RETORNAR
    Fin SI

    Construir acción para el módulo correspondiente
    SI intencion == "destino" entonces:
        accion_sistema ← {tipo: "navegar", destino: entidades.destino}
    Fin SI

    SI intencion == "pausa" entonces:
        accion_sistema ← {tipo: "pausar_navegacion"}
    Fin SI

    SI intencion == reanudar" entonces:
        accion_sistema ← {tipo: reanudar_navegacion"}
    Fin SI

    SI intencion == "detener" entonces:
        accion_sistema ← {tipo: "detener_sistema"}
    Fin SI

    SI intencion == ruta_alternativa" entonces:
        accion_sistema ← {tipo: recalcular_ruta"}
    Fin SI

    RETORNAR intencion, entidades, accion_sistema
```

7.4.3. Módulo de salida

Descripción: Este módulo recibe instrucciones de texto provenientes del módulo de dirección y del módulo de alerta, las sintetiza en voz y las reproduce al usuario mediante el altavoz del dispositivo. Gestiona una cola de mensajes priorizada para garantizar que las alertas de seguridad siempre interrumpan o anteceden a las instrucciones de navegación.

Entradas:

- Instrucciones de navegación (texto, prioridad normal)
- Alertas de obstáculos (texto, prioridad alta)

Salidas:

- Audio sintetizado reproducido por el altavoz

Tecnología: pyttsx3 (motor TTS local, sin internet)

Pseudocódigo del módulo de salida

Algoritmo 10: Pseudocódigo del módulo de salida

INICIALIZACIÓN:
motor_tts ← inicializar_tts(velocidad=150, volumen=1.0)
cola_mensajes ← cola_priorizada() ▷ menor número = mayor urgencia

PROCESO (bucle continuo):
Recibir mensajes entrantes de otros módulos
SI llega alerta de obstáculo **entonces:**
    agregar a cola_mensajes con prioridad = 0 ▷ máxima
Fin SI

SI llega instruccion de navegación **entonces:**
    agregar a cola_mensajes con prioridad = 1
Fin SI

SI llega confirmación o respuesta del sistema **entonces:**
    agregar a cola_mensajes con prioridad = 2 ▷ mínima
Fin SI

SI cola_mensajes no está vacía **entonces:**
    mensaje ← extraer_primero(cola_mensajes) ▷ mayor urgencia primero
    SI motor_tts está hablando Y mensaje.prioridad == 0 **entonces:**
        interrumpir_reproduccion_actual()
    Fin SI
    motor_tts.sintetizar_y_reproducir(mensaje.texto)
Fin SI

Esperar hasta el próximo mensaje o fin de reproducción

FIN PROCESO

7.5. Algoritmo Main: Integración del sistema

El algoritmo principal orquesta la ejecución de todos los módulos en un ciclo continuo. Gestiona la inicialización del sistema, el flujo de datos entre módulos y los estados de la sesión de navegación (en espera, activo, pausado, finalizado). La Figura 6.6 ilustró las conexiones entre módulos.

Algoritmo 11: Algoritmo principal del sistema

INICIALIZACIÓN:
inicializar modulo_reconocimiento_voz
inicializar modulo_pln
inicializar modulo_salida
inicializar modulo_procesamiento_imagen
inicializar modulo_vision ▷ cargar modelo de deteccion
inicializar modulo_sensorial
inicializar modulo_slam ▷ pose=[0,0,0], reticula vacia
inicializar modulo_navegacion ▷ cargar modelo RL entrenado
inicializar modulo_maps
inicializar modulo_alertas

estado_sesion ← .en_espera"
waypoints ← []
tiempo_desde_ultimo_calculo ← 0
modulo_salida.reproducir("Sistema listo. Diga su destino.")

BUCLE PRINCIPAL (mientras sistema_encendido):
// — BLOQUE DE INTERACCION (siempre activo) —
texto ← modulo_reconocimiento_voz.escuchar()
SI texto no está vacío **entonces:**
    intencion, entidades, accion ← modulo_pln.procesar(texto)
    SI accion.tipo == "navegar"Y estado_sesion != .ªctivo" **entonces:**
        waypoints ← modulo_maps.obtener_ruta(
            entidades.destino,
            gps_actual())
        estado_sesion ← .ªctivo"
        modulo_salida.encolar(
            Ruta calculada. Iniciando navegación.")
    Fin SI

```
10:     SI accion.tipo == "pausar_navegacion" entonces:
11:         estado_sesion ← "pausado"
12:         modulo_salida.encolar("Navegación pausada.")
13:     Fin SI

14:     SI accion.tipo == reanudar_navegacion" entonces:
15:         estado_sesion ← .ªctivo"
16:         modulo_salida.encolar(Reanudando navegación.")
17:     Fin SI

18:     SI accion.tipo == "detener_sistema" entonces:
19:         estado_sesion ← .enespera"
20:         waypoints ← []
21:         modulo_salida.encolar("Navegación detenida.")
22:     Fin SI

23:     SI accion.tipo == recalcular_ruta" entonces:
24:         waypoints ← modulo_maps.obtener_ruta_alternativa(
                gps_actual(), waypoints.destino)
25:         modulo_salida.encolar(Çalculando ruta alternativa.")
26:     Fin SI
27: Fin SI

    // — BLOQUE DE PERCEPCION (siempre activo) —
28: fotograma ← camara.capturar()
29: imagen_lista ← modulo_procesamiento_imagen.procesar(fotograma)
30: detecciones ← modulo_vision.detectar(imagen_lista)
31: lecturas_sens ← sensores.leer()
32: obstaculos, alerta_critica ← modulo_sensorial.fusionar(
            detecciones, lecturas_sens)

    // — BLOQUE DE REPRESENTACION (siempre activo) —
33: odometria ← leer_odometria()
34: pose, reticula ← modulo_slam.actualizar(
            odometria, lecturas_sens, reticula)

    // — BLOQUE DE SEGURIDAD (siempre activo) —
35: alerta, recalcular ← modulo_alertas.evaluar(obstaculos, pose)
```

```
36: SI alerta no está vacía entonces:
37:     modulo_salida.encolar(alerta.texto, prioridad=0)
38: Fin SI

    // — BLOQUE DE NAVEGACION (solo cuando activo) —
39: SI estado_sesion == .ªctivo" entonces:
40:     SI recalcular O tiempo_desde_ultimo_calculo >10 segundos entonces:
41:         waypoint_actual ← waypoints.proximo(pose)
42:         accion_nav, instruccion ← modulo_navegacion.decidir(
                reticula, pose,
                waypoint_actual, obstaculos)
43:         tiempo_desde_ultimo_calculo ← 0
44:         modulo_salida.encolar(instruccion, prioridad=1)
45:     Fin SI

46:     SI distancia(pose, waypoints.destino_final) <umbral_llegada entonces:
47:         modulo_salida.encolar(
                "Ha llegado a su destino.", prioridad=1)
48:         estado_sesion ← .en_espera"
49:         waypoints ← []
50:     Fin SI
51: Fin SI

52: tiempo_desde_ultimo_calculo ← tiempo_desde_ultimo_calculo + dt

    FIN BUCLE
```

7.6. Electrónica

El sistema de navegación asistida se apoya en un conjunto de componentes electrónicos que constituyen la capa física de adquisición, procesamiento y salida de información. Su organización sigue la misma arquitectura modular del software: los periféricos de entrada capturan datos del entorno, la Raspberry Pi 4B los procesa y los periféricos de salida comunican el resultado al usuario [84]. Esta sección describe la función de cada componente, las interfaces que los conectan y las decisiones de diseño que explican su distribución en el prototipo.

La Figura 7.1 muestra el diagrama funcional de componentes, organizado en tres capas. En

la capa superior se encuentran los periféricos de entrada: los dos sensores ultrasónicos MB1232, posicionados a la izquierda y a la derecha del dispositivo para cubrir los laterales del usuario [44]; el módulo de cámara Raspberry Pi Camera Module 3 Wide, encargado de capturar el entorno en tiempo real para el módulo de visión artificial [52]; y el módulo GPS UBLOX NEO-6M, utilizado para obtener información de ubicación y apoyar la navegación global en exteriores [85].

La capa central está ocupada por la Raspberry Pi 4B, que funciona como la unidad principal de procesamiento e integra los distintos buses de comunicación del sistema. A través de I2C se realiza la lectura de los sensores ultrasónicos; mediante la interfaz CSI-2 se recibe el flujo de video proveniente de la cámara; y por UART se procesan los datos del módulo GPS [84]. Además, la Raspberry Pi administra el micrófono, la salida de audio y el botón de acción mediante sus puertos USB, GPIO y salida de audio integrada.

Debido a que algunos módulos de inteligencia artificial y visión por computadora pueden requerir una mayor capacidad de procesamiento, el sistema fue diseñado para permitir una arquitectura híbrida. En caso de que durante las pruebas la Raspberry Pi presente limitaciones de rendimiento, se aprovechará su conectividad Wi-Fi integrada para comunicarse con una computadora portátil que actuará como servidor externo. De esta manera, las tareas más demandantes, como ciertos procesos de inferencia o procesamiento visual avanzado, podrán ejecutarse remotamente sin modificar la lógica general del sistema ni la interacción del usuario [84].

En la capa inferior se ubican los periféricos de salida e interacción: el micrófono omnidireccional USB, que alimenta el sistema de reconocimiento de voz; el altavoz con amplificador PAM8403, encargado de reproducir instrucciones y alertas auditivas [86]; el botón de acción, que permite iniciar o alternar modos de navegación sin depender completamente de la voz; y la batería LiPo.

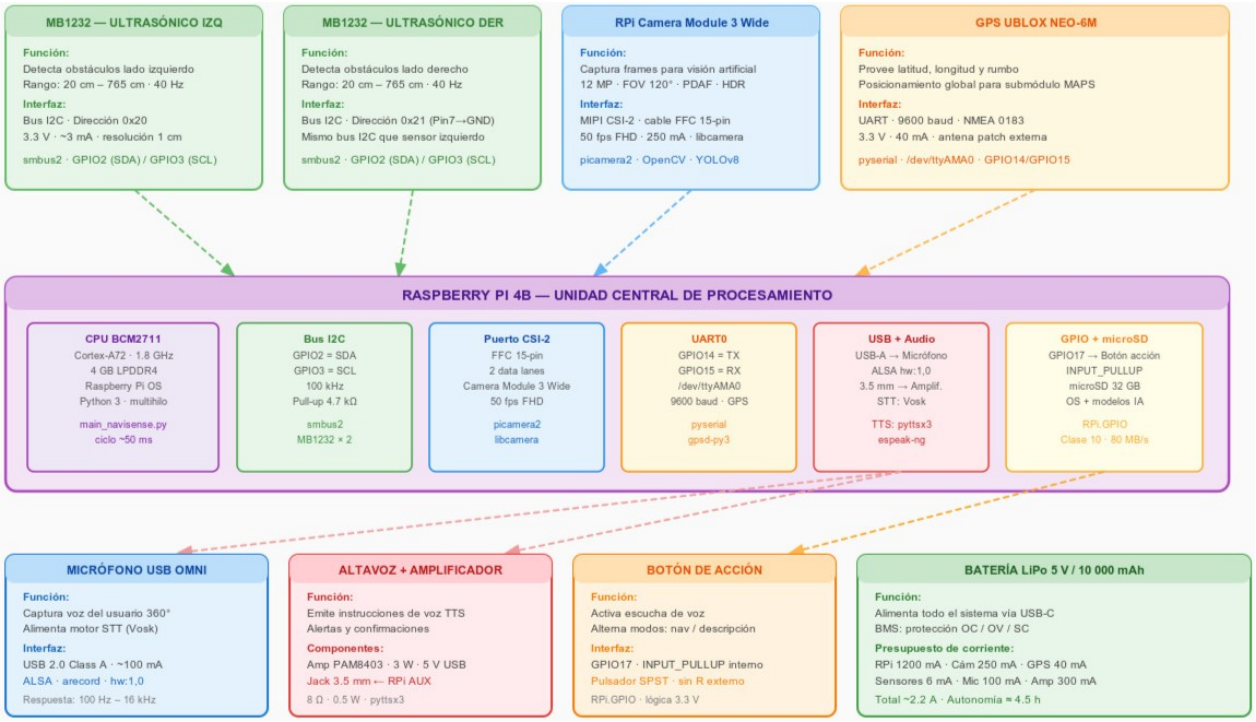


Figura 7.1: Diagrama funcional de componentes electrónicos del sistema

La Figura 7.2 muestra el esquemático de conexiones físicas del prototipo, donde se detallan los pines específicos de la Raspberry Pi implicados en cada interfaz.

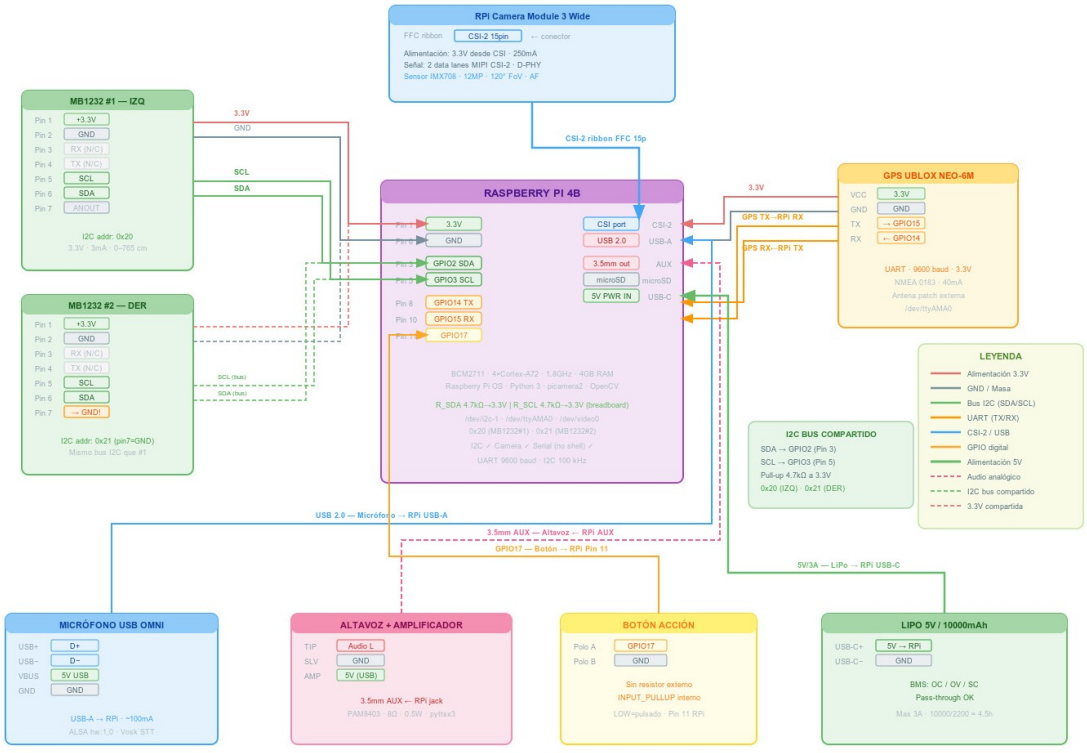


Figura 7.2: Esquemático de conexiones físicas del prototipo

7.6.1. Costos y viabilidad

El sistema está diseñado para ejecutarse sobre hardware embebido de bajo costo, siguiendo las restricciones establecidas en RNF-05 y RNF-10. La plataforma principal es una Raspberry Pi, la cual ofrece la capacidad de procesamiento suficiente para ejecutar la mayoría de los módulos del sistema bajo las latencias definidas en los requerimientos. Sin embargo, debido a que algunas tareas como visión por computadora o modelos de inteligencia artificial pueden demandar mayor capacidad de cómputo, durante las pruebas se evaluará el desempeño general del sistema. En caso de detectar limitaciones de rendimiento, se utilizará la conectividad Wi-Fi integrada de la Raspberry Pi para establecer comunicación con una computadora personal que funcionará como servidor de procesamiento, permitiendo ejecutar las tareas más pesadas de manera remota sin modificar el funcionamiento general del sistema.

Para la percepción visual se selecciona el **Raspberry Pi Camera Module 3**, que incorpora el sensor Sony IMX708 de 12 MP con tecnología CMOS *back-illuminated stacked*. Este módulo ofrece una buena calidad de imagen y resulta especialmente útil en aplicaciones de visión por computadora debido a su sistema de autoenfoque y su capacidad HDR, que ayudan a mantener imágenes más claras en diferentes condiciones de iluminación, como cambios entre interiores y exteriores [52]. La versión Wide, con un campo de visión diagonal de 120°, permite capturar una mayor parte del entorno alrededor del usuario, lo cual es ventajoso para navegación asistida y detección de obstáculos laterales. Además, su compatibilidad nativa con Raspberry Pi y la biblioteca `libcamera` facilita su integración con el sistema de procesamiento de imágenes [52].

Para la detección de proximidad lateral se utilizan **dos sensores MB1232**, colocados uno a cada lado del dispositivo. Estos sensores fueron seleccionados debido a que ofrecen lecturas más estables y confiables que alternativas más básicas como el HC-SR04, especialmente en entornos urbanos donde puede existir ruido acústico o eléctrico [44]. Ambos sensores comparten el bus I2C de la Raspberry Pi mediante la biblioteca `smbus2`, permitiendo una integración compacta y sencilla. Gracias a su alcance y a su ángulo de detección relativamente amplio, los sensores pueden detectar obstáculos cercanos alrededor del usuario y complementar la información obtenida por el módulo de visión. La detección frontal principal continúa siendo responsabilidad del sistema de visión artificial basado en YOLOv8, el cual analiza continuamente la imagen capturada por la cámara.

El sistema incorpora además un **botón para hablar**, que permite al usuario activar o reiniciar la navegación sin depender completamente del reconocimiento de voz. Esto resulta útil en ambientes con demasiado ruido o en situaciones donde el usuario necesite interactuar rápidamente con el sistema.

La Tabla 7.3 detalla los componentes de hardware seleccionados para el prototipo, junto con su función dentro del sistema, sus características principales y su costo aproximado.

Tabla 7.3: Componentes de hardware del prototipo con costo aproximado

Componente	**Función en el sistema**	**Especificación clave**	**Costo aprox.**
Raspberry Pi 4 Model B (4 GB RAM)	Unidad de procesamiento central. Ejecuta todos los módulos de software del sistema.	CPU quad-core Cortex-A72, 4 GB LPDDR4	$75 USD
Raspberry Pi Camera Module 3 Wide (IMX708)	Captura continua de fotogramas para los módulos de percepción visual.	12 MP, PDAF, HDR, FOV 120°, 50 fps FHD, CSI-2	$25 USD
Sensor MB1232 I2CXL-MaxSonar (izquierda)	Medición de distancia lateral izquierda. Dirección I2C `0x20`.	Rango 20 cm–765 cm, 1 cm res., I2C, 40 Hz	$30 USD
Sensor MB1232 I2CXL-MaxSonar (derecha)	Medición de distancia lateral derecha. Dirección I2C `0x21` (Pin 7 a GND).	Rango 20 cm–765 cm, 1 cm res., I2C, 40 Hz	$30 USD
Micrófono USB omnidireccional	Captura de audio para el módulo de reconocimiento de voz.	Respuesta 100 Hz – 16 kHz, UAC class	$15 USD
Altavoz + amplificador PAM8403 (3.5 mm)	Reproducción de instrucciones sintetizadas por el módulo de salida.	8 Ω, 0.5 W, conector AUX	$10 USD
Módulo GPS GNSS (UBLOX NEO-6M)	Posicionamiento global para el submódulo de MAPS en exteriores.	UART 9600 baud, precisión $\sim$5 m	$12 USD
Botón de acción momentáneo	Activación o reinicio de sesión de navegación por hardware (GPIO17, pull-up interno).	SPST, lógica 3.3 V	$2 USD

Batería LiPo 5 V 10000 mAh + BMS	Alimentación portátil del sistema completo vía USB-C. BMS con protección OC/OV/SC.	Corriente de salida máx. 3 A, pass-through	$25 USD
Tarjeta microSD (32 GB, clase 10)	Almacenamiento del sistema operativo y modelos de IA.	Velocidad mínima 80 MB/s	$10 USD
Resistencias pull-up 4.7 kΩ (×2)	Estabilización del bus I2C en líneas SDA y SCL.	3.3 V, tolerancia 1 %	$1 USD
Total estimado			**$235 USD**

La estimación de costo total del prototipo es de aproximadamente $235 USD, lo que lo sitúa en un rango accesible para un prototipo de investigación de asistencia tecnológica. El componente con mayor impacto en el costo son los sensores MB1232, cuyo precio unitario supera al de los sensores ultrasónicos básicos, pero que se justifica por sus capacidades de rechazo de ruido y calibración automática en tiempo real, necesarias para operar con confiabilidad en los entornos ruidosos que describe la restricción RNF-06 [44]. Los precios son estimaciones basadas en componentes disponibles comercialmente y pueden variar según el proveedor y la región.

Capítulo 8

Trabajo a futuro y Conclusiones

Durante TT2 se continuará con la fase de desarrollo e implementación del proyecto, tomando como base la investigación, documentación y análisis realizados previamente.

Como parte del trabajo futuro, se desarrollará la integración del sistema, basado en diagramas de arquitectura, diseño de módulos, flujo de procesamiento de datos e integración para la interacción entre componentes de hardware y software.

Se realizará la implementación progresiva de los módulos previamente investigados, integrando sistemas de visión computacional, reconocimiento de voz, navegación y generación de instrucciones auditivas dentro de una plataforma funcional.

Asimismo, se llevará a cabo el entrenamiento y ajuste de los modelos de inteligencia artificial mediante pruebas controladas y recopilación de datos en escenarios representativos, permitiendo evaluar el comportamiento del sistema ante distintos tipos de obstáculos y condiciones ambientales.

También se documentarán las pruebas experimentales realizadas al prototipo, registrando métricas de desempeño relacionadas con precisión de detección, tiempos de respuesta, estabilidad operativa y efectividad de navegación.

Durante esta etapa se buscará validar el funcionamiento integral del sistema mediante pruebas en entornos interiores y exteriores, analizando la capacidad del prototipo para asistir al usuario durante recorridos básicos de navegación.

Adicionalmente, se contempla continuar investigando estrategias de optimización orientadas a mejorar la eficiencia del procesamiento en hardware embebido, así como técnicas de navegación adaptativa y recalculación de rutas en tiempo real.

Como objetivo principal, se busca obtener un prototipo funcional y documentado que permita

validar la propuesta tecnológica planteada en la etapa de investigación, demostrando la factibilidad del sistema como herramienta de asistencia para personas con visión reducida.

8.1. Conclusiones de Trabajo Terminal I

A partir de la investigación y análisis realizados durante TT1, se determinó la viabilidad técnica del desarrollo de un sistema de navegación asistida para personas con visión reducida, integrando tecnologías de inteligencia artificial, visión computacional y procesamiento de voz.

La etapa de investigación permitió identificar los principales desafíos asociados a la movilidad autónoma asistida, así como las tecnologías más adecuadas para abordar problemas relacionados con detección de obstáculos, orientación espacial y emisión de instrucciones auditivas en tiempo real.

Se estableció una arquitectura modular para el sistema, permitiendo definir de manera estructurada cada uno de los componentes que conformarán el prototipo funcional en TT2. Esta arquitectura facilita la documentación, validación e integración progresiva de los módulos de hardware y software.

Asimismo, se definieron los requerimientos funcionales y no funcionales del sistema, considerando aspectos fundamentales como precisión, latencia, autonomía operativa y capacidad de procesamiento en una raspberry pi.

La investigación tecnológica realizada permitió justificar el uso de herramientas como YOLOv8 para detección de objetos, Vosk para reconocimiento de voz, spaCy para procesamiento de lenguaje natural y algoritmos de navegación basados en SLAM y aprendizaje profundo por refuerzo, debido a su compatibilidad con entornos de tiempo real y hardware de recursos limitados.

De igual manera, se documentaron los flujos operativos, casos de uso, interacciones del sistema y posibles escenarios de funcionamiento, estableciendo una base sólida para el desarrollo e implementación del prototipo durante la siguiente etapa del proyecto terminal.

Finalmente, el proyecto se sustenta en una problemática social relevante, orientada a generar herramientas tecnológicas de apoyo que contribuyan a mejorar la autonomía y seguridad de las personas con discapacidad visual durante sus desplazamientos.

Referencias

[1] World Health Organization. (2023) Ceguera y discapacidad visual. Consultado: 7 de mayo de 2025. [Online]. Available: https://www.who.int/es/news-room/fact-sheets/detail/blindness-and-visual-impairment

[2] E. Covarrubias. (2025, Apr.) Desafíos diarios de la gente con discapacidad visual. Blog de Psicología. Publicado: 24 de abril de 2025. Consultado: 7 de mayo de 2025. [Online]. Available: https://enricova.com/blog/desafios/

[3] The Poona School and Home for the Blind. (2025) Mobility and orientation for the visually impaired. Consultado: 7 de mayo de 2025. [Online]. Available: https://www.blindschoolpune.org

[4] Instituto Politécnico Nacional. (2020) Con tecnología politécnica se mejorará la seguridad y movilidad de invidentes. Comunicado de prensa. Consultado: 7 de mayo de 2025. [Online]. Available: https://www.ipn.mx/comunicados/ver-comunicado.html?n=4535

[5] AhorraSeguros.mx. (2025, Jan.) Primera guía robot creada para invidentes por estudiantes del ipn. AhorraSeguros Blog. Publicado: 27-ene-2025. Consultado: 7 de mayo de 2025. [Online]. Available: https://www.ahorraseguros.mx/noticias/guia-robot-ipn-discapacidad-visual/

[6] Meta. (2024) Introducing the ray-ban meta smart glasses collection. Consultado: febrero de 2026. [Online]. Available: https://about.fb.com/news/2023/09/new-ray-ban-meta-smart-glasses/

[7] E. Bayro-Corrochano. (2019, Nov.) Diseña cinvestav lentes inteligentes guía para discapacitados visuales. Conexión Cinvestav. Publicado: 28-nov-2019. Consultado: 7 de mayo de 2025. [Online]. Available: https://conexion.cinvestav.mx/Notas/Noticia/Lentes-inteligentes-CINVESTAV

[8] CIT UPC. (2024, Jul.) Estudio sobre el uso de gafas inteligentes para personas con discapacidad visual. Blog CIT UPC. Publicado: 26-jul-2024. Consultado: 7 de mayo de 2025. [Online]. Available: https://blog.cit.upc.edu/bcnmob-gafas-discapacidad

[9] A. Abdelhady, A. Alsulami, D. Mashburn, and J. Dossantos. (2023, Jan.) Eye in ai: Smart glasses for blind. MIT Solve: Youth Innovation Challenge. Publicado: 23-ene-2023. Consultado: 7 de mayo de 2025. [Online]. Available: https://solve.mit.edu/challenges/youth-innovation/solutions/70767

[10] Lazarillo. (2023) La app lazarillo: navegación accesible al alcance de tu mano. Consultado: 7 de mayo de 2025. [Online]. Available: https://www.lazarillo.app/es

[11] M. Bordonaba. (2025, Apr.) Adiós al bastón: investigadores desarrollan un navegador con gafas de realidad virtual para personas ciegas. 20 Minutos. Publicado: 17-abr-2025. Consultado: 7 de mayo de 2025. [Online]. Available: https://www.20minutos.es/tecnologia/actualidad/navegador-para-ciegos-con-realidad-virtual-5221278/

[12] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[13] K. Ganesan. (2025, sep) Common ai problem types — cheat sheet. kavita.dev. Consultado: 30 Sep. 2025. [Online]. Available: https://kavita--ganesan-com.translate.goog/common-ai-problem-types-cheat-sheet/?_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_pto=tc

[14] IBM. (2025, sep) Comprenda los diferentes tipos de inteligencia artificial. IBM. Consultado: 30 Sep. 2025. [Online]. Available: https://www.ibm.com/mx-es/think/topics/artificial-intelligence-types

[15] Google Cloud. (2025, sep) ¿qué es la inteligencia artificial o ia? Google Cloud. Consultado: 30 Sep. 2025. [Online]. Available: https://cloud.google.com/learn/what-is-artificial-intelligence?hl=es-419

[16] NASA. (2025, sep) What is artificial intelligence? NASA. Consultado: 30 Sep. 2025. [Online]. Available: https://www.nasa.gov/what-is-artificial-intelligence/

[17] P. Stone, R. Brooks, E. Brynjolfsson, R. Calo, O. Etzioni, G. Hager, J. Hirschberg, S. Kalyanakrishnan, E. Kamar, S. Kraus, K. Leyton-Brown, D. Parkes, W. Press, A. Saxenian, J. Shah, M. Tambe, and A. Teller, "Artificial intelligence and life in 2030. One Hundred Year Study on Artificial Intelligence: Report of the 2015–2016 study panel," Stanford University, Tech. Rep., 2016. [Online]. Available: https://ai100.stanford.edu/2016-report

[18] J. McCarthy, M. L. Minsky, N. Rochester, and C. E. Shannon, "A proposal for the Dartmouth summer research project on artificial intelligence," *AI Magazine*, vol. 27, no. 4, pp. 12–14, 2006, reproducción del documento original de agosto de 1955.

[19] Amazon Web Services, Inc. (2025, sep) ¿qué es la inteligencia artificial? Amazon Web Services. Consultado: 30 Sep. 2025. [Online]. Available: https://aws.amazon.com/es/what-is/artificial-intelligence/

[20] Association for the Advancement of Artificial Intelligence. (2024) About AAAI. Consultado: abril de 2026. [Online]. Available: https://aaai.org/about/

[21] European Commission, High-Level Expert Group on Artificial Intelligence, "A definition of AI: Main capabilities and scientific disciplines," European Commission, Tech. Rep., 4 2019, documento de definición elaborado por el AI HLEG. Consultado: abril de 2026. [Online]. Available: https://ec.europa.eu/futurium/en/system/files/ged/ai_hleg_definition_of_ai_18_december_1.pdf

[22] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. Cambridge, Massachusetts: MIT Press, 2005.

[23] D. Fox, W. Burgard, and S. Thrun, "The Dynamic Window Approach to Collision Avoidance," *IEEE Robotics & Automation Magazine*, vol. 4, no. 1, pp. 23–33, 1997.

[24] A. Elfes, "Using Occupancy Grids for Mobile Robot Perception and Navigation," *Computer*, vol. 22, no. 6, pp. 46–57, 1989.

[25] S. M. LaValle, *Planning Algorithms*. Cambridge, UK: Cambridge University Press, 2006, disponible en: http://planning.cs.uiuc.edu/.

[26] E. W. Dijkstra, "A Note on Two Problems in Connexion with Graphs," *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, 1959.

[27] P. E. Hart, N. J. Nilsson, and B. Raphael, "A Formal Basis for the Heuristic Determination of Minimum Cost Paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.

[28] IBM, "Modelos de IA," https://www.ibm.com/mx-es/think/topics/ai-model, 2025, accessed: 2025-09-30.

[29] ——, "Tipos de machine learning," https://www.ibm.com/mx-es/think/topics/machine-learning-types, 2025, consultado: 30-sep-2025.

[30] IBM. (2024) What is computer vision? Consultado: abril de 2026. [Online]. Available: https://www.ibm.com/think/topics/computer-vision

[31] EDS Robotics. (2024) Visión por computador: Qué es, aplicaciones y objetivos. Consultado: abril de 2026. [Online]. Available: https://www.edsrobotics.com/blog/vision-computador-que-es/

[32] F. Vivas. (2024) Guía básica de visión por computador para principiantes en ia. Consultado: abril de 2026. [Online]. Available: https://fvivas.com/es/guia-basica-de-vision-computacional-ia/

[33] GeeksforGeeks. (2024) What is object detection in computer vision? Consultado: abril de 2026. [Online]. Available: https://www.geeksforgeeks.org/computer-vision/what-is-object-detection-in-computer-vision/

[34] Amazon Web Services. (2024) ¿qué es la visión artificial? Consultado: abril de 2026. [Online]. Available: https://aws.amazon.com/es/what-is/computer-vision/

[35] Ultralytics. (2024) Ia de visión en robótica: Casos de uso y ventajas. Consultado: abril de 2026. [Online]. Available: https://www.ultralytics.com/es/blog/understanding-the-integration-of-computer-vision-in-robotics

[36] UnitX Labs. (2024) Explicación de los sistemas de visión artificial para el procesamiento de imágenes. Consultado: abril de 2026. [Online]. Available: https://es.unitxlabs.com/image-processing-machine-vision-systems/

[37] Varios autores, "Assistive systems for visually impaired persons: Challenges and opportunities for navigation assistance," *PubMed Central (PMC)*, 2024, consultado: abril de 2026. [Online]. Available: https://pmc.ncbi.nlm.nih.gov/articles/PMC11175312/

[38] UnitX Labs. (2024) Posprocesamiento vs. preprocesamiento de imágenes en visión artificial. Consultado: abril de 2026. [Online]. Available: https://www.unitxlabs.com/es/image-post-processing-machine-vision-system-vs-preprocessing/

[39] ——. (2024) Cómo la extracción de características potencia los sistemas de visión artificial. Consultado: abril de 2026. [Online]. Available: https://www.unitxlabs.com/es/feature-extraction-machine-vision-systems/

[40] P. Viola and M. Jones, "Rapid object detection using a boosted cascade of simple features," in *Proceedings of the 2001 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'01)*, vol. 1. IEEE, 2001, pp. I–511–I–518.

[41] N. Dalal and B. Triggs, "Histograms of oriented gradients for human detection," in *2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'05)*, vol. 1. IEEE, 2005, pp. 886–893.

[42] MRDVS. (2024) Top cameras for robots: Enhancing vision and obstacle avoidance. Consultado: abril de 2026. [Online]. Available: https://mrdvs.com/top-cameras-for-robots-enhancing-vision-and-obstacle-avoidance/

[43] Elecfreaks, *Ultrasonic Ranging Module HC-SR04: Product Features and Technical Specifications*, https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf, 2014, consultado: abril de 2026].

[44] MaxBotix Inc., *I2CXL-MaxSonar-EZ Series High Performance Sonar Rangefinder Datasheet*, MaxBotix Inc., 2023, accessed: 2025. [Online]. Available: https://www.maxbotix.com/documents/I2CXL-MaxSonar-EZ_Datasheet.pdf

[45] STMicroelectronics. (2024) VL53L1X – Time-of-Flight Long-Distance Ranging Sensor. https://www.st.com/en/imaging-and-photonics-solutions/vl53l1x.html. Consultado: abril de 2026].

[46] ——, *VL53L0X: World's Smallest Time-of-Flight Ranging and Gesture Detection Sensor. Datasheet*, https://www.st.com/resource/en/datasheet/vl53l0x.pdf, 2023, consultado: abril de 2026].

[47] ——, *VL53L1X: A New Generation, Long Distance Ranging Time-of-Flight Sensor Based on ST's FlightSense Technology. Datasheet*, https://www.st.com/resource/en/datasheet/vl53l1x.pdf, 2024, consultado: abril de 2026].

[48] Benewake (Beijing) Co., Ltd., *Product Manual of TFmini Mini LiDAR Module*, https://cdn-shop.adafruit.com/product-files/3978/3978_manual_SJ-PM-TFmini-T-01_A03ProductManual_EN.pdf, 2018, consultado: abril de 2026].

[49] Garmin Ltd., *LIDAR-Lite v3: Operation Manual and Technical Specifications*, https://static.garmin.com/pumac/LIDAR_Lite_v3_Operation_Manual_and_Technical_Specifications.pdf, 2016, consultado: abril de 2026].

[50] Raspberry Pi Ltd. (2024) Camera – Raspberry Pi Documentation. https://www.raspberrypi.com/documentation/accessories/camera.html. Consultado: abril de 2026].

[51] Arducam Technology Co., Ltd. (2024) 8MP IMX219 – Arducam Wiki. https://docs.arducam.com/Raspberry-Pi-Camera/Native-camera/8MP-IMX219/. Consultado: abril de 2026].

[52] Raspberry Pi Ltd., *Raspberry Pi Camera Module 3 Product Brief*, Raspberry Pi Ltd., 2023, accessed: 2025. [Online]. Available: https://datasheets.raspberrypi.com/camera/camera-module-3-product-brief.pdf

[53] OmniVision Technologies, Inc., *OV2640 Color CMOS UXGA (2.0 MegaPixel) CAMERACHIP™ with OmniPixel2™ Technology. Datasheet*, https://www.uctronics.com/download/cam_module/OV2640DS.pdf, 2006, consultado: abril de 2026].

[54] Meta Platforms, Inc. and EssilorLuxottica. (2023) Discover Ray-Ban Meta AI Glasses: Specs & Features. https://www.ray-ban.com/usa/l/discover-ray-ban-meta-ai-glasses. Consultado: abril de 2026].

[55] Organización Mundial de la Salud. (2026, Feb.) Ceguera y discapacidad visual. Publicado: 10-feb-2026. Consultado: abril de 2026. [Online]. Available: https://www.who.int/es/news-room/fact-sheets/detail/blindness-and-visual-impairment

[56] World Health Organization, *International Classification of Functioning, Disability and Health (ICF)*. Geneva: World Health Organization, 2001. [Online]. Available: https://www.who.int/classifications/icf/en/

[57] Fundación ONCE Baja Visión. (2025) La baja visión: Síntomas, tipos, profesionales y prevención. Consultado: abril de 2026. [Online]. Available: https://fundaciononcebajavision.es/la-baja-vision

[58] K. e. a. Lee, "Beyond the cane: Describing urban scenes to blind people for mobility tasks," *PubMed Central (PMC)*, 2022, consultado: abril de 2026. [Online]. Available: https://pmc.ncbi.nlm.nih.gov/articles/PMC9491388/

[59] Nature Publishing Group. (2025, Apr.) Wearable ai system helps blind people navigate. Publicado: 14-abr-2025. Consultado: abril de 2026. [Online]. Available: https://techxplore.com/news/2025-04-wearable-ai-people.html

[60] M. Elhashash, H. Albanwan, and R. Qin, "A review of mobile mapping systems: From sensors to applications," *arXiv preprint arXiv:2205.15865*, 2022. [Online]. Available: https://arxiv.org/abs/2205.15865

[61] M. S. Grewal, L. R. Weill, and A. P. Andrews, *Global Positioning Systems, Inertial Navigation, and Integration*, 2nd ed. John Wiley & Sons, 2007.

[62] M. Haklay and P. Weber, "OpenStreetMap: User-generated street maps," *IEEE Pervasive Computing*, vol. 7, no. 4, pp. 12–18, 2008.

[63] M. Shan, F. Wang, F. Lin, Z. Gao, Y. Z. Tang, and B. M. Chen, "Google map aided visual navigation for uavs in gps-denied environment," *arXiv preprint arXiv:1703.10125*, 2017. [Online]. Available: https://arxiv.org/abs/1703.10125

[64] Google Developers. (2024) Directions api (legacy) overview. Consultado: abril de 2026. [Online]. Available: https://developers.google.com/maps/documentation/directions/overview

[65] J. Farman, "Mapping the digital empire: Google earth and the process of postmodern cartography," *London Review of Education*, vol. 8, no. 2, pp. 123–134, 2010. [Online]. Available: https://journals.uclpress.co.uk/ucloe/article/id/1063/

[66] Zbotic. (2024) Gps vs glonass vs galileo: Which satellite system is best? Consultado: abril de 2026. [Online]. Available: https://zbotic.in/gps-vs-glonass-vs-galileo-which-satellite-system-is-best/

[67] OpenStreetMap. (2026) About openstreetmap. [Online]. Available: https://www.openstreetmap.org/about

[68] D. Luxen and C. Vetter, "Real-time routing with openstreetmap data," in *Proceedings of the 19th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems*, 2011, pp. 513–516.

[69] A. Aladrén, G. López-Nicolás, L. Puig, and J. J. Guerrero, "Navigation assistance for the visually impaired using rgb-d sensor with range expansion," *IEEE Systems Journal*, vol. 10, no. 3, pp. 922–932, 2016.

[70] Esri India. (2024) What is gis? (geographic information system). Consultado: abril de 2026. [Online]. Available: https://www.esri.in/en-in/what-is-gis/overview

[71] Geoapify. (2024) Raster tile vs vector tile: Pros and cons of both map tile types. Consultado: abril de 2026. [Online]. Available: https://www.geoapify.com/raster-vs-vector-map-tiles/

[72] MobilityData. (2024) General Transit Feed Specification (GTFS) — overview and reference. Consultado: abril de 2026. [Online]. Available: https://gtfs.org/documentation/overview/

[73] Stack Overflow community. (2021) What is the difference between ray tracing, ray casting, ray marching, and path tracing? https://stackoverflow.com/questions/67347177/what-is-the-difference-between-ray-tracing-ray-casting-ray-marching-and-path-t. Consultado: abril de 2026.

[74] F. Sanglard. (2019, 6) How *id Software* built Wolfenstein 3D using Commander Keen tech. Game Developer. Consultado: abril de 2026. [Online]. Available: https://www.gamedeveloper.com/programming/how-id-built-i-wolfenstein-3d-i-using-i-commander-keen-i-tech

[75] A. Elfes, "Using Occupancy Grids for Mobile Robot Perception and Navigation," *Computer*, vol. 22, no. 6, pp. 46–57, 1989.

[76] J. E. Bresenham, "Algorithm for Computer Control of a Digital Plotter," *IBM Systems Journal*, vol. 4, no. 1, pp. 25–30, 1965.

[77] A. Watt, *3D Computer Graphics*, 3rd ed. Addison-Wesley, 2000.

[78] H. Durrant-Whyte and T. Bailey, “Simultaneous localisation and mapping (SLAM): Part I the essential algorithms,” *IEEE Robotics & Automation Magazine*, vol. 13, no. 2, pp. 99–110, 2006.

[79] Zbotic. (2024) GPS vs GLONASS vs Galileo: Which Satellite System Is Best? https://zbotic.in/gps-vs-glonass-vs-galileo-which-satellite-system-is-best/. Consultado: abril de 2026].

[80] R. Hernández Sampieri, C. Fernández Collado, and P. Baptista Lucio, *Metodología de la investigación*, 6th ed. México: McGraw-Hill, 2014, consultado: abril de 2026.

[81] I. Sommerville, *Ingeniería del software*, 9th ed. Madrid: Pearson Educación, 2011, consultado: abril de 2026.

[82] R. S. Pressman and B. R. Maxim, *Ingeniería del software: un enfoque práctico*, 8th ed. México: McGraw-Hill, 2014, consultado: abril de 2026.

[83] L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*, 3rd ed. Upper Saddle River, NJ: Addison-Wesley, 2012, consultado: abril de 2026.

[84] Raspberry Pi Foundation, *Raspberry Pi 4 Model B Datasheet*, 2023. [Online]. Available: https://datasheets.raspberrypi.com/rpi4/raspberry-pi-4-datasheet.pdf

[85] u-blox, *NEO-6 u-blox 6 GPS Modules Data Sheet*, 2011. [Online]. Available: https://content.u-blox.com/sites/default/files/products/documents/NEO-6_DataSheet_%28GPS.G6-HW-09005%29.pdf

[86] Power Analog Microelectronics, *PAM8403 3W Class-D Stereo Audio Amplifier Datasheet*, 2014. [Online]. Available: https://components101.com/sites/default/files/component_datasheet/PAM8403-Datasheet.pdf

Apéndice A

Pseudocódigos auxiliares

A.1. Pseudocódigo del módulo de Ray Casting

Algoritmo 12: Pseudocódigo del módulo de Ray Casting

ENTRADA:

pose ▷ [x, y, theta] del usuario
angulos_rayos ▷ lista de ángulos relativos a theta
distancias ▷ distancia de impacto para cada rayo
// Variables a inicializar
reticula ▷ mapa de ocupación en log-odds
resolucion ▷ tamaño de cada celda en metros
incremento_libre ▷ incremento log-odds para celda libre (negativo)
incremento_ocup ▷ incremento log-odds para celda ocupada (positivo)
lo_min, lo_max ▷ límites de saturación

SALIDA:

1: reticula ▷ mapa actualizado

PROCESO

2: **PARA** cada rayo i EN angulos_rayos **hacer:**
3: Calcular dirección absoluta del rayo
4: angulo_abs ← pose.theta + angulos_rayos[i]
5: distancia_impacto ← distancias[i]

6: Obtener secuencia de celdas con algoritmo de Bresenham

```
    celda_origen ← convertir_a_celda(pose.x, pose.y, resolucion)
    celda_fin ← convertir_a_celda(
          pose.x + distancia_impacto * coseno(angulo_abs),
          pose.y + distancia_impacto * seno(angulo_abs),
          resolucion)
    celdas_rayo ← bresenham(celda_origen, celda_fin)

    Marcar todas las celdas antes del impacto como libres
    PARA cada celda c EN celdas_rayo excepto la última hacer:
        reticula[c] ← saturar(reticula[c] + incremento_libre,
              lo_min, lo_max)
    Fin PARA

    Si hubo impacto (distancia menor a la máxima), marcar la celda:
    SI distancia_impacto <distancia_maxima entonces:
        celda_impacto ← celdas_rayo[última]
        reticula[celda_impacto] ← saturar(
              reticula[celda_impacto] + incremento_ocup,
              lo_min, lo_max)
    Fin SI
Fin PARA

RETORNAR reticula
```

Algoritmo 13: Pseudocódigo de representación

A.2. Pseudocódigo integrado de los módulos de representación

El siguiente pseudocódigo describe el ciclo completo de operación de los módulos de representación, integrando SLAM y Ray Casting en un bucle continuo que corre mientras el sistema está activo.

Algoritmo 13: Pseudocódigo de representación

A.3. Pseudocódigo integrado de los módulos de representación

El siguiente pseudocódigo describe el ciclo completo de operación de los módulos de representación, integrando SLAM y Ray Casting en un bucle continuo que corre mientras el sistema está

activo.

Algoritmo 14: Pseudocódigo de representación

```
INICIALIZACIÓN:
pose ← [0, 0, 0]                                   ▷ posición y orientación inicial
covarianza ← matriz_identidad * varianza_inicial
reticula ← inicializar_con_log_odds(0)
                                                   ▷ todo desconocido

MIENTRAS sistema_activo hacer:
    // Adquisición de datos del ciclo actual
    odometria ← leer_odometria()
                                                   ▷ (delta_d, delta_theta)
    mediciones ← leer_sensores()
                                                   ▷ distancias por ángulo

    // Predicción de pose
    pose_pred ← aplicar_modelo_movimiento(pose, odometria)
    F ← jacobiano_movimiento(pose, odometria)
    covarianza_pred ← F * covarianza * F^T + Q

    // Corrección con mediciones
    medicion_esperada ← modelo_observacion(pose_pred, reticula)
    innovacion ← mediciones - medicion_esperada
    H ← jacobiano_observacion(pose_pred)
    S ← H * covarianza_pred * H^T + R
    K ← covarianza_pred * H^T * inversa(S)
    pose ← pose_pred + K * innovacion
    covarianza ← (I - K * H) * covarianza_pred

    // Actualización del mapa con Ray Casting
    PARA cada rayo i hacer:
        celdas ← bresenham(pose, angulo[i], mediciones[i], resolucion)
        PARA c EN celdas excepto la última hacer:
            reticula[c] ← saturar(reticula[c] + lo_libre, lo_min, lo_max)
        Fin PARA
        SI mediciones[i] <distancia_maxima entonces:
            reticula[celdas[última]] ← saturar(
```

reticula[celdas[última]] + lo_ocupado,
lo_min, lo_max)
Fin SI
Fin PARA

// Publicar resultados para los demás módulos
publicar(pose)
publicar(reticula)
Fin MIENTRAS

A.4. Pseudocódigos Voz

Algoritmo 1: Detección de actividad de voz (VAD)

La detección de actividad de voz separa los segmentos de audio que contienen voz de los periodos de silencio o ruido de fondo, evitando procesar señal irrelevante.

Algoritmo 15: Pseudocódigo VAD

ENTRADA:
flujo_audio ▷ flujo continuo de muestras de audio

SALIDA:
segmentos_voz ▷ lista de fragmentos de audio con voz activa

PROCESO
Inicializar:
buffer_acumulado ← vacío
segmentos_voz ← []
en_voz ← falso
silencio_acumulado ← 0 ms

PARA cada bloque de 512 muestras EN flujo_audio **hacer:**
Calcular energía del bloque
energia ← promedio(cuadrado(muestras))

SI energia > umbral_inicio Y NO en_voz **entonces:**

```
        en_voz ← verdadero
        inicio_segmento ← tiempo_actual()
        buffer_acumulado ← [bloque]
        silencio_acumulado ← 0

    SINO SI en_voz entonces:
        agregar bloque a buffer_acumulado
        SI energia < umbral_fin entonces:
            silencio_acumulado ← silencio_acumulado + duracion(bloque)
            SI silencio_acumulado > pausa_maxima (700 ms) entonces:
                agregar buffer_acumulado a segmentos_voz
                en_voz ← falso
                silencio_acumulado ← 0
                buffer_acumulado ← vacío
            Fin SI
        SINO
            silencio_acumulado ← 0                ▷ la voz continuó, reinciar pausa
        Fin SI

    Fin SI

Fin PARA

RETORNAR segmentos_voz
```

Algoritmo 2: Reducción de ruido (Filtro de Wiener)

Sobre cada segmento de voz detectado se aplica un filtro espectral que estima el ruido de fondo y lo elimina, mejorando la calidad de la señal antes de transcribirla.

Sea $X(f)$ el espectro del audio capturado, $N(f)$ el espectro estimado del ruido de fondo y $G(f)$ la ganancia de Wiener aplicada a cada frecuencia f:

$$G(f) = \frac{|X(f)|^2 - |N(f)|^2}{|X(f)|^2}$$

La señal limpia se obtiene como:

$$Y(f) = G(f) \cdot X(f)$$

y se recupera mediante la transformada inversa.

Algoritmo 16: Pseudocódigo reducción de ruido

ENTRADA:
segmento_audio ▷ fragmento de audio con voz

SALIDA:
audio_limpio ▷ audio con ruido reducido

PROCESO

Calcular el espectro de frecuencias del segmento
espectro ← transformada_fourier(segmento_audio)

Estimar el espectro del ruido de fondo ▷ usando los primeros 100 ms sin voz activa
ruido_estimado ← estimar_ruido_fondo(espectro)

PARA cada frecuencia f EN espectro **hacer:**
    potencia_señal ← $|espectro[f]|^2$
    potencia_ruido ← $|ruido_estimado[f]|^2$
    // Calcular ganancia de Wiener (mínimo 0 para evitar amplificación)
    ganancia[f] ← maximo(0,
            (potencia_señal - potencia_ruido) / potencia_señal)
    espectro_filtrado[f] ← ganancia[f] * espectro[f]
Fin PARA

Reconstruir señal en el dominio del tiempo
audio_limpio ← transformada_fourier_inversa(espectro_filtrado)

RETORNAR audio_limpio